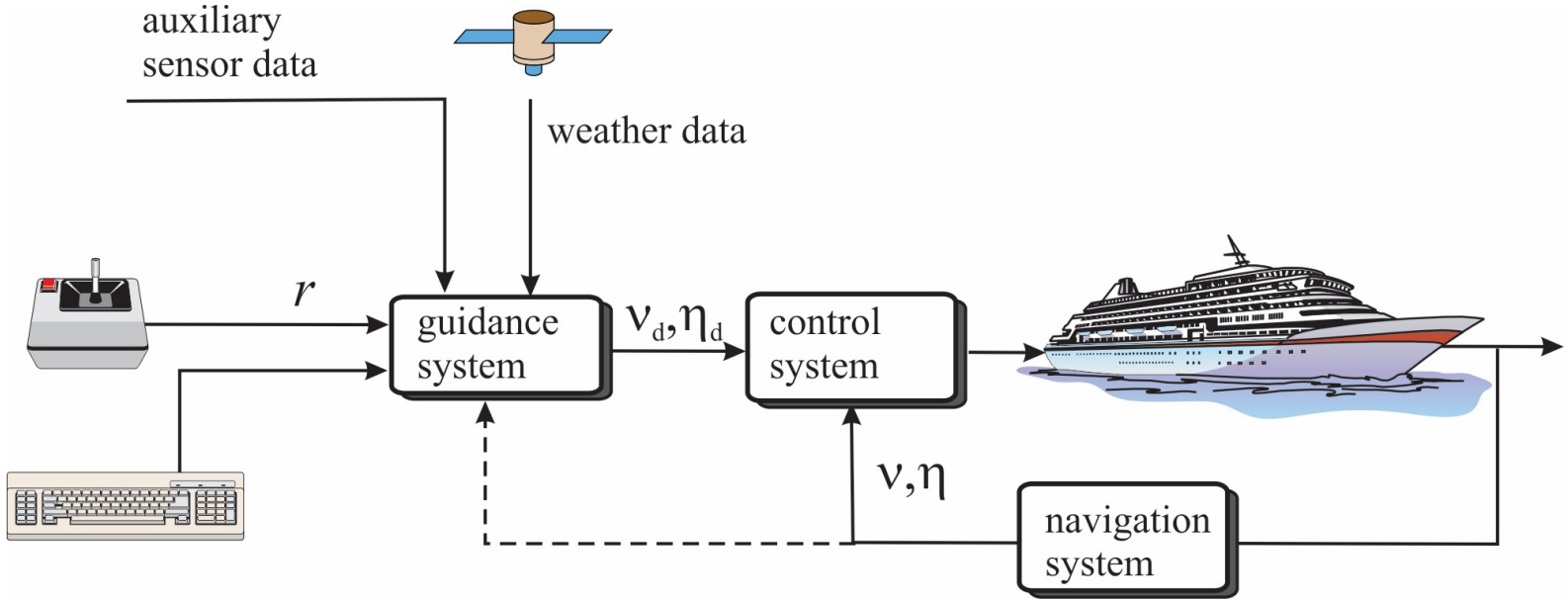
Chapter 12 – Guidance Systems

12.1 Trajectory Tracking
12.2 Guidance laws for Target Tracking
12.3 Linear Designs Methods for Path Following
12.4 LOS Guidance Laws for Path Following using Course Autopilots
12.5 LOS Guidance Laws for Path Following using Heading Autopilots
12.6 Curved-Path Path Following



The guidance system answers the question: "Where do you want to go?" by providing the reference position, path, or trajectory defined by the mission plan.

Chapter Goals

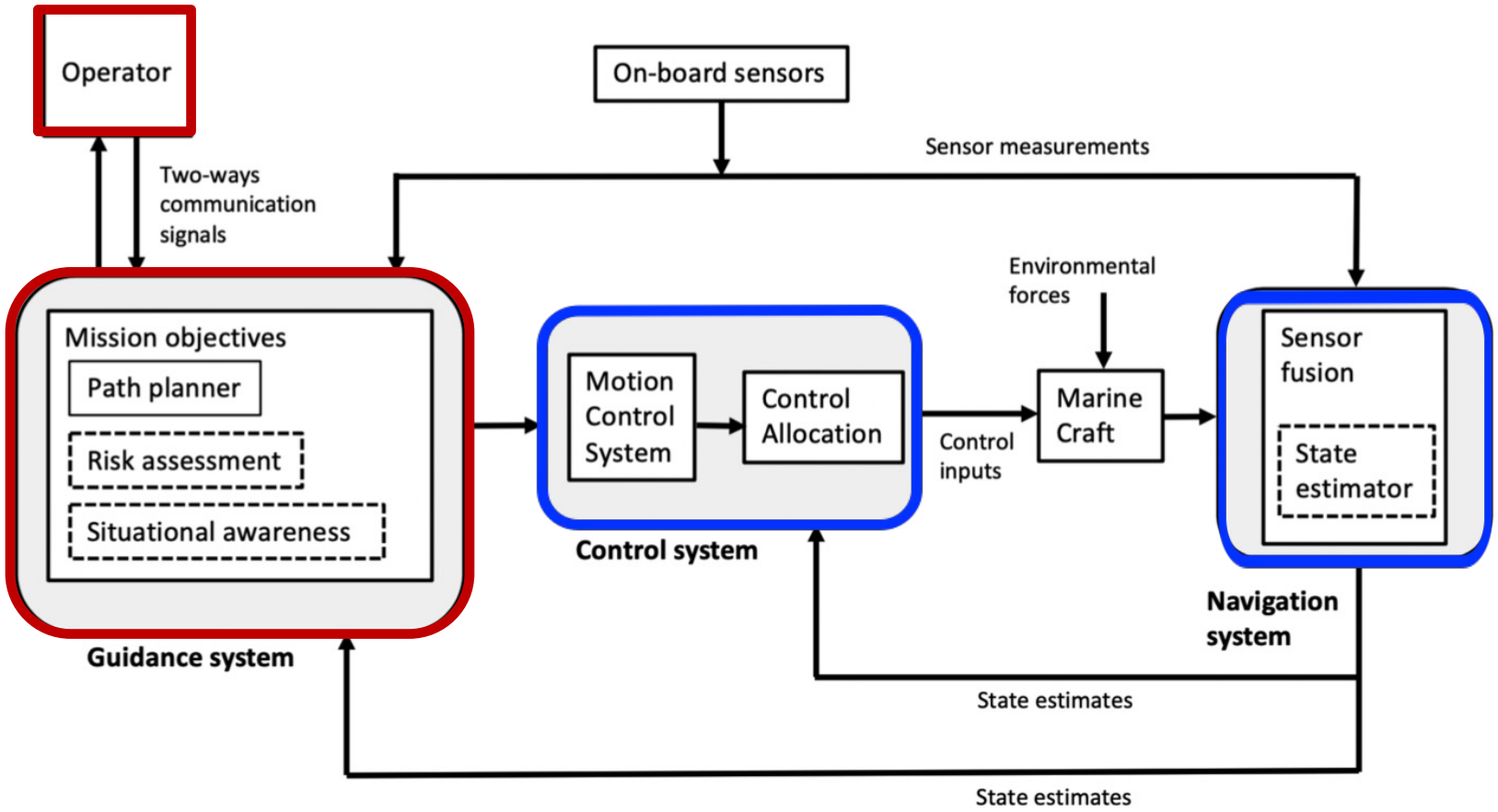
Guidance systems:

- Be able to design open-loop guidance systems using reference models (low-pass filters) for position and velocity control.
- Understand how to design a closed-loop guidance system using feedback control theory.

Guidance Laws:

- Be able to design guidance laws for target tracking, that is a moving object for which no future motion information, nor the path is known in advance.
- Be able to design a state-of-the-art linear path-following controller. This is the classical design method where the path is generated using waypoints to connect straight lines between the waypoints.
- Be able to design LOS guidance laws for path following using course autopilots.
- Be able to design LOS guidance laws for path following using heading autopilots, with extension to ILOS to compensate for sideslip and unmodelled dynamics.

Chapter 12 - Guidance Systems



Guidance refers to determining the desired trajectory from the craft's current location to a designated target, together with the associated velocity, attitude, and acceleration commands required to follow that trajectory.

Guidance, Navigation and Control (GNC)

GNC is a branch of engineering dealing with the design of systems to control the movement of vehicles, especially, automobiles, drones, marine craft, aircraft, and spacecraft.

- **Guidance** focuses on the methods and systems used to follow a route or path to a destination once it has been identified. It's about steering, direction, and instruction on moving from one point to another.
  - How do I get there? is fundamentally a question of guidance.

Once you know your current location and your destination (navigation), guidance provides the step-by-step instructions or actions required to reach the destination.

- **Navigation** refers to the process of determining one's current position and planning a route to a desired destination.
  - Where am I? identifies your current location. This is the primary navigation question.
  - Where do I want to go? decides your destination
  - Which way should I go? plans the route to get there, considering various factors such as distance, time, obstacles, and preferences

- **Control** refers to the manipulation of the forces, by way of steering controls, thrusters, etc., needed to execute guidance commands whilst maintaining vehicle stability.

Lecture Notes: Marine Craft Hydrodynamics and Motion Control (T. I. Fossen)

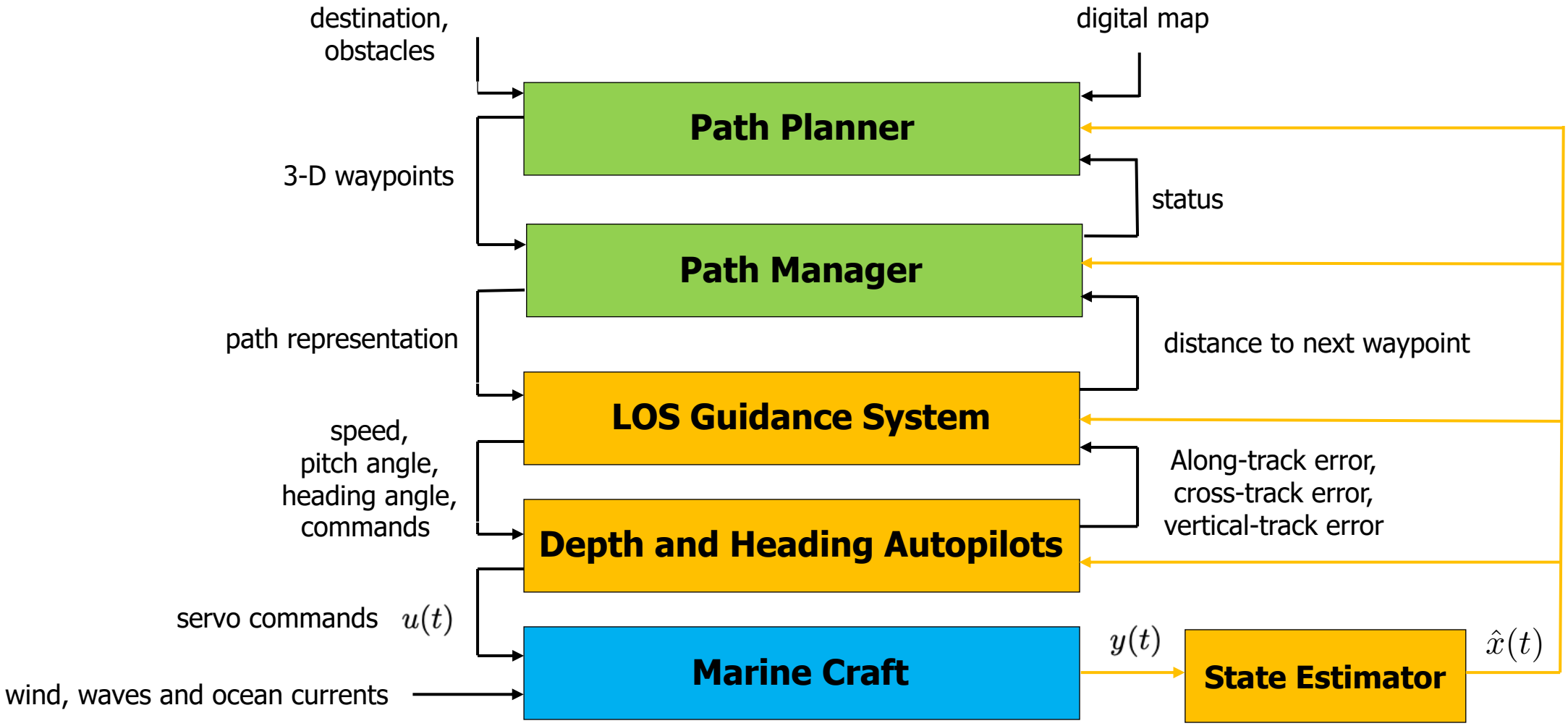
Chapter 12 - Guidance Systems

Control objectives:

In chapter 12, we will consider the following motion control scenarios:

- **Setpoint regulation (point stabilization)** is a special case where the desired position and attitude are chosen to be constant.
- **Trajectory tracking** forces the system output $y(t)$ to track a desired output $y_d(t)$. The desired trajectory can be computed using reference models generated by low-pass filters, optimization methods or by simulating the marine craft motion.
- **Target tracking** methods for motion control scenarios then there are no information about the path and there is no trajectory to track. This is useful if the goal is to track a moving object such as a ship or an underwater vehicle, for which no future motion information is available. For this we use the target position and velocity as reference signals.
- **Path following** is following a predefined path independent of time. A popular approach is the Dubins path representing the shortest curve that connects two points in the 2-D Euclidean plane with a constraint on the curvature of the path and with prescribed initial and terminal tangents to the path. For path following no restrictions are placed on the temporal propagation along the path. Spatial constraints can, however, be added to represent obstacles and other positional constraints if they are known in advance.

Guidance System Architecture



Trajectory Tracking

12.1 Trajectory Tracking

Definition 12.1 (Trajectory Tracking)
A control system that forces the system output $\boldsymbol{y} \in \mathbb{R}^m$ *to track a desired output* $\boldsymbol{y}_d \in \mathbb{R}^m$ *solves a trajectory tracking problem.*

Tracking of a time-varying reference trajectory in 3 DOF (surge, sway and yaw) is achieved by minimizing the tracking error

$$\boldsymbol{e} := \boldsymbol{\eta} - \boldsymbol{\eta}_d = \begin{bmatrix} x^n - x_d^n \\ y^n - y_d^n \\ \psi - \psi_d \end{bmatrix}$$

Trajectory-tracking control laws are classified according to the number of available actuators.

Classification of Trajectory-Tracking Controllers:

- **Three or more controls:** This is referred to as a fully actuated dynamic positioning (DP) system where the configuration space is chosen as *surge, sway* and *yaw.* Typical applications are crab-wise motions (low-speed maneuvering) and stationkeeping where the goal is to drive the trajectory tracking error $e(t) = [e_x(t), e_y(t), e_\psi(t)]^T \rightarrow \mathbf{0}$.

This is the standard configuration for offshore DP vessels.



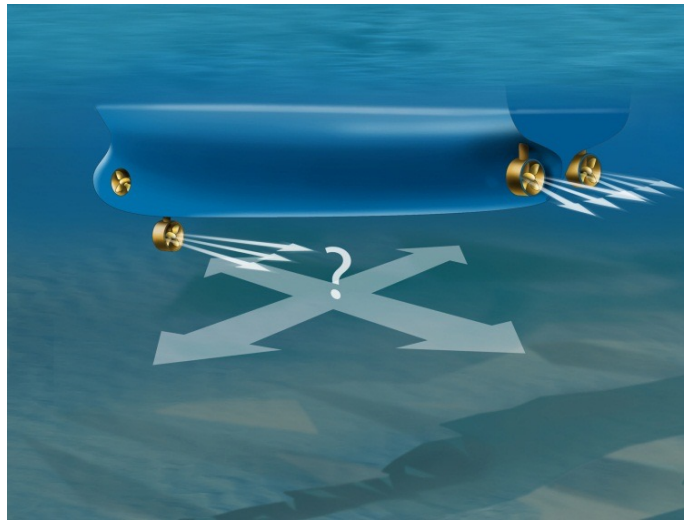
12.1 Trajectory Tracking

- **Two controls and trajectory tracking:** Trajectory-tracking in 3 DOFs, $\boldsymbol{e(t)} \in \mathbb{R}^3$, with only two controls, $\boldsymbol{u(t)} \in \mathbb{R}^2$, is an underactuated control problem which cannot be solved using linear theory. This problem has limited practical use if any!

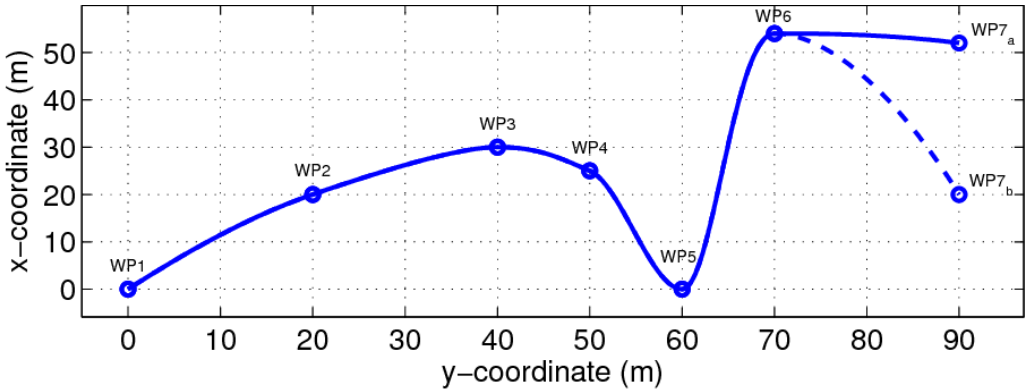
- **Two controls and weather-optimal heading:**
If the ship is aligned up against the mean resulting force due to wind, waves and ocean currents a weathervaning controller can be designed such that only two controls, $\boldsymbol{u(t)} \in \mathbb{R}^2$, are needed to stabilize the ship in 3 DOFs. In this approach, the heading angle is left uncontrolled and allowed to vary automatically with the mean environmental forces.

- **Two controls and path following:** It is standard procedure to define a 2-D workspace (along-track and cross-track errors) and minimize the cross-track error by means of a LOS path-following controller. Hence, it is possible to follow a path by using only two controls (surge speed and yaw moment). For a conventional ship this is achieved by using a rudder and a propeller only. Since the input and output vectors are of dimension two, the 6-DOF system model must be internally stable.

- **One control:** It is impossible to design stationkeeping systems and trajectory-tracking control systems in 3 DOFs for a marine craft using only one control.



Copyright © Bjarne Stenberg



2-D path generated using waypoints in the xy plane

12.1.1 Reference Models for Trajectory Generation

Simplest form is to use a *low-pass (LP) filter*

$$\frac{x_d}{r}(s) = h_{lp}(s)$$

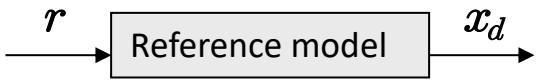
where ***r*** is the command and $\boldsymbol{x_d}$ is the desired state

Choice of filter should reflect the dynamics of the craft feasible trajectory

- Speed limitations
- Acceleration limitations



Copyright © Bjarne Stenberg



Bandwidth of the reference model $<$ Bandwidth of the vessel motion control system

12.1.1 Reference Models for Trajectory Generation

For marine craft it is convenient to use reference models motivated by the dynamics of *mass-damper-spring systems* to generate desired velocities

$$\frac{\nu_{d_i}}{r_i^b}(s) = \frac{\omega_{n_i}^2}{s^2 + 2\zeta_i\omega_{n_i}s + \omega_{n_i}^2}$$

$\zeta_i \ (i = 1, \ldots, n)$ relative damping ratio

$\omega_{n_i} \ (i = 1, \ldots, n)$ natural frequency

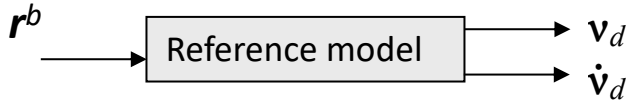
This model can be written as a *MIMO mass-damper-spring system*

$$\dot{\boldsymbol{x}}_d = \boldsymbol{A}_d\boldsymbol{x}_d + \boldsymbol{B}_d\boldsymbol{r}^b \qquad \boldsymbol{x}_d := [\boldsymbol{\nu}_d^\top, \dot{\boldsymbol{\nu}}_d^\top]^\top \in \mathbb{R}^{2n}$$

$$\boldsymbol{A}_d = \begin{bmatrix} \boldsymbol{0}_{n\times n} & \boldsymbol{I}_n \\ -\boldsymbol{\Omega}^2 & -2\boldsymbol{\Delta}\boldsymbol{\Omega} \end{bmatrix}, \quad \boldsymbol{B}_d = \begin{bmatrix} \boldsymbol{0}_{n\times n} \\ \boldsymbol{\Omega}^2 \end{bmatrix}$$

$$\boldsymbol{\Delta} = \mathrm{diag}\{\zeta_1, \zeta_2, \ldots, \zeta_n\}$$

$$\boldsymbol{\Omega} = \mathrm{diag}\{\omega_{n_1}, \omega_{n_2}, \ldots, \omega_{n_n}\}$$



$$\lim_{t\to\infty} \boldsymbol{\nu}_d = \boldsymbol{r}^b$$

Note that a step in the command $\boldsymbol{r}^b$ will give a step in $\dot{\boldsymbol{\nu}}_d$

12.1.1 Reference Models for Trajectory Generation

The position and attitude reference model $\boldsymbol{\eta}_d$ is typically chosen of third order for filtering of steps in the command $\mathbf{r}^n$

$$\frac{\eta_{d_i}}{r_i^n}(s) = \frac{\omega_{n_i}^2}{(1+T_i s)\left(s^2 + 2\zeta_i \omega_{n_i} s + \omega_{n_i}^2\right)}$$

$$\frac{\eta_{d_i}}{r_i^n}(s) = \frac{\omega_{n_i}^3}{s^3 + (2\zeta_i + 1)\omega_{n_i} s^2 + (2\zeta_i + 1)\omega_{n_i}^2 s + \omega_{n_i}^3}$$

State-space representation

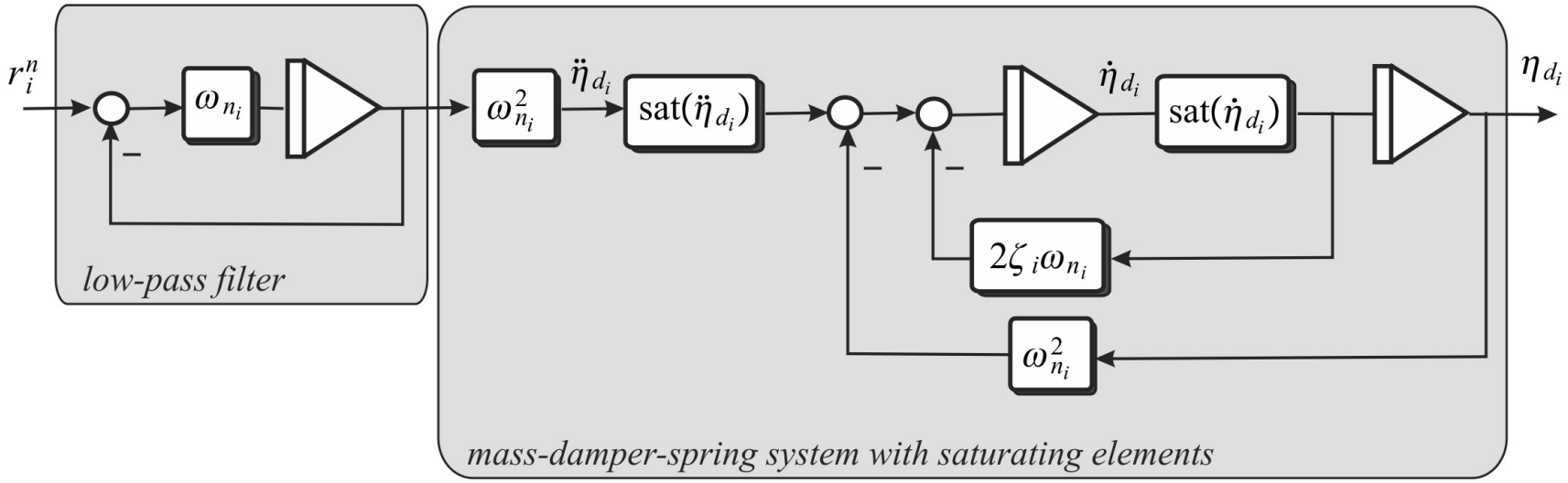
$$\dot{\boldsymbol{x}}_d = \boldsymbol{A}_d \boldsymbol{x}_d + \boldsymbol{B}_d \boldsymbol{r}^n \qquad \boldsymbol{x}_d := [\boldsymbol{\eta}_d^\top, \dot{\boldsymbol{\eta}}_d^\top, \ddot{\boldsymbol{\eta}}_d^\top]^\top \in \mathbb{R}^{3n}$$

$$\boldsymbol{A}_d = \begin{bmatrix} \mathbf{0}_{n\times n} & \boldsymbol{I}_n & \mathbf{0}_{n\times n} \\ \mathbf{0}_{n\times n} & \mathbf{0}_{n\times n} & \boldsymbol{I}_n \\ -\boldsymbol{\Omega}^3 & -(2\boldsymbol{\Delta} + \boldsymbol{I}_n)\boldsymbol{\Omega}^2 & -(2\boldsymbol{\Delta} + \boldsymbol{I}_n)\boldsymbol{\Omega} \end{bmatrix}, \quad \boldsymbol{B}_d = \begin{bmatrix} \mathbf{0}_{n\times n} \\ \mathbf{0}_{n\times n} \\ \boldsymbol{\Omega}^3 \end{bmatrix}$$

$$\lim_{t\to\infty} \boldsymbol{\eta}_d = \boldsymbol{r}^n$$

12.1.1 Reference Models for Trajectory Generation

One *drawback* with a *linear reference model* is that the time constants in the model often yields a satisfactory response for one operating point of the system while the response for other amplitudes of the operator input r_i results in a completely different behavior. This is due to the exponential convergence of the signals in a linear system.



The performance of the linear reference model can also be improved by including saturation elements for *velocity* and *acceleration*

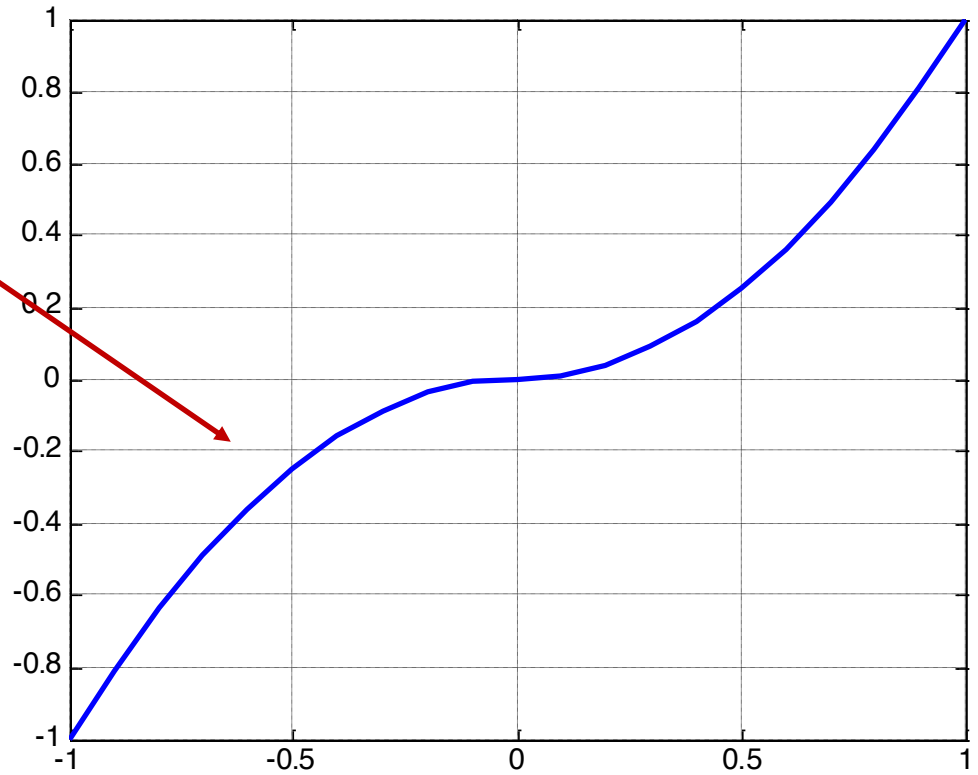
$$\text{sat}(x) = \begin{cases} \text{sgn}(x)x^{\max} & \text{if } |x| \geq x^{\max} \\ x & \text{else} \end{cases}$$

12.1.1 Reference Models for Trajectory Generation

Nonlinear damping can also be included in the reference model to reduce the velocity for large amplitudes or step inputs *r*. This suggests the modification:

$$\dot{\eta}_d = \nu_d$$

$$\dot{\nu}_d + 2\zeta\omega_n\nu_d + \delta\,|\nu_d|\,\nu_d + \omega_n^2\eta_d = \omega_n^2 r$$



12.1.1 Reference Models for Trajectory Generation

Example: Reference Model:

Consider the *mass-damper-spring* reference model

$$\dot{\eta}_d = \nu_d$$

$$\dot{\nu}_d + 2\zeta\omega_n\nu_d + \delta\,|\nu_d|\,\nu_d + \omega_n^2\eta_d = \omega_n^2 r$$

$$\zeta = \omega_n = 1$$

In the figure, we compare responses using:

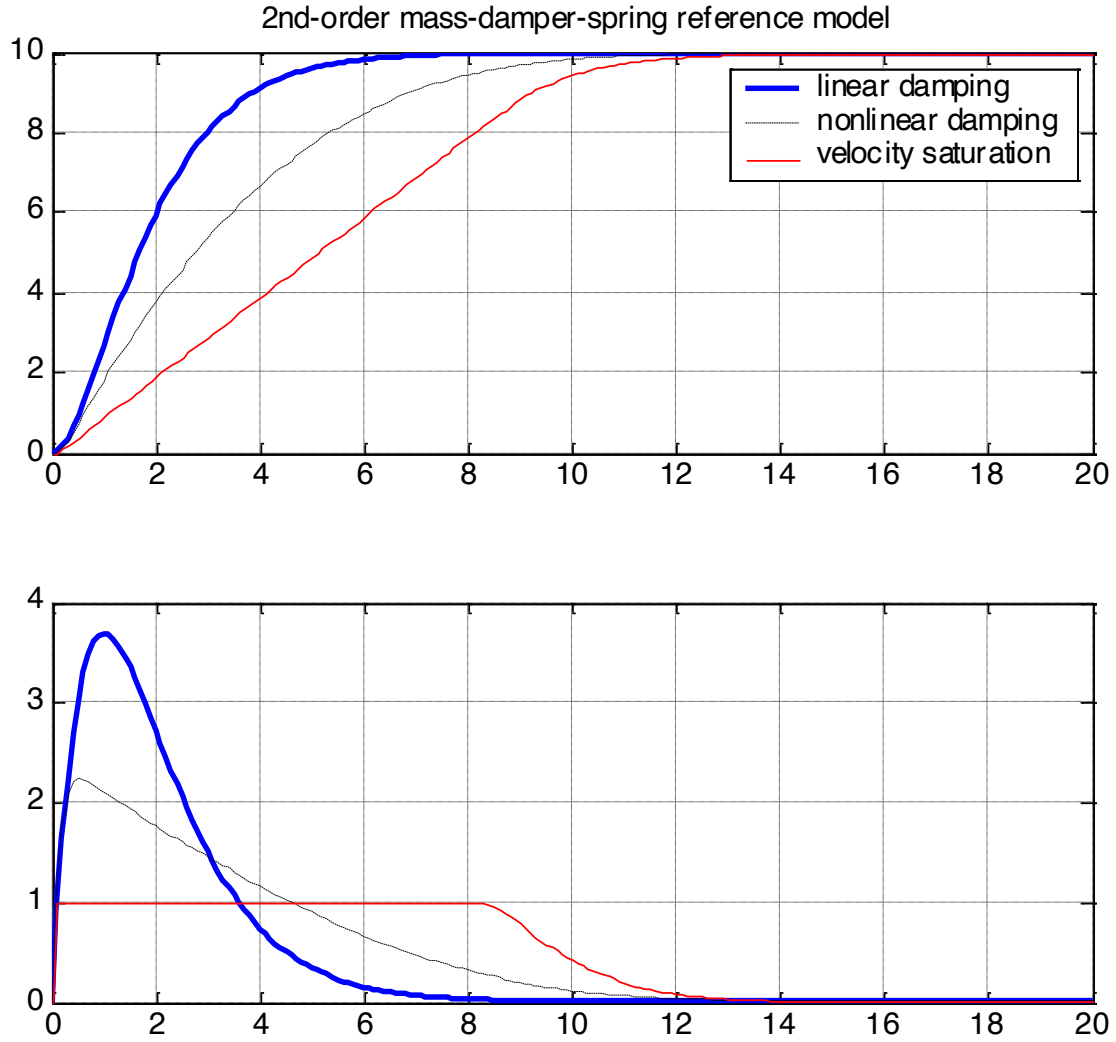
$$\delta = 0, \delta = 1$$

and a saturating element:

$$v_{\max} = 1$$

for an operator step input *r = 1*.

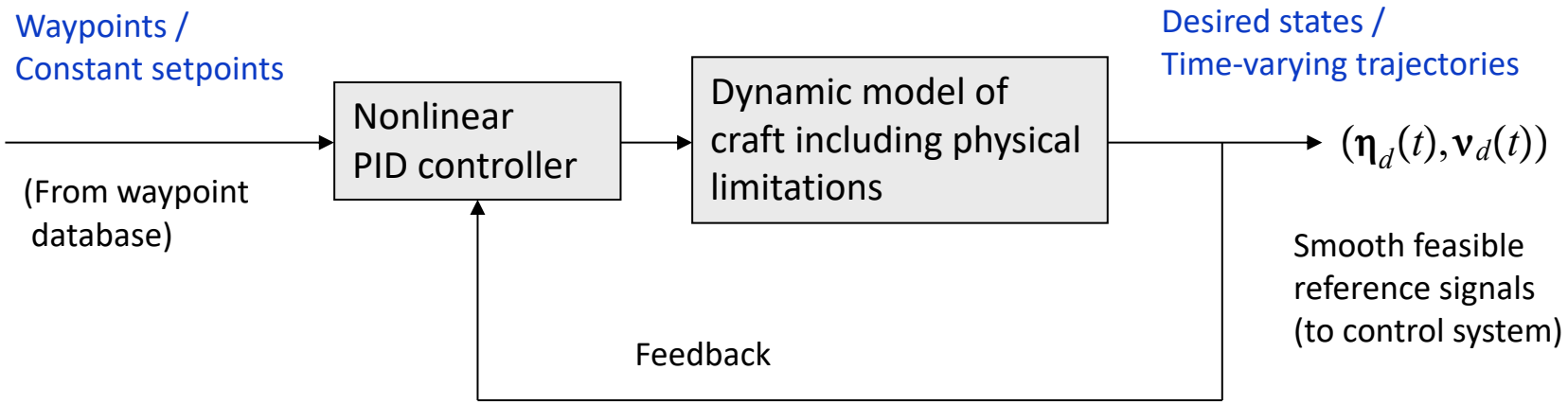
MSS Toolbox: exRefMod.m



Desired position and velocity for a step input *r = 10*.

12.1.2 Trajectory Generation using a Marine Craft Simulator

Guidance System: Dynamic marine craft model + Feedback control system



Trajectory generation using a marine craft simulator means computing a feasible path in space and time for the vessel, while testing it in realistic dynamic and environmental conditions provided by the simulator.

- Saturation elements for velocity and acceleration should be included to keep these quantities within their physical limits.
- A switching strategy between the setpoints (waypoints) must also be adopted.

12.1.2 Trajectory Generation using a Marine Craft Simulator

Generate a time-varying reference trajectory using a closed-loop model of the craft where the time constants, relative damping ratios and natural frequencies are chosen to reflect physical limitations of the craft.

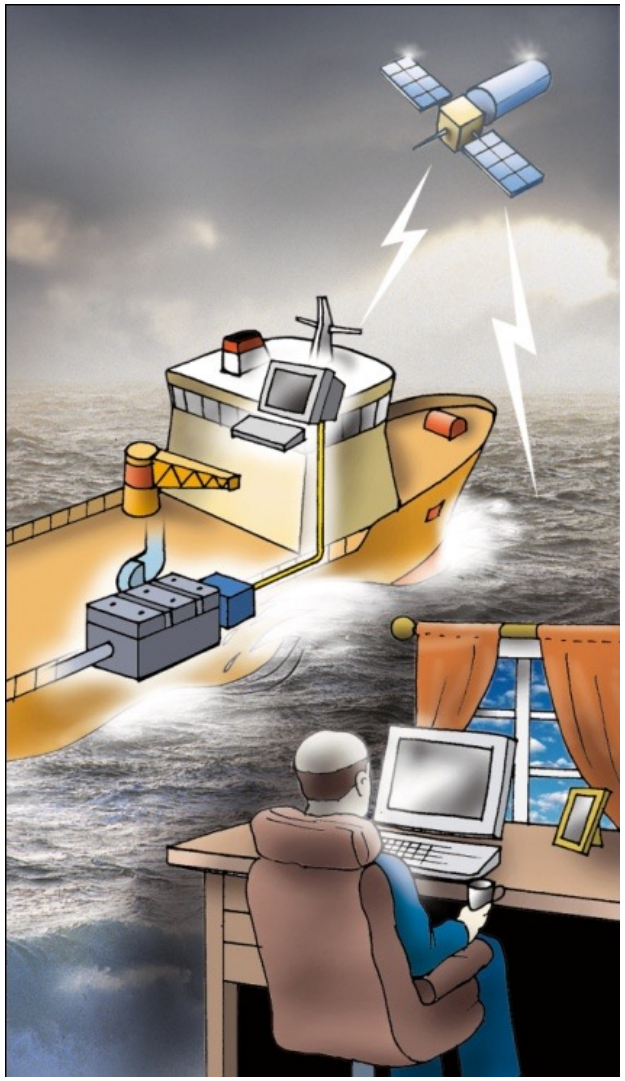
$$\dot{\boldsymbol{\eta}}_d = \boldsymbol{J}_\Theta(\boldsymbol{\eta}_d)\boldsymbol{\nu}_d$$
$$\boldsymbol{M}\dot{\boldsymbol{\nu}}_d + \boldsymbol{D}\boldsymbol{\nu}_d + \boldsymbol{g}(\boldsymbol{\eta}_d) = \boldsymbol{\tau}$$

$$\boldsymbol{D} = \mathrm{diag}\{d_1, \ldots, d_6\} > 0$$

$$\boldsymbol{\tau} = \boldsymbol{g}(\boldsymbol{\eta}_d) - \boldsymbol{J}_\Theta^\top(\boldsymbol{\eta}_d)\left(\boldsymbol{K}_p(\boldsymbol{\eta}_d - \boldsymbol{\eta}_{\mathrm{ref}}) + \boldsymbol{K}_d\dot{\boldsymbol{\eta}}_d\right)$$

where $\boldsymbol{\eta}_{ref}$ is the desired reference and $(\boldsymbol{\eta}_d, \boldsymbol{v}_d)$ represents the desired states.
The PD control law is a guidance controller since it is applied to the reference model.

In addition to this, it is smart to include saturation elements for velocity and acceleration to keep these quantities within their physical limits.



Copyright © Bjarne Stenberg

12.1.2 Trajectory Generation using a Marine Craft Simulator

Example: Generation of Reference Trajectories using a Marine Craft Simulator

The kinematics of a ship can be modeled as

$$\dot{x}_d^n = u_d \cos(\psi_d)$$
$$\dot{y}_d^n = u_d \sin(\psi_d)$$

Kinematic equations

with speed dynamics

$$(m - X_{\dot{u}})\dot{u}_d + \frac{1}{2}\rho C_d A |u_d| u_d = T_{|n|n}|n|n$$

Mass + Quadratic drag formula for damping + Quadratic RPM for thrust

The yaw dynamics is chosen as

$$\dot{\psi}_d = r_d$$
$$T\dot{r}_d + r_d = K\delta$$

Nomoto model

where *K* and *T* are design parameters.

The guidance system has two inputs, *thrust* and *rudder angle*

$$\tau = -K_{p_u}(u_d - u_{ref}) - K_{i_u}\int_0^t (u_d - u_{ref})\mathrm{d}\tau, \qquad n = \mathrm{sgn}(\tau)\sqrt{|\tau|}$$
$$\delta = -K_{p_\psi}(\psi_d - \psi_{ref}) - K_{i_\psi}\int_0^t (\psi_d - \psi_{ref})\mathrm{d}\tau - K_{d_\psi} r_d$$

LOS guidance law

$$\psi_{ref} = \mathrm{atan2}\left(y_{los} - y_d,\ x_{los} - x_d\right)$$

Numerical integration of the ODEs with feedback yields a smooth reference trajectory.

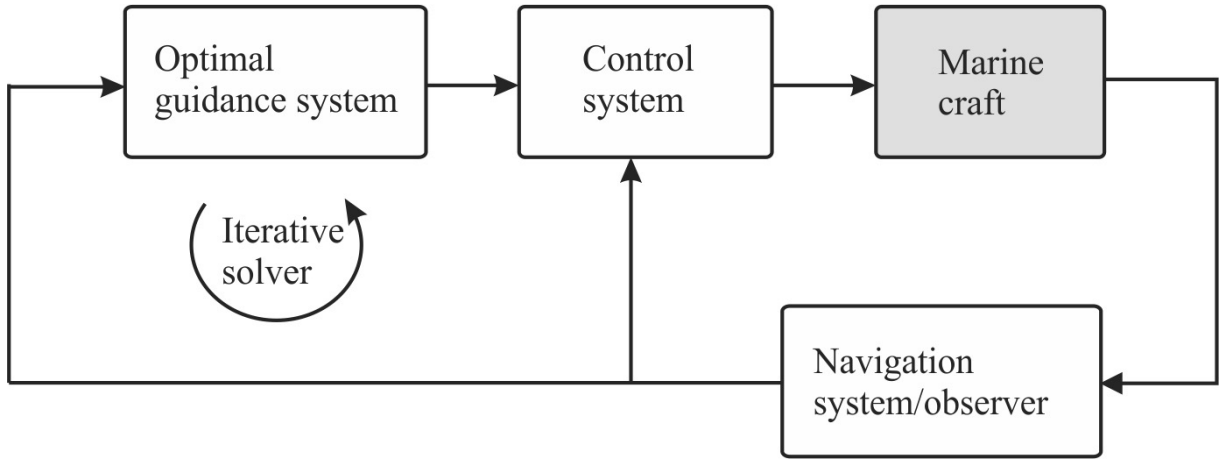
12.1.3 Optimal Trajectory Generation

Optimization methods can be applied to generate trajectories and paths in a systematic way that incorporates both static and dynamic constraints. The drawback is that a constrained optimization problem must be solved online to produce a feasible time-varying trajectory.

Implementation can be carried out using linear programming (LP), quadratic programming (QP), or nonlinear optimization methods, all of which require an appropriate solver integrated into the program.

$$J = \min_{\boldsymbol{\eta}_d, \boldsymbol{\nu}_d} \{\text{power, time}\}$$

$$U \leq U^{\max} \text{ (maximum speed)}$$
$$|r| \leq r^{\max} \text{ (maximum turning rate)}$$
$$|u_i| \leq u_i^{\max} \text{ (saturating limit of control } u_i)$$
$$|\dot{u}_i| \leq \dot{u}_i^{\max} \text{ (saturating limit of rate } \dot{u}_i)$$



Once an optimal solution is found, a neural network (NN) can be trained to approximate it, replacing the QP solver in the control system.

12.1.3 Path Generation using Dynamic Optimization

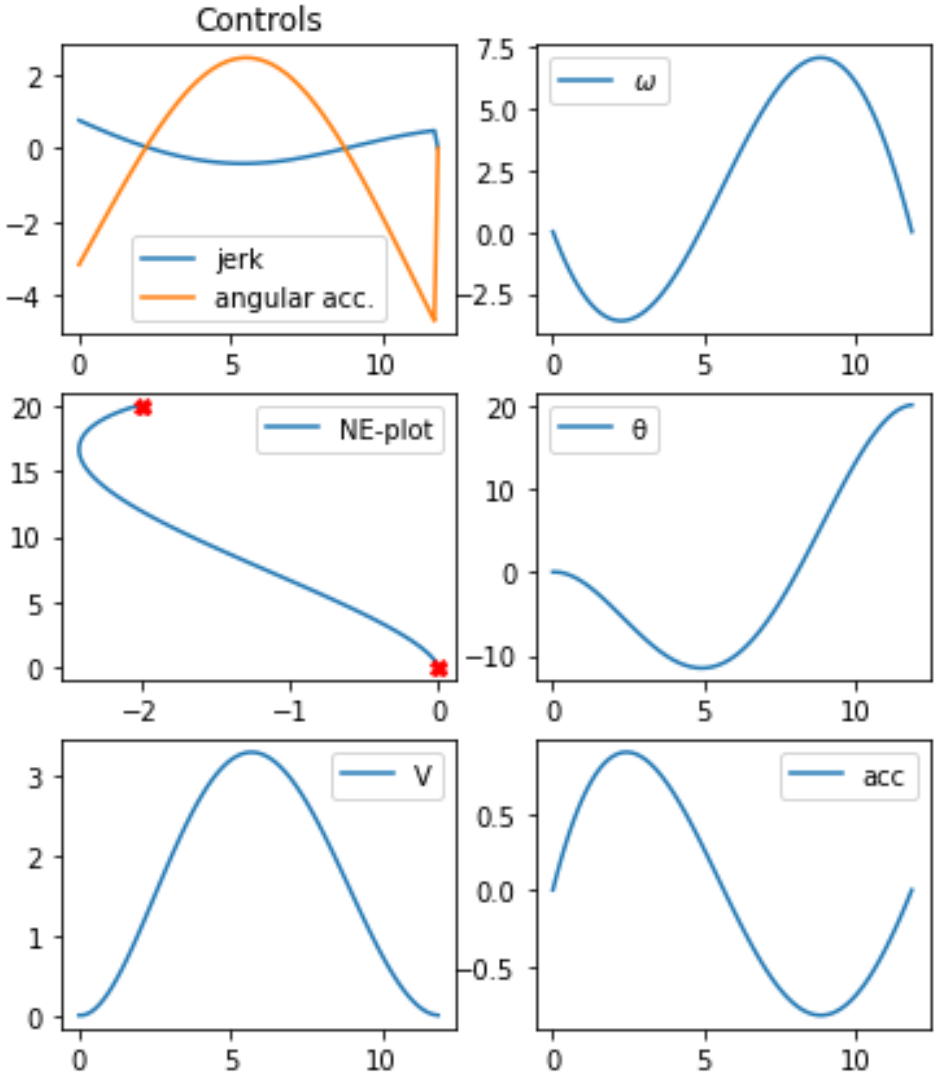
In this example, CasADi is used to solve the QP trajectory optimization problem subject to the given dynamics, constraints, and cost function.

$$
\begin{aligned}
\dot{x} &= V\cos(\theta), & x(0) &= x_0 \\
\dot{y} &= V\sin(\theta), & y(0) &= y_0 \\
\dot{\theta} &= \omega, & \theta(0) &= \theta_0 \\
\dot{V} &= a, & V(0) &= V_0 \\
\dot{a} &= u_1, & \dot{a}(0) &= 0 \\
\dot{\omega} &= u_2, & \dot{\omega}(0) &= 0
\end{aligned}
$$

$$
\text{Minimize} \quad J = w_1 t_f + w_2 \int_0^{t_f} u_1^2\, dt + w_3 \int_0^{t_f} u_2^2\, dt
$$

Subject to

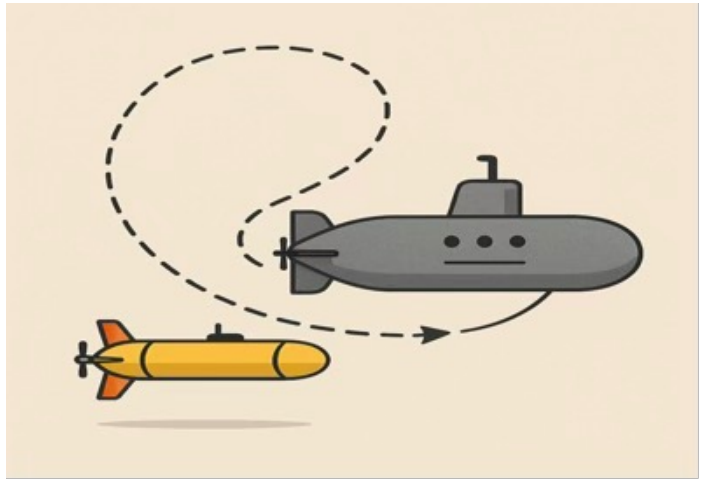
$$
\begin{aligned}
x(t_f) &= x_{\text{final}} \\
y(t_f) &= y_{\text{final}} \\
\theta(t_f) &= \theta_{\text{final}} \\
V(t_f) &= V_{\text{final}} \\
\mathrm{a}(t_f) &= \mathrm{a}_{\text{final}} \\
\omega(t_f) &= \omega_{\text{final}} \\
0 \le V &\le V_{\max}
\end{aligned}
$$



https://web.casadi.org

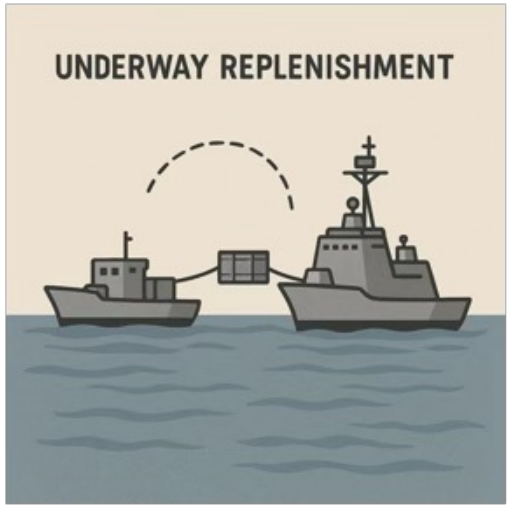
Guidance laws for Target Tracking

12.2 Guidance Laws for Target Tracking



Guidance systems were originally developed and applied in missiles, rockets, and torpedoes, especially during and after WWII. While control theory was broadly developed in the same era and applied to aircraft and marine craft (e.g., autopilots, ship steering), guidance systems for marine craft and aircraft came later, particularly with the rise of automation and autonomous vehicles.

Guidance laws for target tracking can also be used in marine operations such as underway replenishment (UNREP) operations and formation control. UNREP operations involve cargo transfer between two cooperating surface craft in transit.





12.2 Guidance Laws for Target Tracking

An intercepting ship can control its speed U and course angle χ using a conventional autopilot system.

Kinematic transformation (NE velocities ←→ Course angle/speed)

$$u_d^n = U_d \cos(\chi_d)$$
$$v_d^n = U_d \sin(\chi_d)$$



$$\chi_d = \text{atan2}(v_d^n, u_d^n)$$
$$U_d = \sqrt{(u_d^n)^2 + (v_d^n)^2}$$

This suggests that the desired velocities (u_d, v_d) obtained by a target tracking guidance law can be transformed to autopilot reference signals (χ_d , U_d) to achieve target tracking.

It is convenient to distinguish between maneuvering (accelerating) and nonmaneuvering (nonaccelerating) objects. An interceptor typically undergoes three operational phases; Launch, Midcourse and Terminal.

Terminal guidance strategies:

- Line-of-Sight (LOS) Guidance
- Pure Pursuit (PP) Guidance
- Constant Bearing (CB) Guidance

12.2 Guidance Laws for Target Tracking

For surface craft, the **North-East position and velocity** of the target is denoted by

$$\boldsymbol{p}_t^n = [x_t^n, y_t^n]^\top$$

$$\boldsymbol{v}_t^n = \dot{\boldsymbol{p}}_t^n = [u_t^n, v_t^n]^\top \qquad U_t = \|\boldsymbol{v}_t^n\| = \sqrt{(u_t^n)^2 + (v_t^n)^2}$$

The control objective of a target-tracking scenario can be formulated as

$$\lim_{t\to\infty} (\boldsymbol{p}^n - \boldsymbol{p}_t^n) = \boldsymbol{0}$$

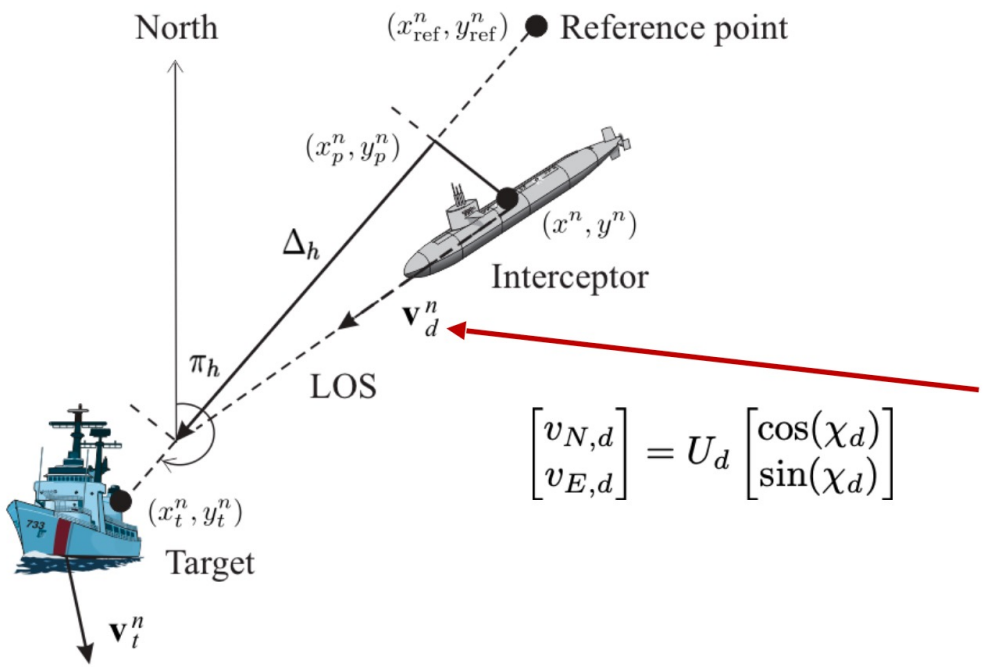
The guidance laws are based on the following measurements

- Target position
- Target velocity (optional)

The target velocity can be obtained by numerical differentiation of the target position, or alternatively by using a target-tracking model with a Kalman filter.



12.2.1 Line-of-Sight Guidance Law



$$\begin{bmatrix} v_{N,d} \\ v_{E,d} \end{bmatrix} = U_d \begin{bmatrix} \cos(\chi_d) \\ \sin(\chi_d) \end{bmatrix}$$

- Three-point guidance scheme.
- The interceptor must constrain its motion along the reference-target line of sight.
- Typically employed for surface-to-air missiles (beam-rider guidance).

The interceptor velocity is aligned with the LOS vector to obtain the desired velocity $\mathbf{v}_d^n$

Path-tangential angle in the horizontal plane (slope of the line or tangent to a curve)

$$\pi_h = \text{atan2}(y^n_t - y^n_{\text{ref}}, x^n_t - x^n_{\text{ref}})$$

Desired course angle

$$\chi_d = \pi_h - \tan^{-1}\left(\frac{y^p_e}{\Delta_h}\right) \qquad \Delta_h > 0 \quad \text{Lookahead distance}$$

12.2.1 Line-of-Sight Guidance Law

Computation of the cross-track error

For a craft located at the position (x^n, y^n), the along-track and cross-track errors $(x_e^p,\ y_e^p)$ are computed as the orthogonal distance to the path-tangential reference frame by setting $x_e^p \equiv 0$

$$\begin{bmatrix} 0 \\ y_e^p \end{bmatrix} = \mathbf{R}_p^n(\pi_h)^\top \left(\begin{bmatrix} x^n \\ y^n \end{bmatrix} - \begin{bmatrix} x_p^n \\ y_p^n \end{bmatrix} \right)$$

$$\tan(\pi_h) = \frac{y_{i+1}^n - y_i^n}{x_{i+1}^n - x_i^n}$$

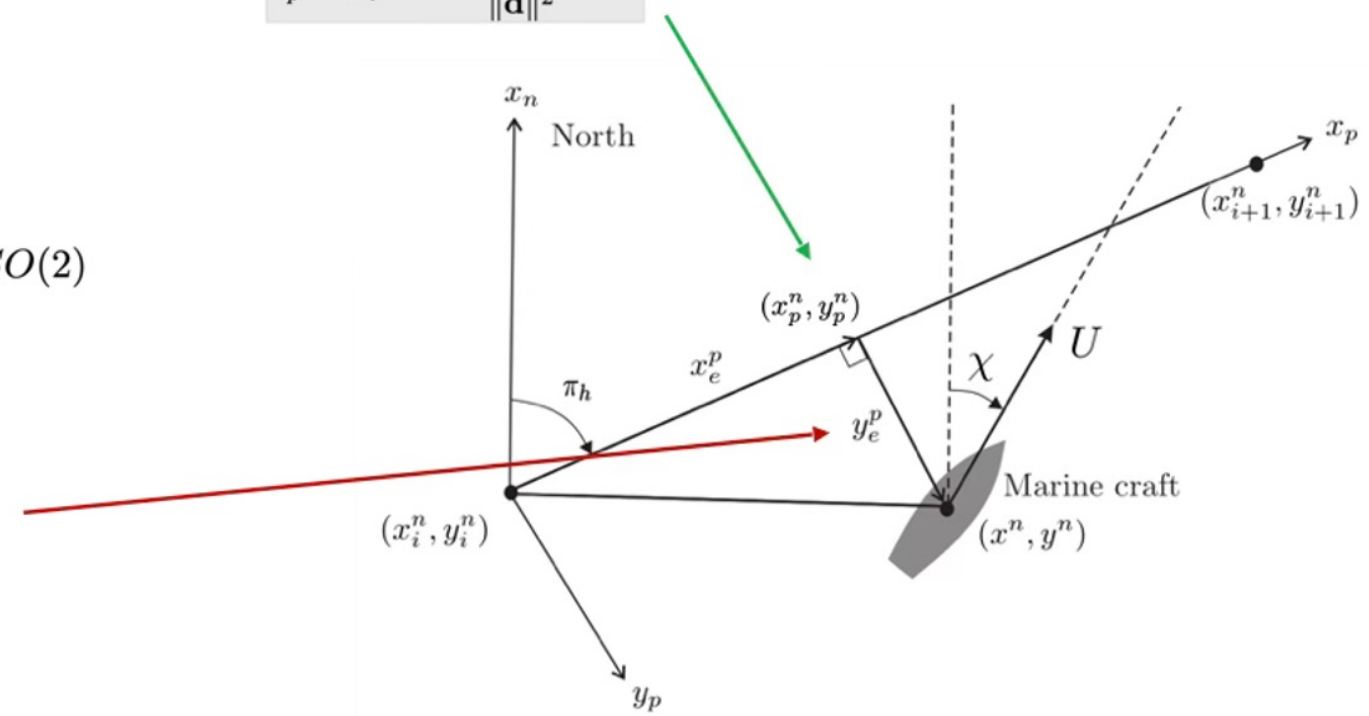
$$\mathbf{R}_p^n(\pi_h) = \begin{bmatrix} \cos(\pi_h) & -\sin(\pi_h) \\ \sin(\pi_h) & \cos(\pi_h) \end{bmatrix} \in SO(2)$$

Cross-track error

$$y_e^p = [0,1]^\top \mathbf{R}^\top(\pi_h)(\mathbf{p}^n - \mathbf{p}_{\text{ref}}^n)$$

The closest point on the path between two waypoints is given by the projection of the vehicle position onto the line

$$\mathbf{p}_p^n = \mathbf{p}_i^n + \frac{(\mathbf{p}^n - \mathbf{p}_i^n)\cdot \mathbf{d}}{\|\mathbf{d}\|^2}\mathbf{d} \qquad \mathbf{d} = \mathbf{p}_{i+1}^n - \mathbf{p}_i^n$$



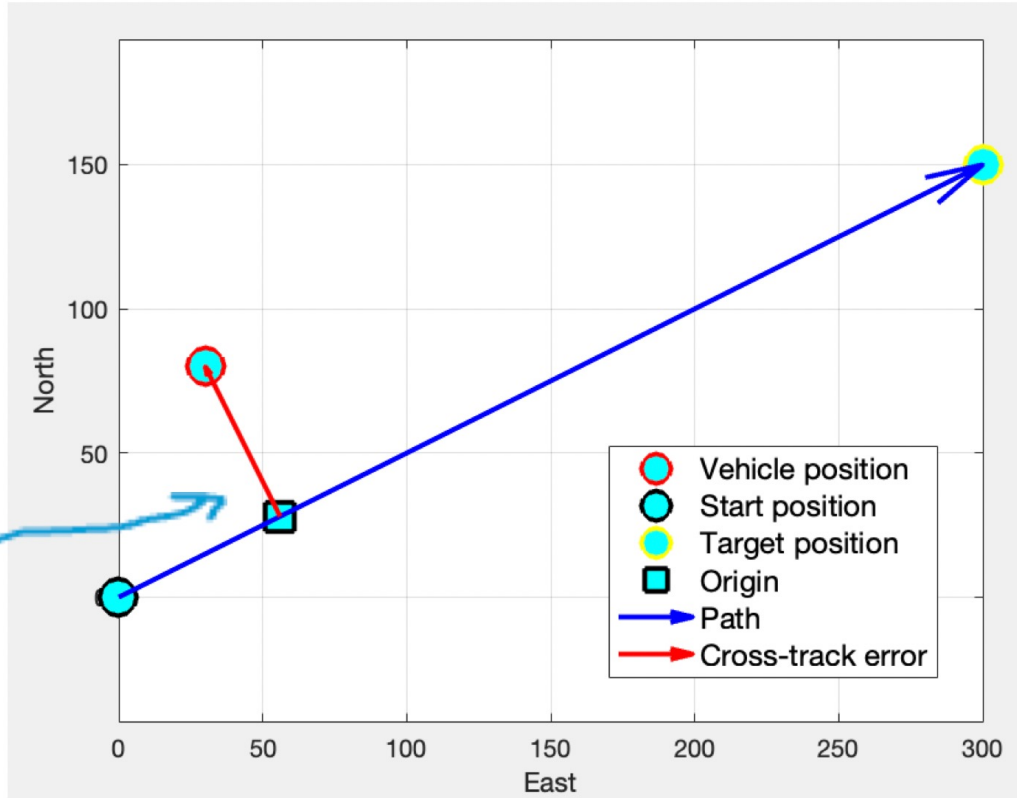
MSS Toolbox: Computation of the Cross-Track Error

Matlab:
The cross-track error can be computed using the MSS toolbox function

```
[x_p, y_p, y_e] = crosstrack(x_t, y_t, x_ref, y_ref, x, y, flag)
```

where `flag = 1` is an optional input for plotting the geometry of the problem.

```
>> [xp,yp,ye]= crosstrack(150, 300, 0, 0, 80, 30,1)
xp =
   28.0000   Origin of path-tangential frame
yp =
   56.0000
ye =
  -58.1378   Cross-track error
```

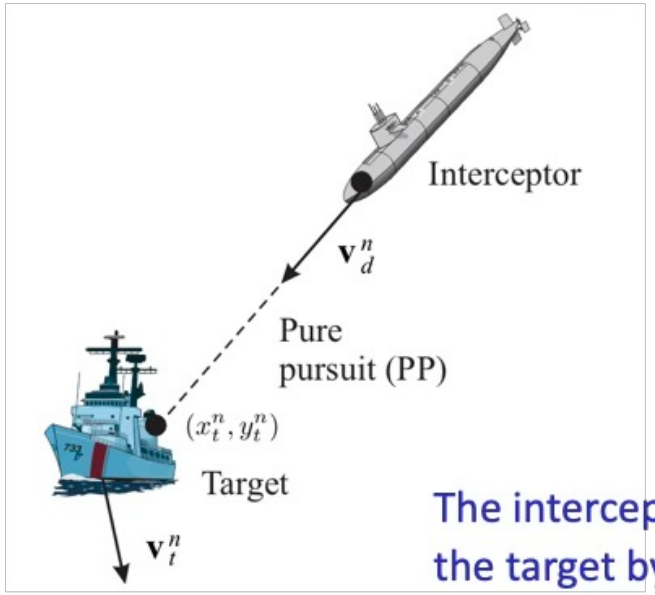


Simplified versions for control applications avoiding visualization and matrix inversion:

```
2-D waypoints: crosstrackWpt(x2, y2, x1, y1, x, y)
3-D waypoints: crosstrackWpt3D([x2, y2, z2], [x1, y1, z1], [x, y, z])

ye = crosstrackWpt(150, 300, 0, 0, 80, 30)
ye =
  -58.1378
```

12.2.2 Pure Pursuit Guidance Law



- Two-point guidance scheme.
- The interceptor must align its linear velocity along the interceptor-target line.
- Equivalent to a predator chasing a prey in the animal world.
- Typically employed for air-to-surface missiles.
- Deviated pursuit guidance is a variant of PP guidance (also called fixed-lead guidance).

The interceptor aligns its velocity along the LOS vector between the interceptor and the target by choosing the desired velocity as:

$$\boldsymbol{v}_d^n = [u_d^n,\, v_d^n]^\top$$

$$\boldsymbol{v}_d^n = -\kappa \frac{\tilde{\boldsymbol{p}}^n}{\|\tilde{\boldsymbol{p}}^n\|}$$

$$\tilde{\boldsymbol{p}}^n := \boldsymbol{p}^n - \boldsymbol{p}_t^n$$

$$\kappa > 0$$

Autopilot commands:

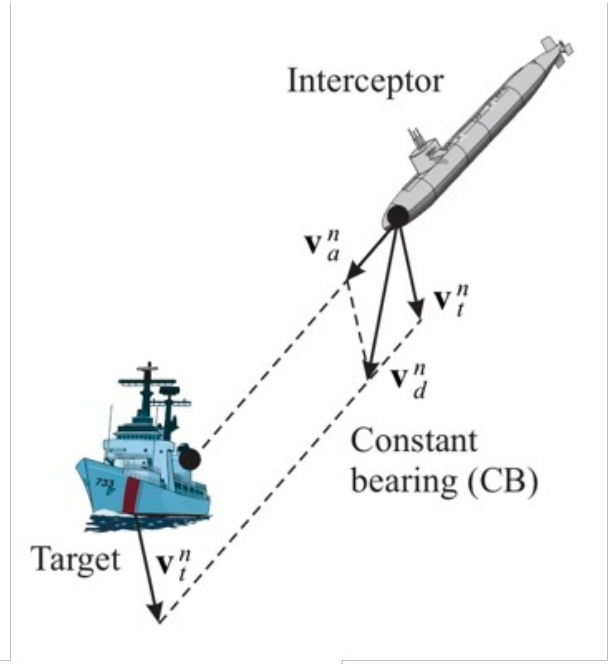
$$\chi_d = \operatorname{atan2}\left(v_{E,d}, v_{N,d}\right)$$

$$U_d = \sqrt{v_{N,d}^2 + v_{E,d}^2}$$



Lecture Notes: Marine Craft Hydrodynamics and Motion Control (T. I. Fossen)

12.2.3 Constant Bearing Guidance Law



$$\boldsymbol{v}_d^n = \boldsymbol{v}_t^n + \boldsymbol{v}_a^n$$

$$\boldsymbol{v}_a^n = -\kappa \frac{\tilde{\boldsymbol{p}}^n}{\|\tilde{\boldsymbol{p}}^n\|}$$

$$\kappa = U_a^{\max} \frac{\|\tilde{\boldsymbol{p}}^n\|}{\sqrt{(\tilde{\boldsymbol{p}}^n)^\top \tilde{\boldsymbol{p}}^n + \Delta^2}}$$

- Two-point guidance scheme.
- Often referred to as Parallel Navigation.
- Typically employed for air-to-air missiles.
- Used for centuries by mariners to avoid collisions at sea.
- Proportional navigation is the most common implementation method (optimal for scenarios involving non-maneuvering targets).

The interceptor aligns the interceptor-target velocity along the LOS vector between the interceptor and the target.

CB guidance tries to maintain a constant angle between the interceptor and the LOS vector. **It works best when**:

- The target moves straight at constant speed.
- And doesn't maneuver aggressively.

When the target performs turns, it will try to intercept on the original path and can overshoot or miss the actual trajectory.

12.2.3 Constant Bearing Guidance Law

Lyapunov function candidate (LFC)

$$V = \frac{1}{2}(\tilde{\boldsymbol{p}}^n)^\top \tilde{\boldsymbol{p}}^n > 0, \quad \forall \tilde{\boldsymbol{p}}^n \neq \boldsymbol{0}$$

Assume that the ship autopilot guarantees that $\boldsymbol{v}^n = \boldsymbol{v}_d^n$. Time differentiation of V along the trajectories of $\tilde{\boldsymbol{p}}^n = \boldsymbol{p}^n - \boldsymbol{p}_t^n$ gives

$$\begin{aligned}
\dot{V} &= (\tilde{\boldsymbol{p}}^n)^\top (\boldsymbol{v}^n - \boldsymbol{v}_t^n) \\
&= -\kappa \frac{(\tilde{\boldsymbol{p}}^n)^\top \tilde{\boldsymbol{p}}^n}{\|\tilde{\boldsymbol{p}}^n\|} \\
&= -U_a^{\max} \frac{(\tilde{\boldsymbol{p}}^n)^\top \tilde{\boldsymbol{p}}^n}{\sqrt{(\tilde{\boldsymbol{p}}^n)^\top \tilde{\boldsymbol{p}}^n + \Delta^2}} \\
&< 0, \quad \forall \tilde{\boldsymbol{p}}^n \neq \boldsymbol{0}
\end{aligned}$$

The LFC is positive definite and radially unbounded, while its derivative with respect to time is negative definite when adhering to

$$U \geq U_a^{\max} > 0$$

Hence, by standard Lyapunov arguments the origin is USGES as shown in **Appendix A.2.3**

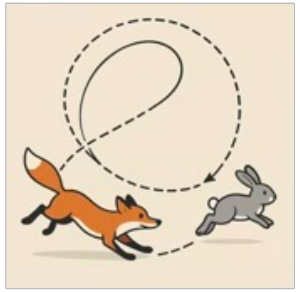
12.2.3 Summary: Guidance Laws for Target Tracking

Feature	Pure Pursuit (PP)	Constant Bearing (CB)	Line-of-Sight (LOS)
Concept	Steer toward target	Hold constant LOS angle	Steer to lookahead point
Reactive	✅ High	❌ Low	✅ Moderate
Predictive	❌ None	✅ Good for straight paths	✅ Lookahead-based
Performs on curves	❌ Spirals behind	❌ Overshoots or fails	✅ Handles well
Smoothness	Moderate	Can be erratic	✅ Smooth
Tuning	Constant gain κ	Time-varying gain κ	Lookahead distance Δ
Implementation	✅ Easiest	Moderate	✅ Simple + robust

Autopilot NED velocity to course/speed commands:

$$\chi_d = \text{atan2}\left(v_{E,d}, v_{N,d}\right)$$
$$U_d = \sqrt{v_{N,d}^2 + v_{E,d}^2}$$

$$\boldsymbol{v}_d^n = -\kappa \frac{\tilde{\boldsymbol{p}}^n}{\|\tilde{\boldsymbol{p}}^n\|}$$



$$\boldsymbol{v}_d^n = \boldsymbol{v}_t^n + \boldsymbol{v}_a^n$$
$$\boldsymbol{v}_a^n = -\kappa \frac{\tilde{\boldsymbol{p}}^n}{\|\tilde{\boldsymbol{p}}^n\|}$$
$$\kappa = U_a^{\max} \frac{\|\tilde{\boldsymbol{p}}^n\|}{\sqrt{(\tilde{\boldsymbol{p}}^n)^\top \tilde{\boldsymbol{p}}^n + \Delta^2}}$$

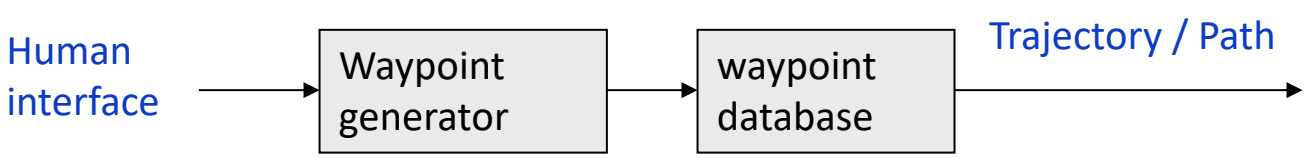
$$\chi_d = \pi_h - \tan^{-1}\left(\frac{y_e^p}{\Delta_h}\right)$$

Next, we will take a closer look at the LOS guidance law and explore its behavior in more detail

Linear Design Methods for Path Following

12.3 Linear Design Methods for Path Following

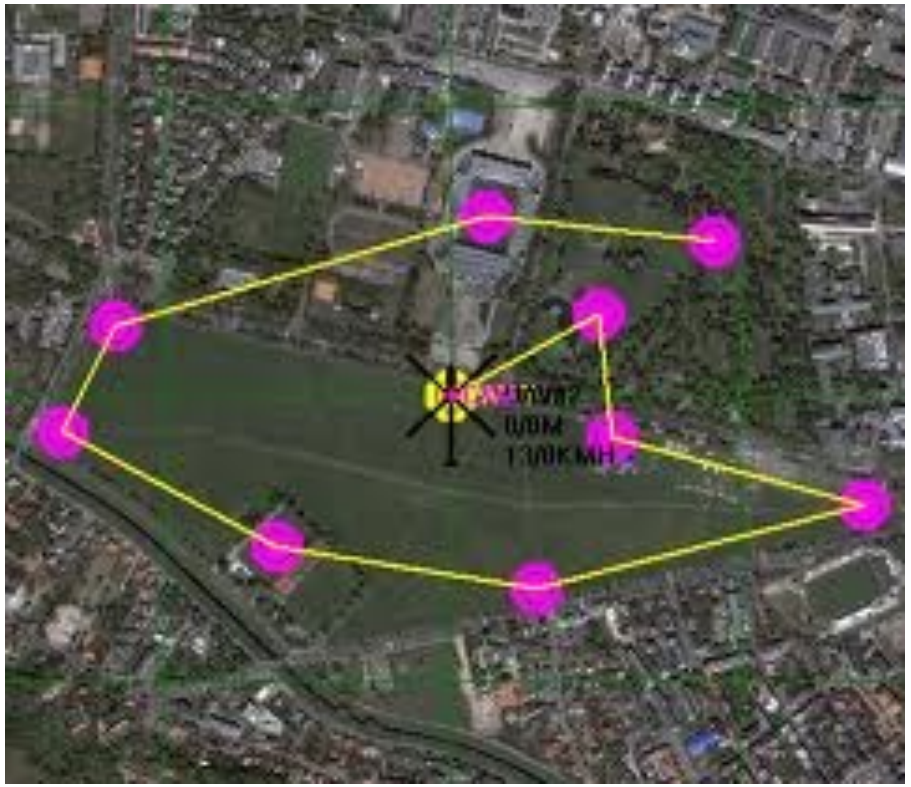
Waypoints are used to indicate changes in direction, speed and altitude along a desired path. Systems for waypoint generation are used both for ships and underwater vehicles. These systems consist of a waypoint generator with a human interface.



A waypoint is an intermediate point along a route where the course may change, or the vehicle may briefly stop before continuing.

The selected waypoints are stored in a waypoint database and used for generation of a trajectory or a path for the moving craft to follow.

Sophisticated features like *weather routing*, *obstacle avoidance* and *mission planning* can be incorporated in the design of waypoint guidance systems.



In a **path-following problem**, the time dependence of the trajectory is removed; instead of requiring the craft to be at a certain position at a specific time, the objective is simply to converge to and follow the geometric path, moving along it at any speed.

Trajectory = position as a function of time.
Path = geometric curve (time removed).

12.3.1 Waypoints

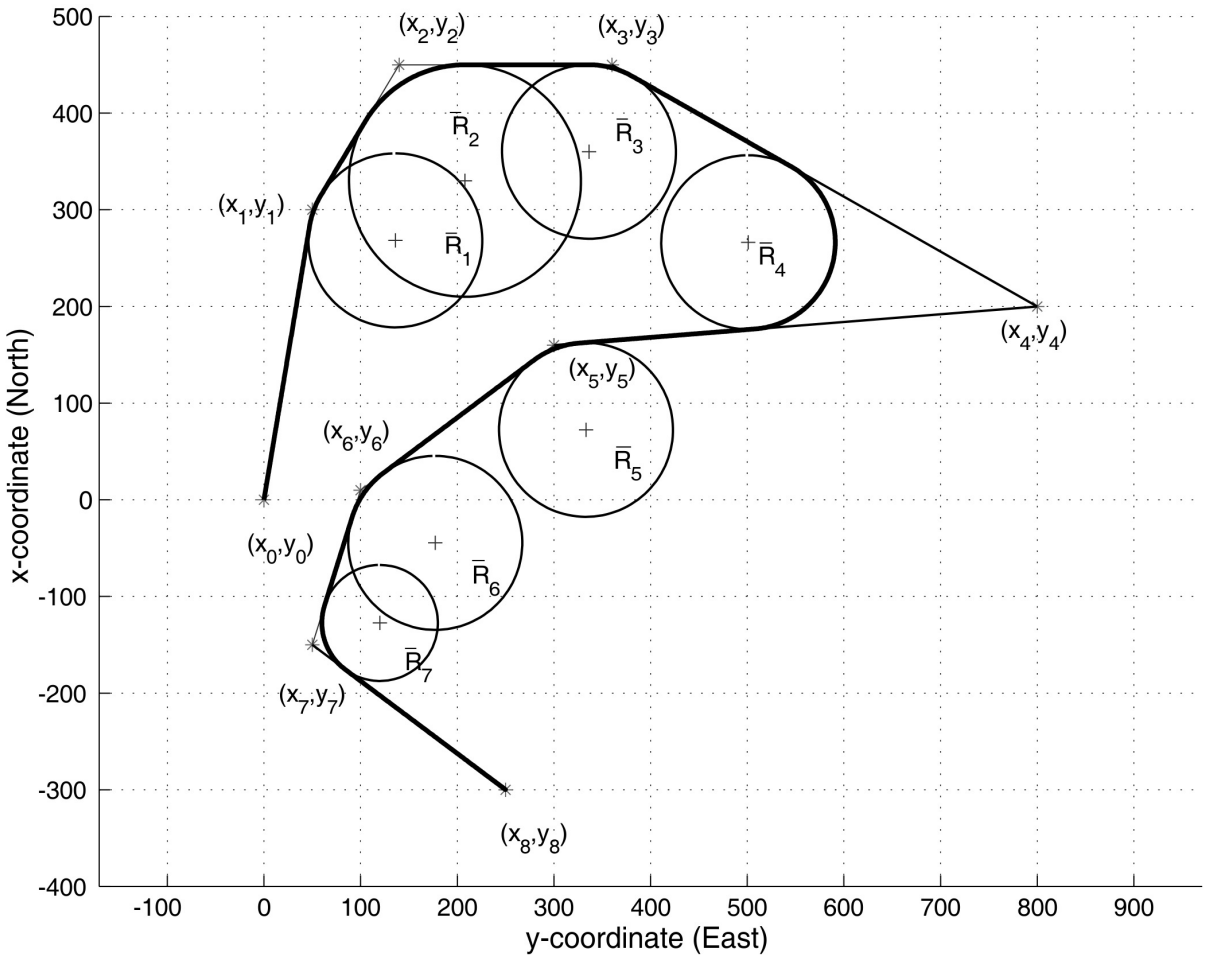
The route of a ship or an underwater vehicle can be specified in terms of waypoints. Each waypoint is defined using Cartesian coordinates (x_i, y_i, z_i) for $i=1,\ldots,n$. The *waypoint database* can be represented by a waypoint data structure:

$$\text{wpt.pos} = \{(x_0, y_0, z_0), (x_1, y_1, z_1), \ldots, (x_n, y_n, z_n)\}$$

and other properties such as

$$\text{wpt.speed} = \{U_0, U_1, \ldots, U_n\}$$
$$\text{wpt.heading} = \{\psi_0, \psi_1, \ldots, \psi_n\}$$
$$\text{wpt.radius} = \{R_0, R_1, \ldots, R_n\}$$

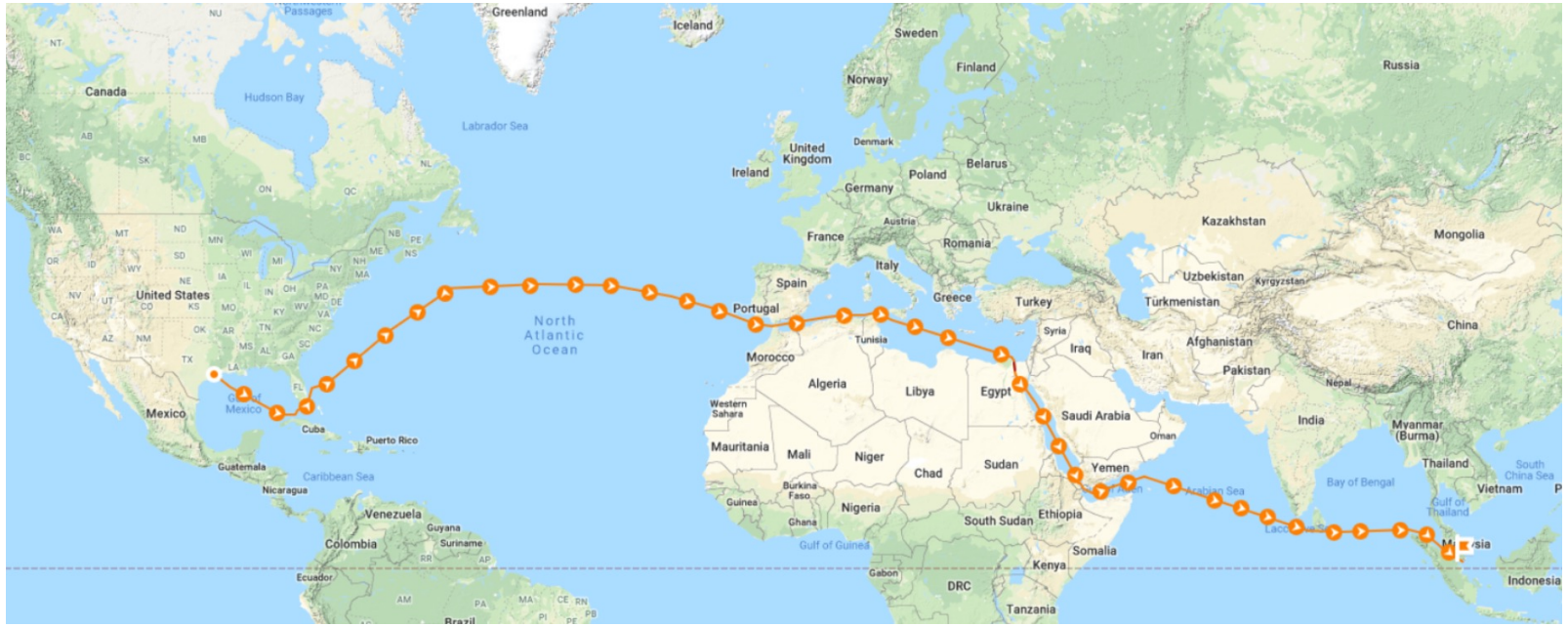
The three states (x_i, y_i, ψ_i) are also called the pose and they describe the start and end configurations of the craft given by two waypoints.



12.3.1 Waypoint Database

The waypoint database can be generated using the following criteria:

- **Mission:** the vessel should move from some starting point (x_o, y_o, z_o) to the terminal point (x_n, y_n, z_n) via the waypoints (x_i, y_i, z_i).
- **Environmental data:** information about wind, waves, and currents can be used for energy optimal routing (or avoidance of bad weather for safety reasons)
- **Collision avoidance:** avoiding moving vessels close to your own route by introducing safety margins.
- **Feasibility:** each waypoint must be feasible; in that it must be possible to maneuver to the next waypoint without exceeding maximum speed and turning rate.
- **Geographical data:** information about shallow waters, islands etc. should be included.
- **Obstacles:** floating constructions and other obstacles must be avoided.



https://www.marinetraffic.com

12.3.3 Straight-Line Paths based on Circles or Spheres of Acceptance

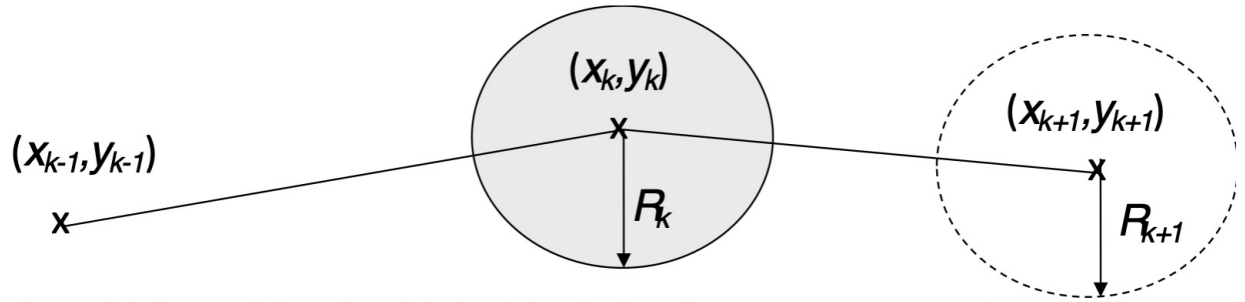
An effective method for computer implementation is to use straight-line segments going through the waypoints. When moving along a piecewise linear path made up of n-1-line segments connected by n waypoints, a switching mechanism for selecting the next waypoint is needed.

Circle of Acceptance

When moving along the path a *switching mechanism for selecting the next waypoint* is needed. Waypoint (x_{i+1}, y_{i+1}) can be selected on a basis of whether the craft lies within a circle of acceptance with radius R_i around waypoint (x_i, y_i). If

$$(x_{i+1}^n - x^n)^2 + (y_{i+1}^n - y^n)^2 \leq R_{i+1}^2$$

the next waypoint (x_{i+1}, y_{i+1}) should be selected (*i* should be incremented to *i=i+1*).

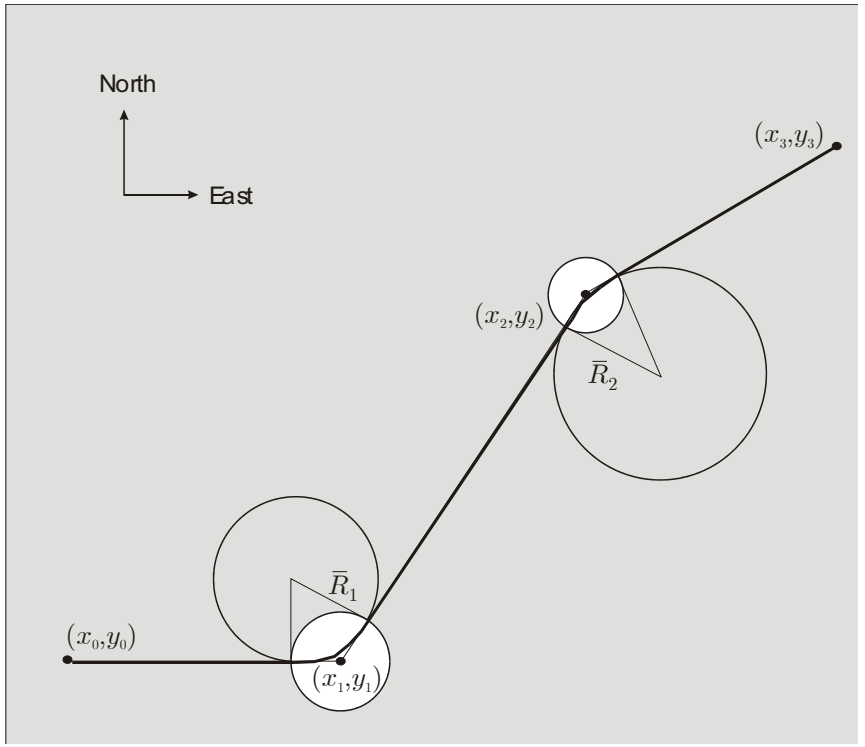


If the vehicle position (*x, y*) is inside circle, choose next waypoint.

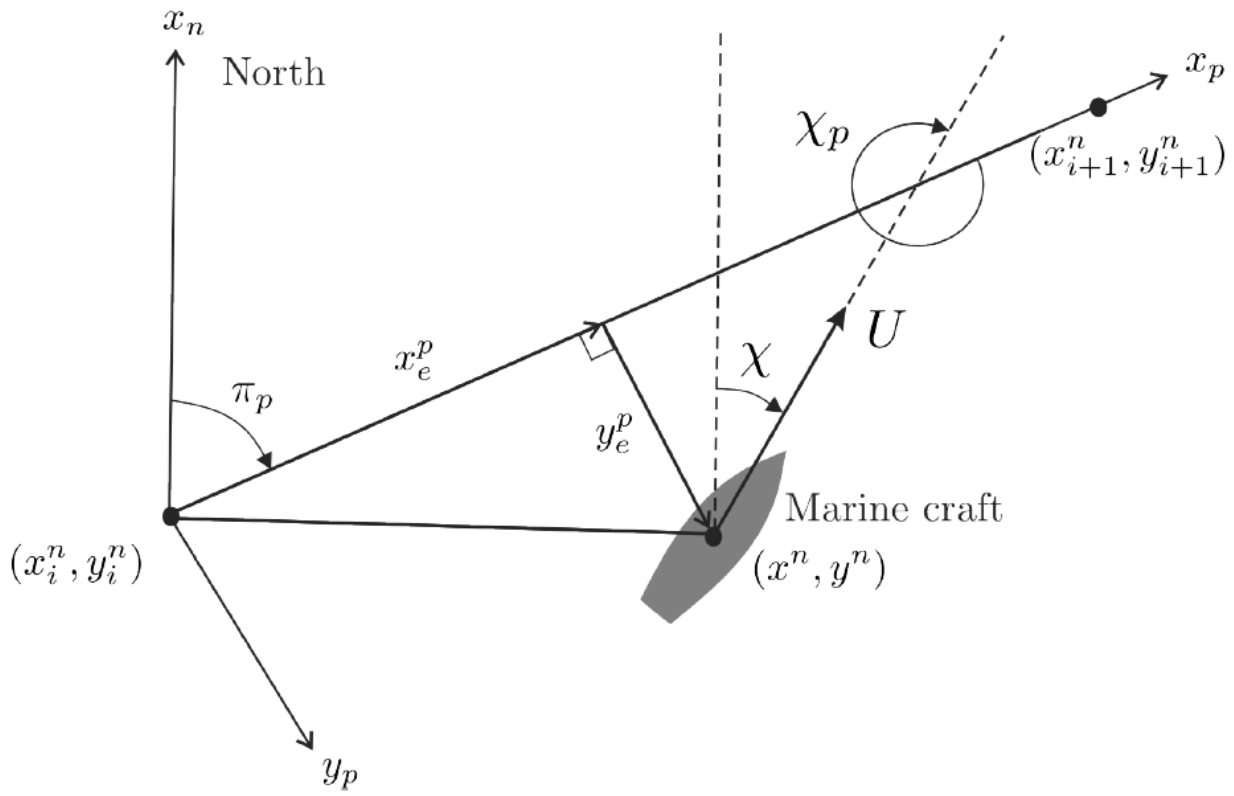
Sphere of Acceptance

This extends to 3-D by replacing the circle with a sphere according to

$$(x_{i+1}^n - x^n)^2 + (y_{i+1}^n - y^n)^2 + (z_{i+1}^n - z^n)^2 \leq R_{i+1}^2$$



12.3.3 Straight-Line Paths based on Circles of Acceptance



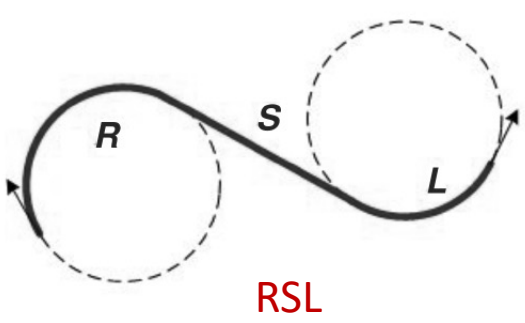
$$\pi_p = \text{atan2}(y^n_{i+1} - y^n_i,\, x^n_{i+1} - x^n_i)$$

$$\begin{bmatrix} x^p_e \\ y^p_e \end{bmatrix} = \boldsymbol{R}^n_p(\pi_p)^\top \left(\begin{bmatrix} x^n \\ y^n \end{bmatrix} - \begin{bmatrix} x^n_i \\ y^n_i \end{bmatrix} \right)$$

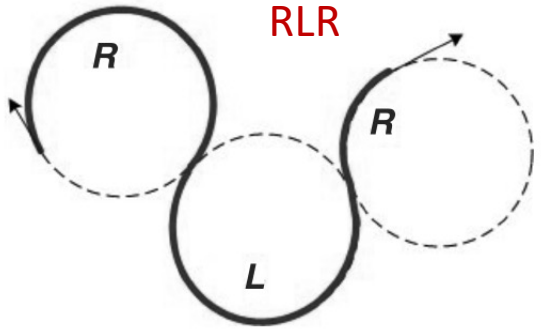
A perhaps more suitable switching criterion solely involves computation of the along-track distance, such that if the total along-track distance

$$d_{i+1} - |x^p_e| \leq R_{\text{switch}} \qquad R_{\text{switch}} > 0 \qquad d_{i+1} = \|\boldsymbol{p}^n_{i+1} - \boldsymbol{p}^n_i\|$$

12.3.3 Path Generation using Dubins Path



RSL



RLR

The desired yaw rate along the straight line is $r_d = 0$ while r_d = constant on the circle arc during steady turning.

The shortest path (minimum time) between two poses (x_i^n, y_i^n, ψ_i) *and* $(x_{i+1}^n, y_{i+1}^n, \psi_{i+1})$ *of a craft moving at constant speed* U *is a path formed by straight lines and circular arc segments.*

The drawback, in comparison to a cubic interpolation strategy, is that a jump in the desired yaw rate r_d is experienced.

The optimal path will be a combination of a right turn (R), left turn (L) or going straight (S). An optimal path will always be at least one of the six types: RSR, RSL, LSR, LSL, RLR, LRL.

The so-called Dubins path can also be proven by using Pontryagin's maximum principle.

In the case of time-varying speed, a dynamic optimization problem including the AUV dynamics must be solved.

L. E. Dubins (1957). On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents". American Journal of Mathematics. 79 (3): 497–516

Path-Planning Methods: Advantages and Disadvantages

Straight Lines Going Through Waypoints

Pros:

- **Intuitive and Easy to Implement:** The concept of connecting waypoints with straight lines is straightforward and can be quickly coded.
- **Simple Computation:** Requires minimal computational resources compared to other methods.

Cons:

- **Switching Problem:** Introduces abrupt changes (steps) in turning rate and curvature at waypoints, leading to tracking errors.
- **Obstacle Handling:** Obstacles can be managed using replanning, but this approach can be awkward and inefficient.

Dubins Paths

Pros:

- **Minimum Distance:** Provides the shortest path between two points with a constant speed, making it efficient in terms of travel distance.
- **Ease of Implementation:** Relatively easy to implement with well-established algorithms.

Cons:

- **Turning Rate/Curvature Steps:** Similar to straight lines, Dubins paths can introduce abrupt changes in turning rate and curvature.
- **Not Optimal for Time-Varying Speed:** Requires solving a minimum time optimization problem for varying speeds, which can be complex.
- **Obstacle Handling:** Modifications such as Pythagorean Hodographs can handle obstacles but add complexity.

Dynamic Optimization

Pros:

- **Comprehensive Solutions:** Can solve both minimum time and minimum distance problems while considering obstacle avoidance.
- **Flexibility:** Capable of adapting to various constraints and objectives.

Cons:

- **Implementation Effort:** Requires purchasing and implementing a solver, which can be costly and time-consuming.
- **Fallback Strategy:** A fallback strategy is necessary when the solver fails to find a solution.

Lecture Notes: Marine Craft Hydrodynamics and Motion Control (T. I. Fossen)

12.3.5 Transfer Function Models for Straight-Line Path Following

Assume that the path-tangential coordinate system *{p}* has its origin at the waypoint $(x_i^n,\ y_i^n)$
The cross-track error expressed in *{p} satisfies*

$$\begin{aligned} \dot{y}_e^p &= U \sin(\chi_p) \\ &\approx U \chi_p \end{aligned}$$

GNSS course over ground (COG) measurement

$$\chi = \pi_p + \chi_p$$

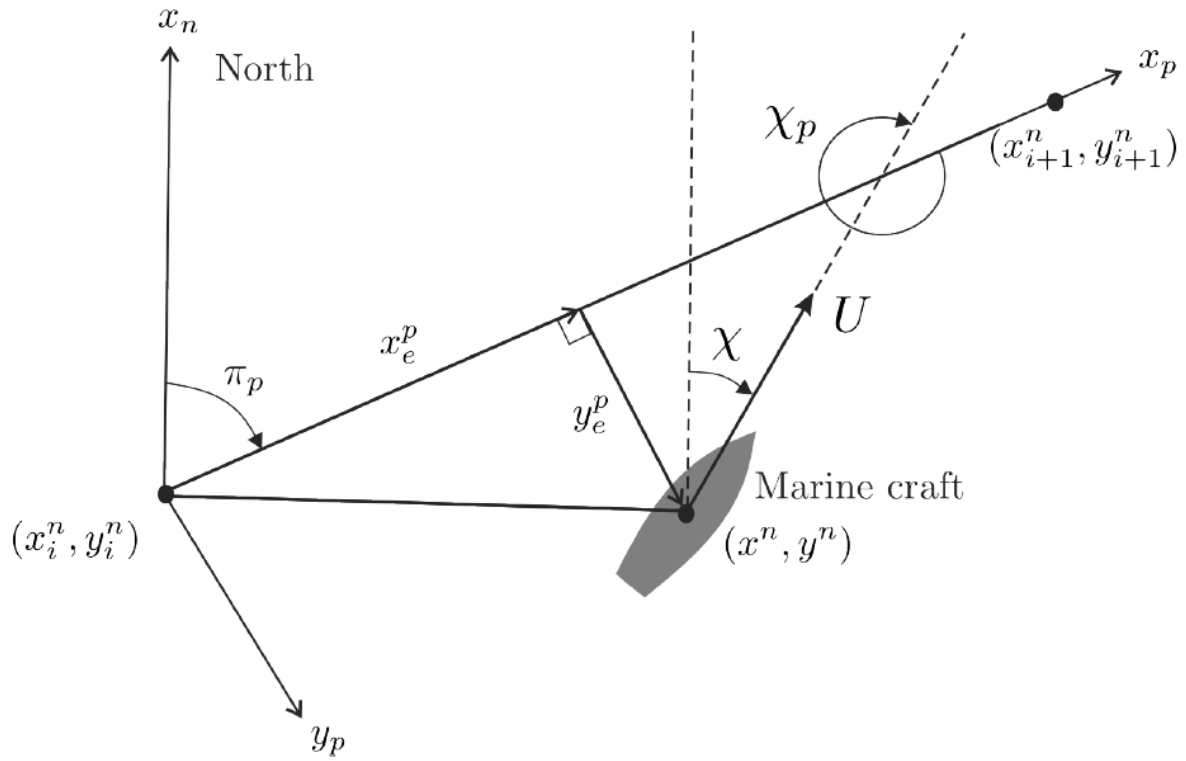
Transfer functions

$$y_e^p = \frac{U}{s}(\chi - \pi_p)$$

$$\psi = \frac{K}{s(Ts+1)}\delta + \frac{1}{s}d_r \qquad \text{Nomoto model}$$

Cross-track error transfer function for PID control design

$$y_e^p = \frac{KU}{s^2(Ts+1)}\delta + d_y \qquad d_y = \frac{U}{s}(d_\chi - \pi_p)$$



Linear design: The transfer function can be represented in a Bode diagram, where the PID control law for δ is tuned to regulate the cross-track error y_e to zero and suppress the disturbance d_y.

LOS Guidance Laws for Path Following using Course Autopilots

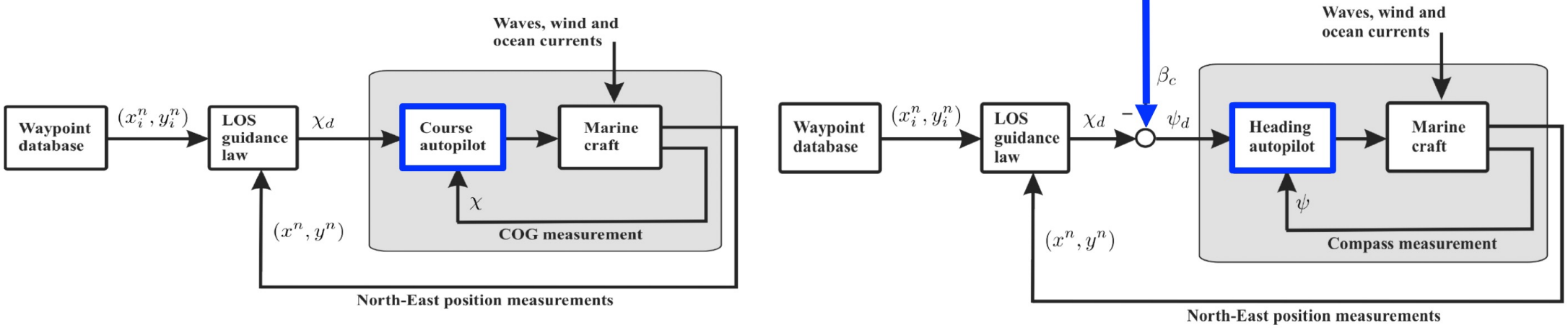
Recap: Course and Heading Autopilots

It is important to stress the concepts for course and heading control since there are many conceptual misunderstandings regarding the course and heading of an aircraft and marine craft.

- The course angle χ of a vehicle is the cardinal direction in which the vehicle is moving $\chi = \psi + \beta_c$
- The heading angle ψ, is the direction the craft's bow (x_b axis) is pointed

The difference between the course and heading angles is the horizontal crab angle $\beta_c = \sin^{-1}\left(\frac{v}{U}\right)$

If your vehicle is exposed to an environmental force, the sway velocity *v* will be non-zero. This gives a non-zero horizontal crab angle β_c and the vehicle is said to **sideslip**.



Recap: NED Differential Equations

Euler Angle Rotation Matrix Representation

$$\dot{\boldsymbol{p}}^n = \boldsymbol{R}_b^n \boldsymbol{v}^b$$

$$\boldsymbol{p}^n = [x^n, y^n, z^n]^\top$$

$$\boldsymbol{v}^b = [u, v, w]^\top$$

$$\boldsymbol{R}_b^n = \begin{bmatrix} c\psi c\theta & -s\psi c\phi + c\psi s\theta s\phi & s\psi s\phi + c\psi c\phi s\theta \\ s\psi c\theta & c\psi c\phi + s\phi s\theta s\psi & -c\psi s\phi + s\theta s\psi c\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix}$$

$$\begin{aligned}
\dot{x}^n &= u\cos(\psi)\cos(\theta) \\
&\quad + v\left(\cos(\psi)\sin(\theta)\sin(\phi) - \sin(\psi)\cos(\phi)\right) \\
&\quad + w\left(\sin(\psi)\sin(\phi) + \cos(\psi)\cos(\phi)\sin(\theta)\right) \\
\dot{y}^n &= u\sin(\psi)\cos(\theta) \\
&\quad + v\left(\cos(\psi)\cos(\phi) + \sin(\phi)\sin(\theta)\sin(\psi)\right) \\
&\quad + w\left(\sin(\theta)\sin(\psi)\cos(\phi) - \cos(\psi)\sin(\phi)\right) \\
\dot{z}^n &= -u\sin(\theta) \\
&\quad + v\cos(\theta)\sin(\phi) \\
&\quad + w\cos(\theta)\cos(\phi)
\end{aligned}$$

Spherical Amplitude-Phase Representation

$$\begin{bmatrix} \dot{x}^n \\ \dot{y}^n \\ \dot{z}^n \end{bmatrix} = \begin{bmatrix} U_h\cos(\psi + \beta_c) \\ U_h\sin(\psi + \beta_c) \\ -U\sin(\theta - \alpha_c) \end{bmatrix}$$

Control inputs:

- Roll angle
- Yaw angle
- Speed

Course angle

$$\chi = \psi + \beta_c$$

Horizontal speed

$$U_h := U\cos(\gamma)$$

Flight-path angle

$$\gamma := \theta - \alpha_c$$

Vertical and horizontal crab angles

$$\alpha_c = \theta - \sin^{-1}\left(\frac{u\sin(\theta) - [v\sin(\phi) + w\cos(\phi)]\cos(\theta)}{U}\right)$$

$$\beta_c = \tan^{-1}\left(\frac{v\cos(\phi) - w\sin(\phi)}{u\cos(\theta) + [v\sin(\phi) + w\cos(\phi)]\sin(\theta)}\right)$$

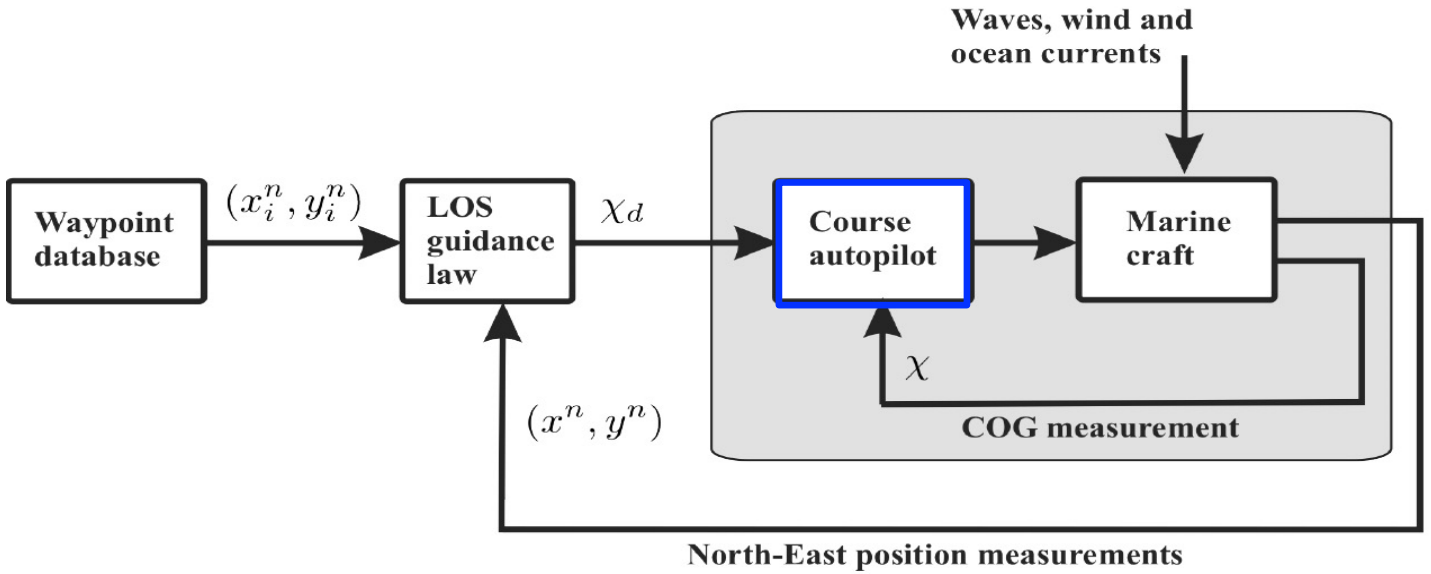
Coates, E. M. and T. I. Fossen (2025). A Spherical Amplitude-Phase Formulation for 3-D Adaptive Line-of-Sight (ALOS) Guidance with USGES Stability Guarantees. Submitted to Automatica. Open Access arxiv.org/abs/2505.08344

Coates, E. M. and T. I. Fossen (2025). A Spherical Amplitude-Phase Formulation for 3-D Adaptive Line-of-Sight (ALOS) Guidance with USGES Stability Guarantees. Submitted to Automatica. Open Access arxiv.org/abs/2505.08344

12.4 LOS Guidance Laws for Path Following using Course Autopilots

Path following is the task of following a predefined path independent of time; that is there are no temporal constraints.

No restrictions are placed on the temporal propagation along the path. Spatial constraints, however, can be added to represent obstacles and other positional constraints.



The Course Over Ground (COG) can also be estimated from position or latitude-longitude measurements

The LOS guidance laws that can be used together with commercial course autopilots under the assumption that $\chi = \chi_d$

12.4.1 Vector Field Guidance Law

Vector field guidance law (Nelson et al. 2007)

$$\chi_d = \pi_p - \chi^\infty \frac{2}{\pi} \tan^{-1}(K_p\, y_e^p)$$

When $|y_e^p|$ is large, the craft is directed to approach the path with

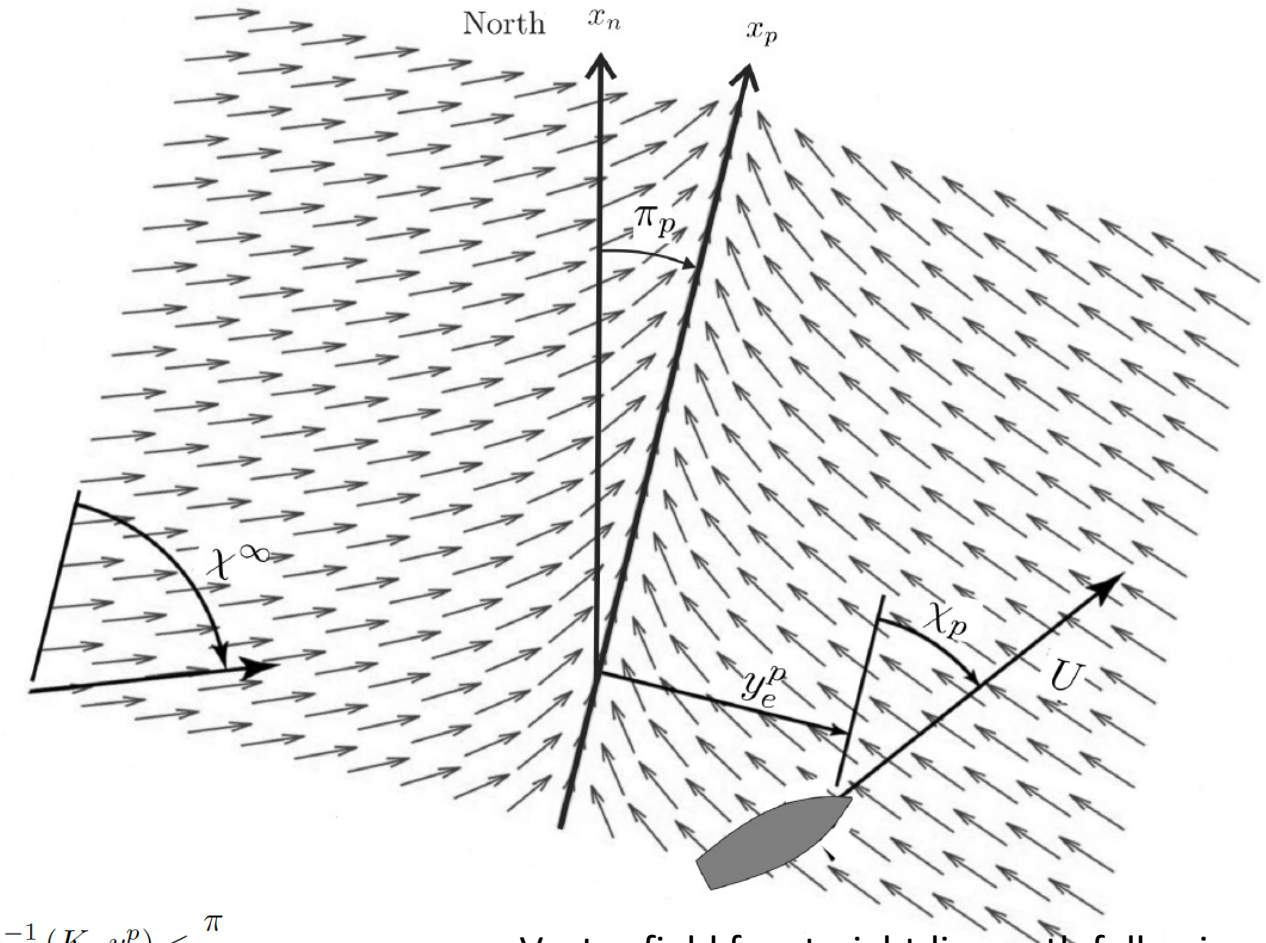
$$\chi_p = \pm\chi^\infty \text{ for } \chi^\infty \in (0, \pi/2]$$

angle whereas when y_e^p approaches zero, the angle $\chi_p \to 0$

Lyapunov stability analysis

$$\dot{y}_e^p = U \sin(\chi - \pi_p)$$
$$= -U \sin\left(\chi^\infty \frac{2}{\pi} \tan^{-1}(K_p\, y_e^p)\right)$$
$$V = 1/2(y_e^p)^2$$
$$\dot{V} = -y_e^p\, U \sin\left(\chi^\infty \frac{2}{\pi} \tan^{-1}(K_p\, y_e^p)\right) \qquad -\frac{\pi}{2} < \chi^\infty \frac{2}{\pi} \tan^{-1}(K_p\, y_e^p) < \frac{\pi}{2}$$
$$< 0, \qquad \forall y_e^p \neq 0$$

Hence, the equilibrium point $y_e^p = 0$ is GAS



Vector field for straight-line path following

D. R. Nelson, B. Barber, T. W. McLain and R. W. Beard (2007). Vector Field Path Following for Miniature Air Vehicles IEEE Trans. on Robotics 33(3), 519–529.

12.4.2 Proportional LOS Guidance Law

The vector-field guidance law is almost similar to a class of proportional LOS guidance laws used for marine craft path-following control. The difference is that we choose $\chi^{\infty} = \pi/2$ such that

$$\chi_d = \pi_h - \tan^{-1}\left(K_p y_e^p\right) \qquad K_p = \frac{1}{\Delta}$$

The desired course rate $\dot{\chi}_d = \omega_{\chi_d}$ is

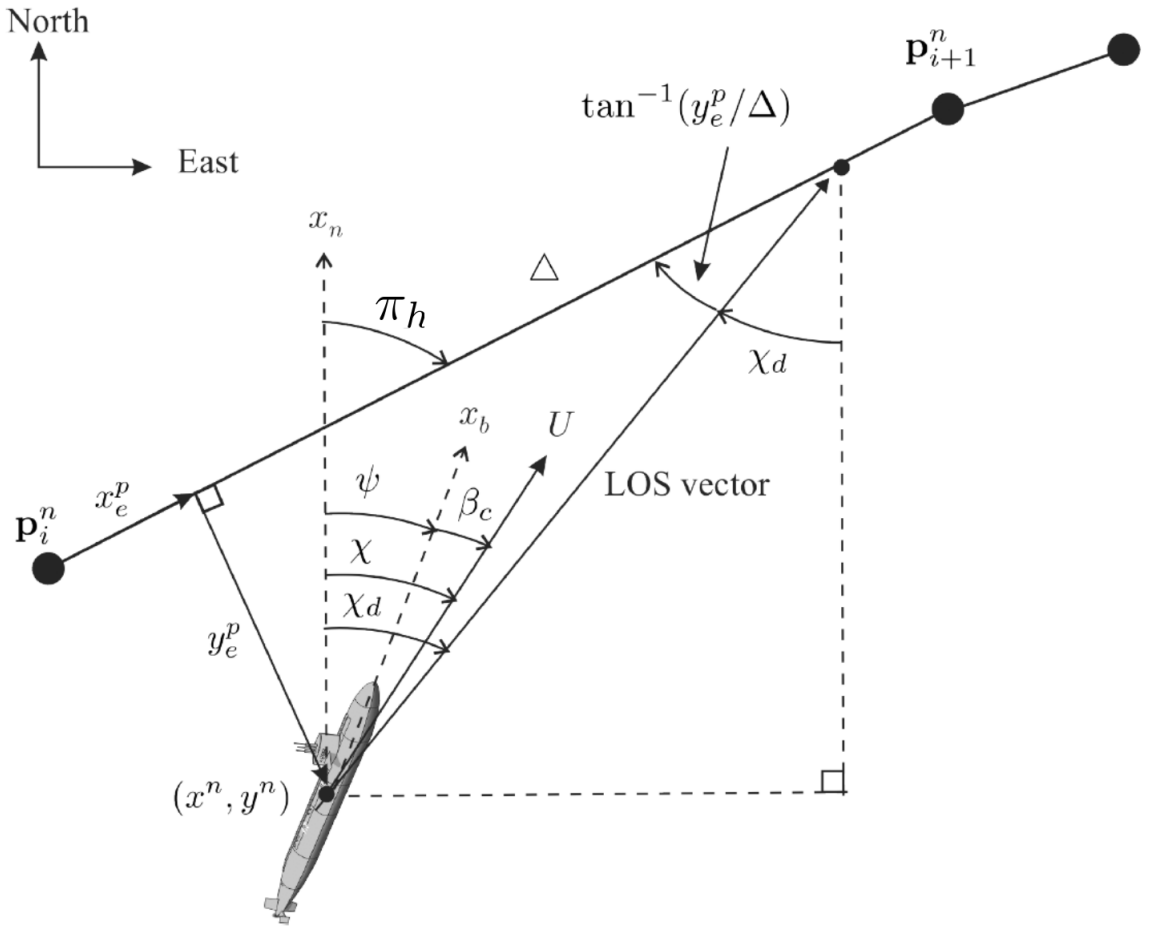
$$\omega_{\chi_d} = -\frac{K_p \dot{y}_e}{(K_p y_e^p)^2 + 1} = -\frac{K_p U \sin(\chi - \pi_h)}{(K_p y_e^p)^2 + 1}$$

Cross-track error dynamics

$$\begin{aligned}\dot{y}_e^p &= U \sin(\chi_p) \\ &= U \sin(\chi - \pi_h) \\ &= -U \sin\left(\tan^{-1}\left(y_e^p/\Delta\right)\right) \\ &= -\frac{U}{\sqrt{\Delta^2 + (y_e^p)^2}}\, y_e^p\end{aligned}$$

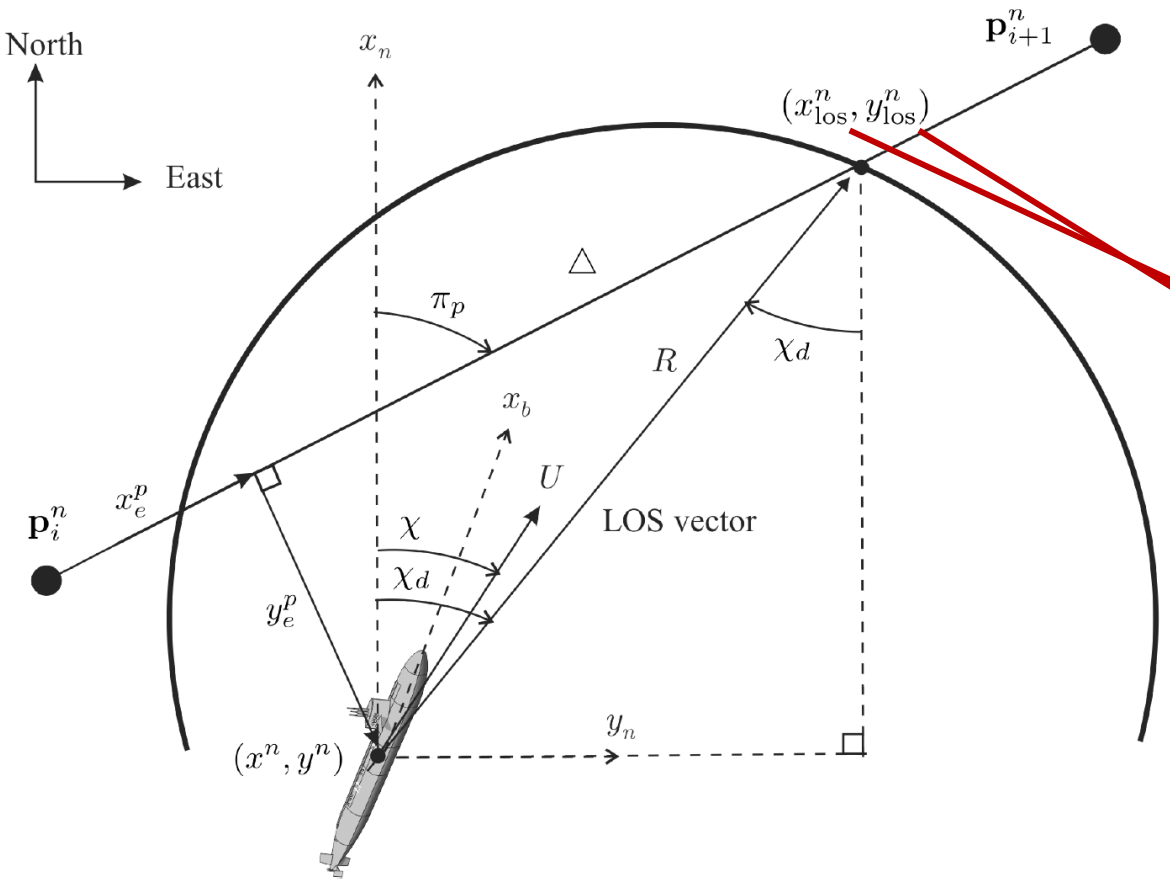
The equilibrium point $y_e^p = 0$ is USGES. We cannot prove GES!

$$V = \frac{1}{2}(y_e^p)^2 \quad \Rightarrow \quad \dot{V} = -\frac{U}{\sqrt{\Delta^2 + (y_e^p)^2}} y_e^p$$



Fossen, T. I. and K. Y. Pettersen (2014). On Uniform Semiglobal Exponential Stability (USGES) of Proportional LOS Guidance Laws. Automatica 50, 2912–2917.

12.4.3 Lookahead- and Enclosure-Based LOS Steering



The proportional LOS guidance laws can be classified according to (Breivik and Fossen 2009)

- Lookahead-based steering

$$\chi_d = \pi_h - \tan^{-1}\left(K_p y_e^p\right) \qquad K_p = \frac{1}{\Delta}$$

- Enclosure-based steering

$$\chi_d = \text{atan2}\left(y_{\text{los}}^n - y^n, x_{\text{los}}^n - x^n\right)$$

$$\Delta(t) = \sqrt{R^2 - y_e^p(t)^2}$$

The two steering methods essentially operate by the same principle, but as will be made clear, the lookahead-based approach motivated by missile guidance has several advantages over the enclosure-based approach.

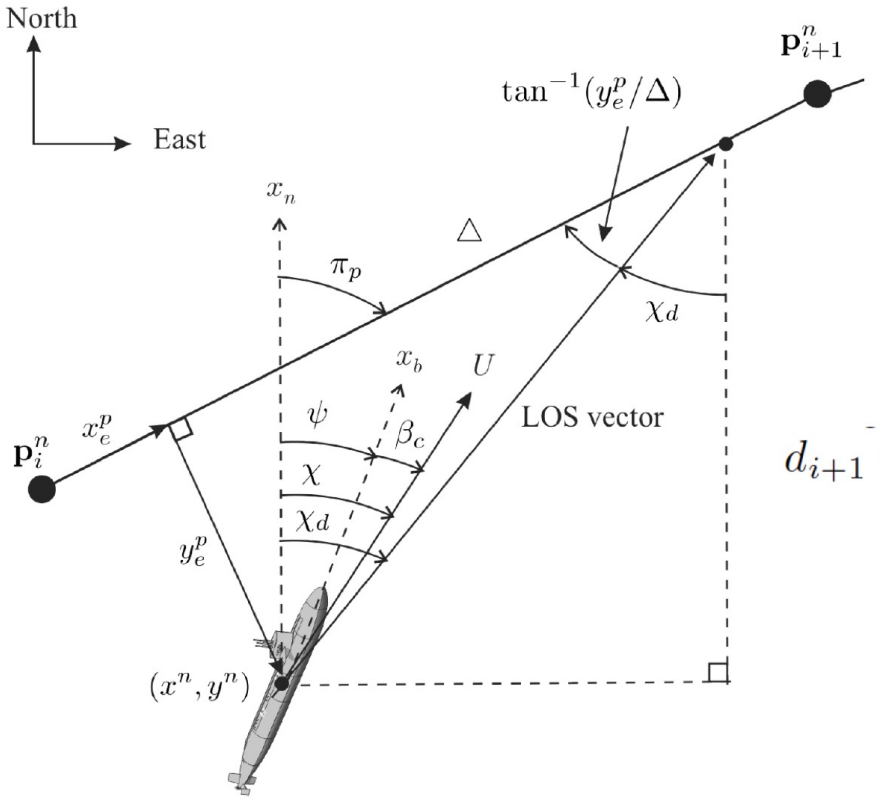
MSS Toolbox: LOSchi.m

```
x = 0; y= 0; Delta_h = 50; R_switch = 5;
wpt.pos.x = [0 100 200]';
wpt.pos.y = [0 100 200]';
chi_d = LOSchi(x, y, Delta_h, R_switch, wpt)
chi_d =
    0.7854
```

$$K_p = \frac{1}{\Delta}$$

$$\chi_d = \pi_h - \tan^{-1}\left(K_p y_e^p\right)$$

Switch to next waypoint if $d_{i+1} - |x_e^p| \leq R_{\text{switch}}$



$$d_{i+1} = \|\boldsymbol{p}_{i+1}^n - \boldsymbol{p}_i^n\|$$

```
function [chi_ref, y_e] = LOSchi(x, y, Delta_h, R_switch, wpt)
```

```
% [chi_ref, y_e]  = LOSchi(x, y, Delta_h, R_switch, wpt)
% LOSchi computes the desired course angle, chi_ref, and cross-track
% error, y_e, for paths that consist of straight lines connecting waypoints
% (wpt.pos.x, wpt.pos.y). The desired course angle is computed using the
% adaptive LOS (ALOS) guidance law
%
%  chi_ref[k] = pi_h - atan( y_e[k] / Delta_h )
%
% where pi_h is the path-tangential (azimuth) angle with respect to the
% North axis and y_e is the cross-track error. To handle steps in the course
% angle command, chi_ref, due to waypoint switching, the following observer
% should be used:
%
%  [chi_d, omega_chi_d] = LOSobserver(chi_d omega_chi_d,chi_ref,h,K_f,T_f)
%
% where chi_d is the desired course angle and omega_chi_d isthe desired
% course rate.
%
% Initialization:
%   The active waypoint (xk, yk) where k = 1,2,...,n is a persistent
%   integer should be initialized to the first waypoint, k = 1, using
%   >> clear LOSchi
%
% Inputs:
%   (x,y):    craft North-East positions (m)
%   Delta_h:  positive look-ahead distance (m)
%   R_switch: go to next waypoint when the along-track distance x_e
%             is less than R_switch (m)
%   wpt.pos.x = [x1, x2,..,xn]': array of waypoints expressed in NED (m)
%   wpt.pos.y = [y1, y2,..,yn]': array of waypoints expressed in NED (m)
%
% Feasibility citerion:
%   The switching parameter R_switch > 0 must satisfy, R_switch < dist,
%   where dist is the distance between the two waypoints at k and k+1:
%     dist = sqrt(    ( wpt.pos.x(k+1) - wpt.pos.x(k) )^2
%                   + ( wpt.pos.y(k+1) - wpt.pos.y(k) )^2 )
% Outputs:
%    chi_ref: LOS course angle (rad)
%    y_e:     cross-track error (m)
```

12.4.3 Lookahead- and Enclosure-Based LOS Steering

The enclosure-based strategy for driving y_e^p to zero is then to direct the velocity toward the intersection point

$$\boldsymbol{p}_{\text{los}}^n = [x_{\text{los}}^n, y_{\text{los}}^n]^\top$$

that corresponds to the desired direction of travel. We can compute the intersection point from the following two equations

$$(x_{\text{los}}^n - x^n)^2 + (y_{\text{los}}^n - y^n)^2 = R^2$$

$$\tan(\pi_p) = \frac{y_{i+1}^n - y_i^n}{x_{i+1}^n - x_i^n} = \frac{y_{\text{los}}^n - y_i^n}{x_{\text{los}}^n - x_i^n}$$

$$\boldsymbol{p}_{\text{los}}^n = [x_{\text{los}}^n, y_{\text{los}}^n]^\top$$

These equations can be solved numerically or analytically for

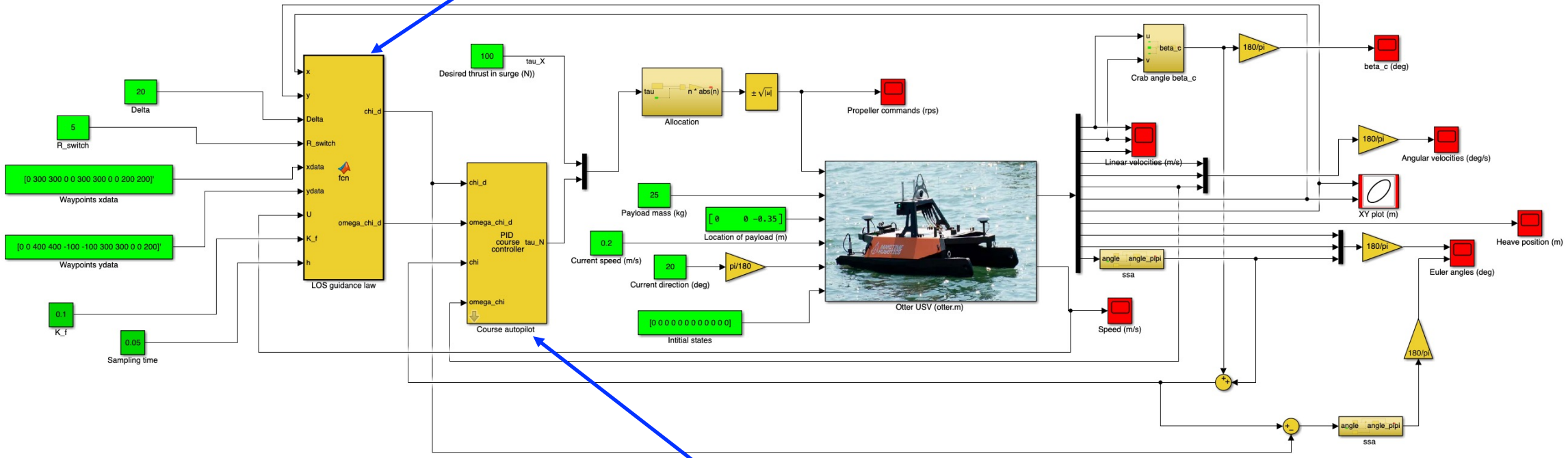
As can be immediately noticed, the lookahead-based steering scheme is less computationally intensive than the enclosure-based approach. It is also valid for all cross-track errors, whereas the enclosure-based strategy requires

$$R > |y_e^p|$$

MSS Simulink: demoOtterUSVPathFollowingCourseControl.slx

Simulink demo calling the MSS Matlab function LOSchi.m for LOS path-following control using a course autopilot

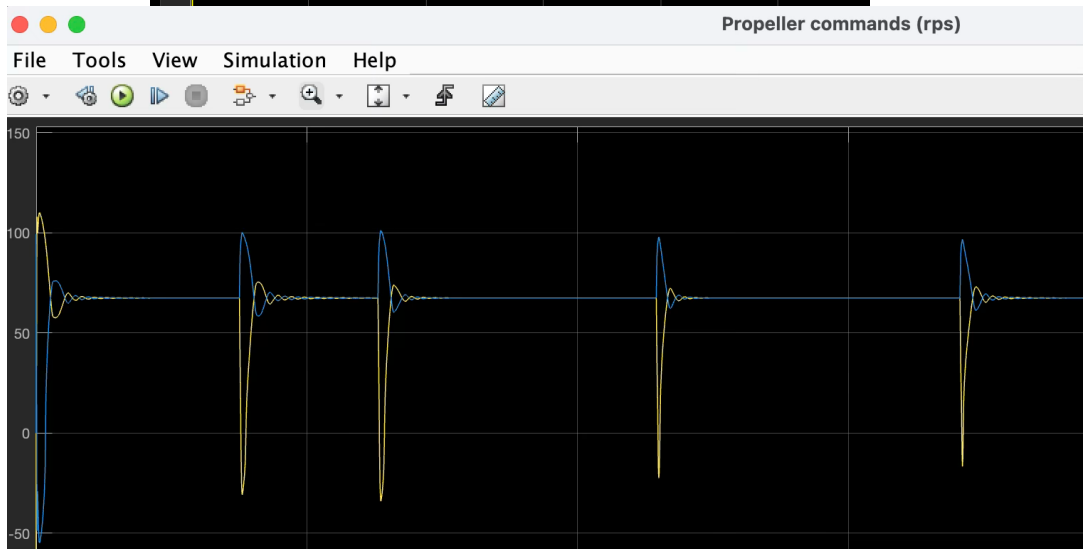
LOS guidance law for course autopilot

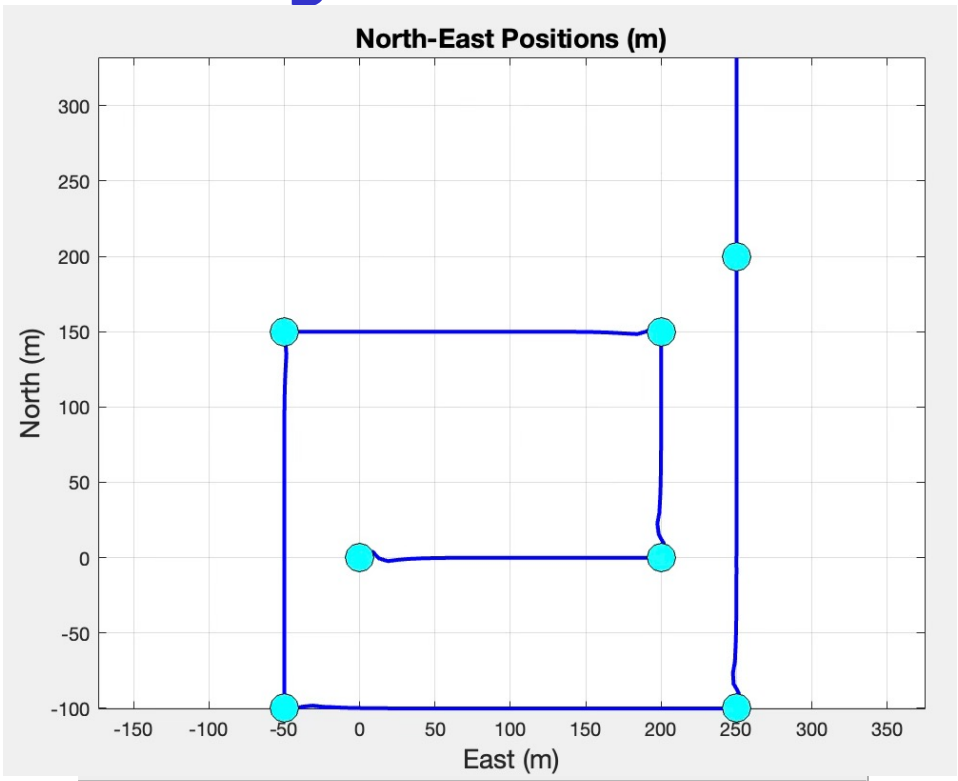


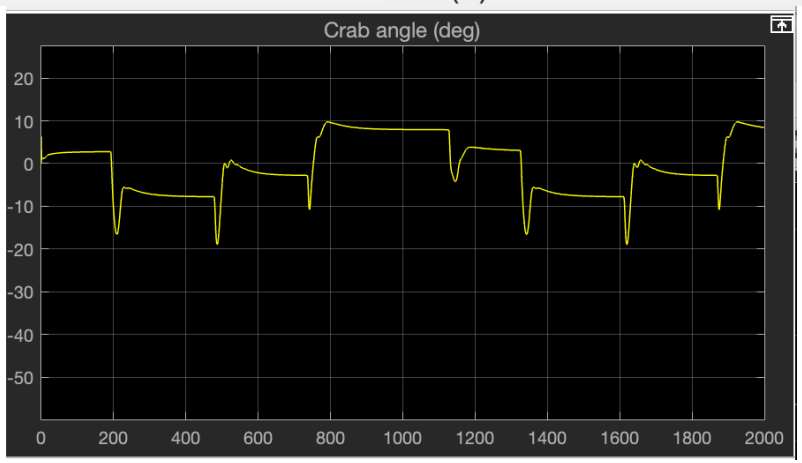
Course autopilot using course angle and course rate

MSS Simulink: demoOtterUSVPathFollowingCourseControl.slx









LOS Guidance Laws for Path Following using Heading Autopilots

12.5 LOS Guidance Laws for Path Following using Heading Autopilots

- At zero speed ($u = 0$), the course angle χ is not well defined
- Marine craft use compass measurements to measure the heading angle ψ, which is well defined at zero speed

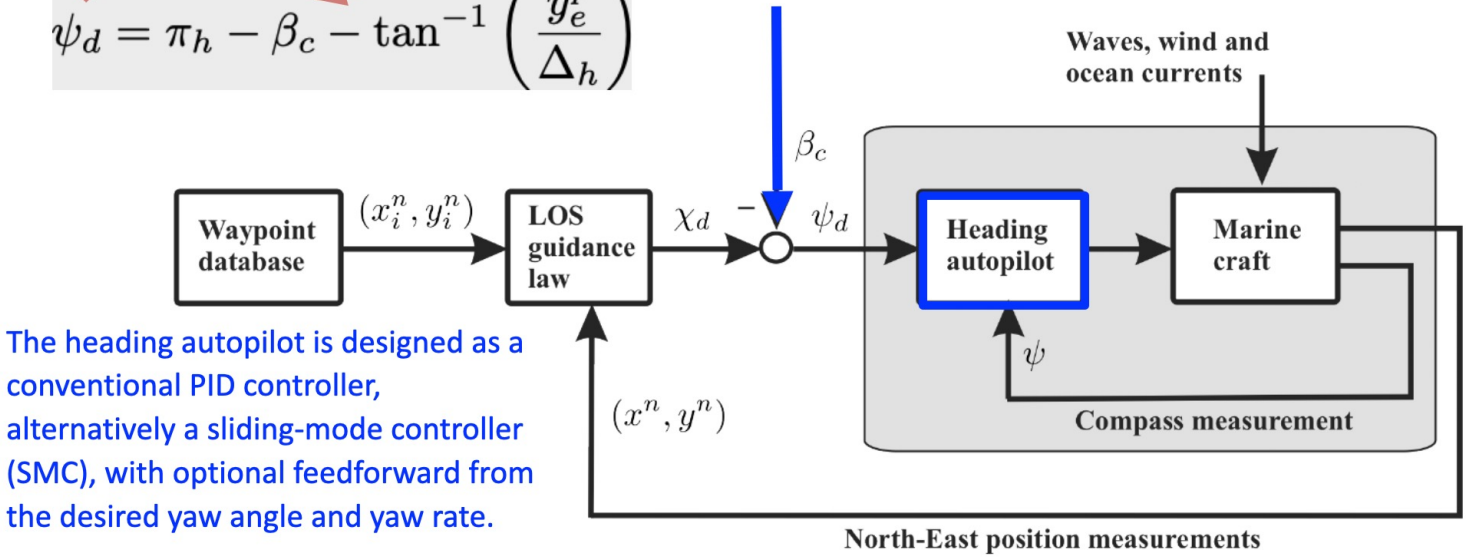
$$\dot{y}_e^p = U \sin(\psi + \beta_c - \pi_h)$$

$$\chi = \psi + \beta_c$$

$$\beta_c = \text{atan2}(v, u)$$

When designing the heading autopilot, the horizontal crab angle is treated as a disturbance

$$\psi_d = \pi_h - \beta_c - \tan^{-1}\left(\frac{y_e^p}{\Delta_h}\right)$$



The heading autopilot is designed as a conventional PID controller, alternatively a sliding-mode controller (SMC), with optional feedforward from the desired yaw angle and yaw rate.

Horizontal crab Angle Compensation by Direct Measurements?

An estimate of the crab angle can be obtained by using velocity measurements. However, the formula for the horizontal crab angle is numerical ill-conditioned for small u and v.

12.5.1 Crab Angle due to Crosswind/Crosscurrent Component

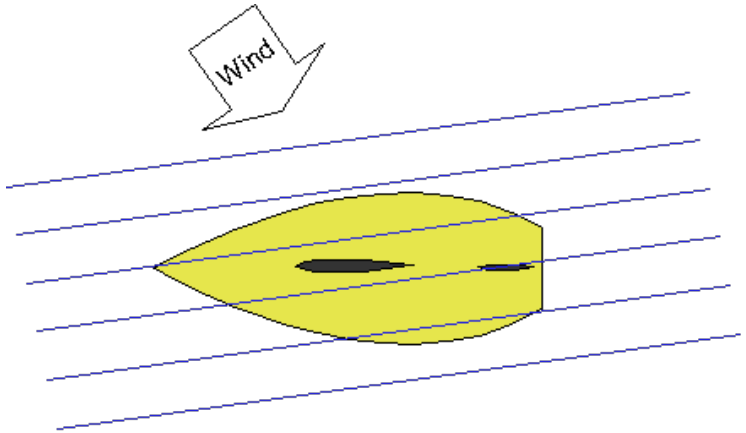
A B777 (as in the photo) would generally have a maximum crab angle of about 16 degrees when approaching and landing at the maximum demonstrated crosswind component of 38 knots.

$$\chi = \psi + \beta_c$$

$$\beta_c = \tan^{-1}\left(\frac{v}{u}\right)$$



A marine craft exposed to wind, waves and ocean currents will also have a non-zero crab angle.

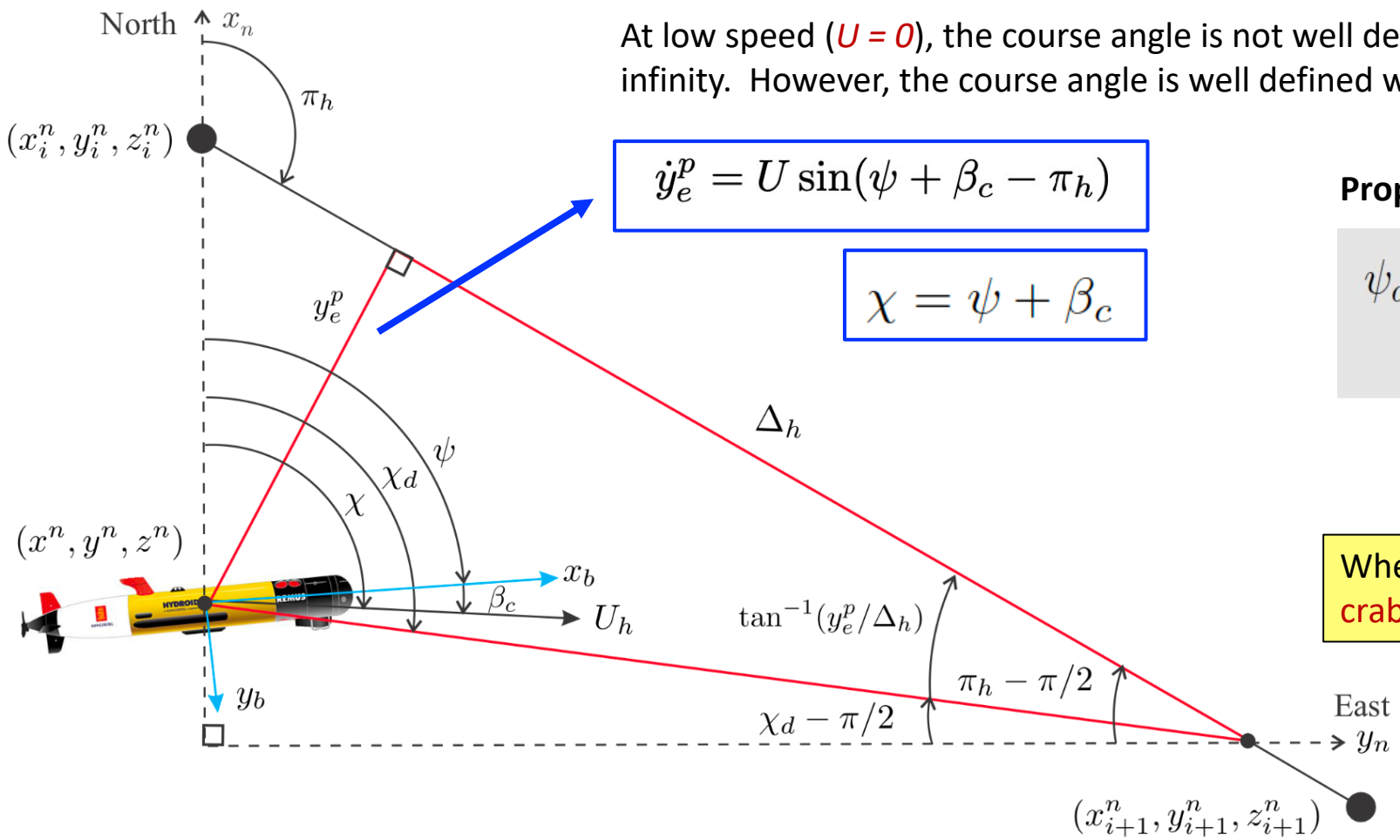


https://aviation.stackexchange.com/questions/58861/what-is-the-maximum-angle-between-an-airplane-and-runway-centerline-when-touchin

12.5.1 Proportional LOS Guidance Law for Heading Control

Marine craft often use compass measurements to measure the heading angle since they can operate at zero speed.

At low speed (*U = 0*), the course angle is not well defined since the crab angle can reach infinity. However, the course angle is well defined when moving from A to B (*U > 0*).



$$\dot{y}_e^p = U \sin(\psi + \beta_c - \pi_h)$$

$$\chi = \psi + \beta_c$$

Proportional LOS Guidance Law

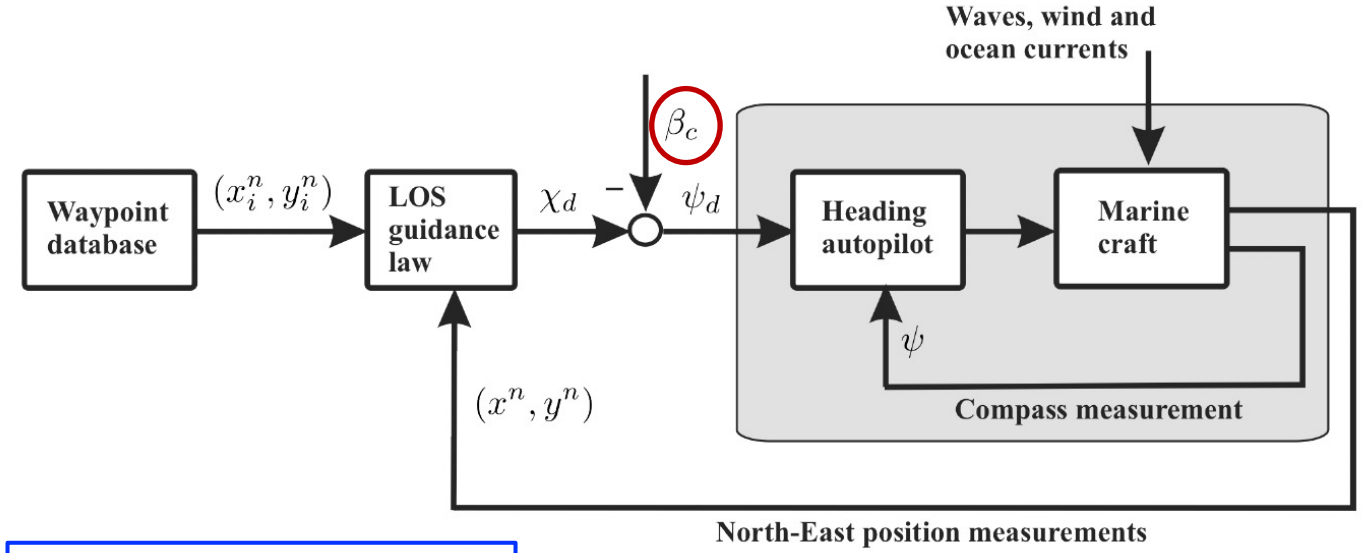
$$\begin{aligned}\psi_d &= \chi_d - \beta_c \\ &= \pi_h - \tan^{-1}(K_p y_e^p) - \beta_c\end{aligned}$$

$$\beta_c = \tan^{-1}\left(\frac{v}{u}\right)$$

When designing the heading autopilot, the crab angle is treated as a disturbance in yaw

12.5.2 Integral LOS (ILOS) for Compensation of β_c

The classical ILOS algorithm by Børhaug et al. (2008) is frequently used to compensate for the unknown drift term β_c



Børhaug et al. (2008) used Lyapunov stability theory to prove UGAS/ULES.

The stability analysis is based on a 3-DOF ship model (surge, sway and yaw) in cascade with the ILOS guidance law.

$$\dot{y}_e^p = U \sin(\psi + \beta_c - \pi_h)$$

Classical ILOS (Børhaug et al. 2008)

$$\psi_d = \pi_h - \tan^{-1}(K_p\, y_e^p + K_i\, y_{\text{int}}^p)$$

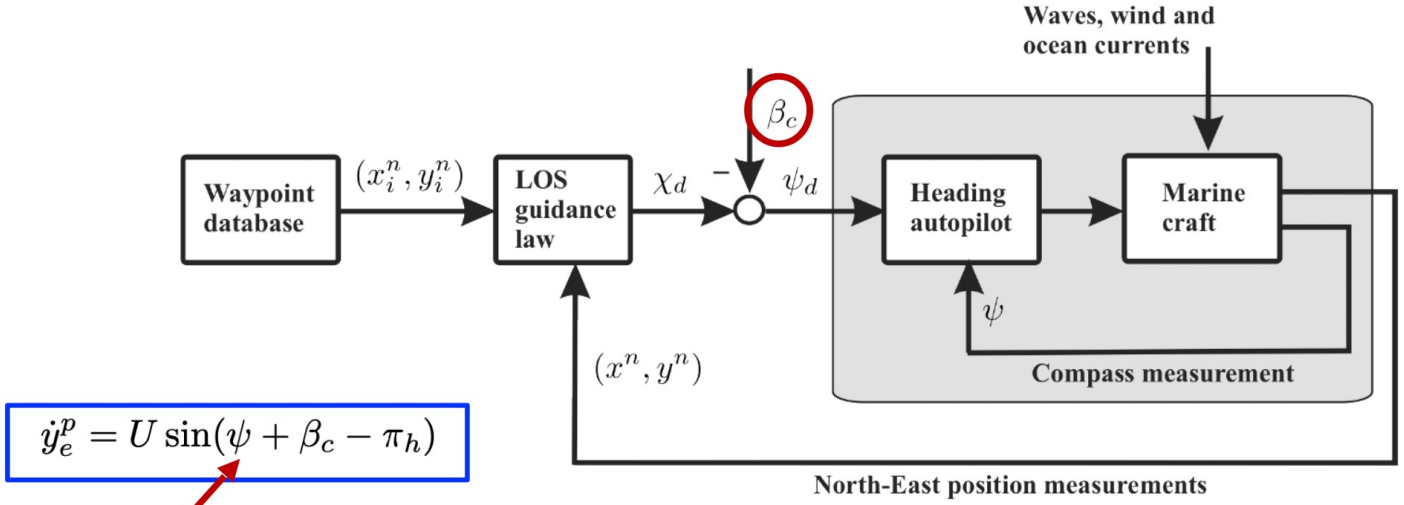
$$\dot{y}_{\text{int}}^p = \frac{\Delta y_e^p}{\Delta^2 + (y_e^p + \kappa\, y_{\text{int}}^p)^2}$$

where $K_p = 1/\Delta$, $K_i = \kappa K_p$ and $\kappa > 0$ are design parameters.

E. Børhaug, A. Pavlov and K. Y. Pettersen (2008). Integral LOS control for Path Following of Underactuated Marine Surface Vessels in the presence of Constant Ocean Currents, 47th IEEE Conference on Decision and Control (CDC), pp. 4984–4991, Cancun, Mexico.

2-D Adaptive LOS (ALOS) for Compensation of β_c

The ALOS algorithm by Fossen (2023) estimates the unknown drift term β_c as a (nearly) constant parameter and compensates β_c by direct matching. This is equivalent to adding integral action.



$$\dot{y}_e^p = U \sin(\psi + \beta_c - \pi_h)$$

2-D ALOS (Fossen 2023)

$$\psi_d = \pi_h - \hat{\beta}_c - \tan^{-1}\left(\frac{y_e^p}{\Delta}\right)$$

$$\dot{\hat{\beta}}_c = \gamma \frac{\Delta}{\sqrt{\Delta^2 + (y_e^p)^2}} y_e^p$$

The stability proof guarantees that the equilibrium point is USGES.

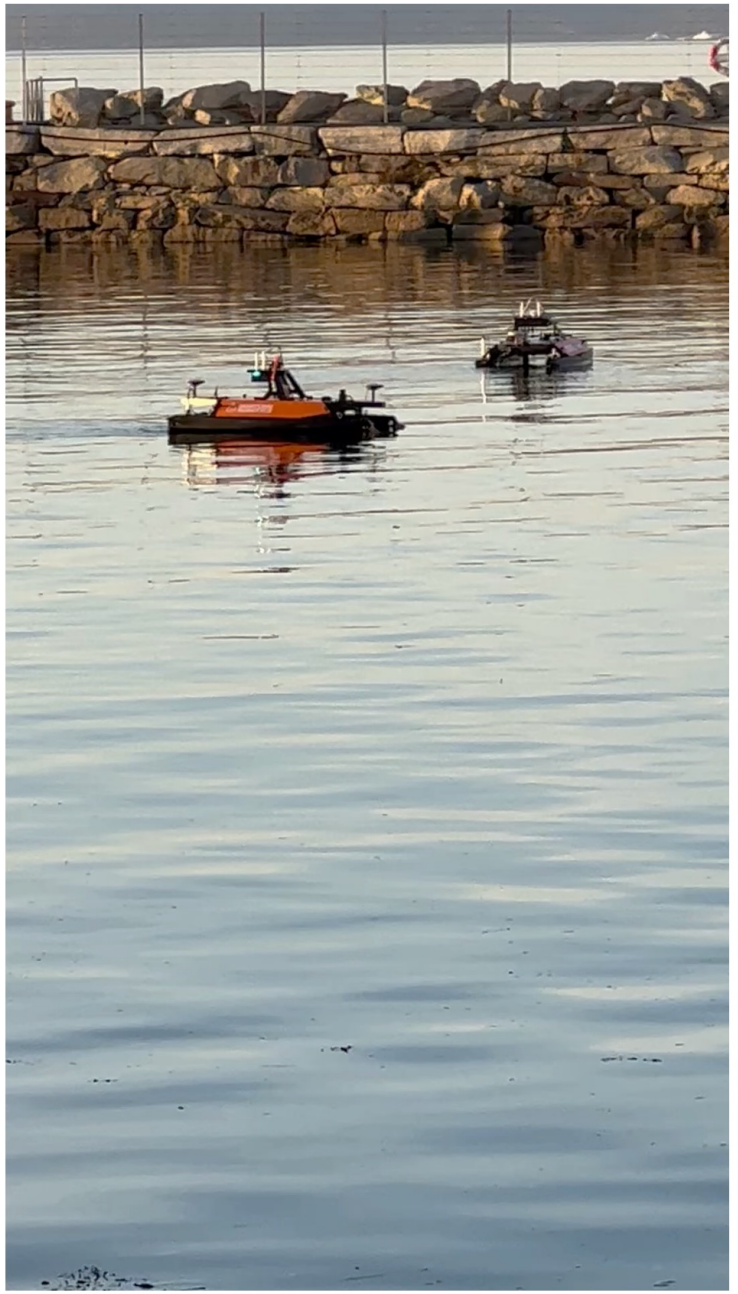
T. I. Fossen (2023). An Adaptive Line-of-sight (ALOS) Guidance Law for Path Following of Aircraft and Marine Craft. *IEEE Transactions on Control Systems Technology* 31(6), 2887-2894.

Case Study

Maritime Robotics Otter USV

www.maritimerobotics.com





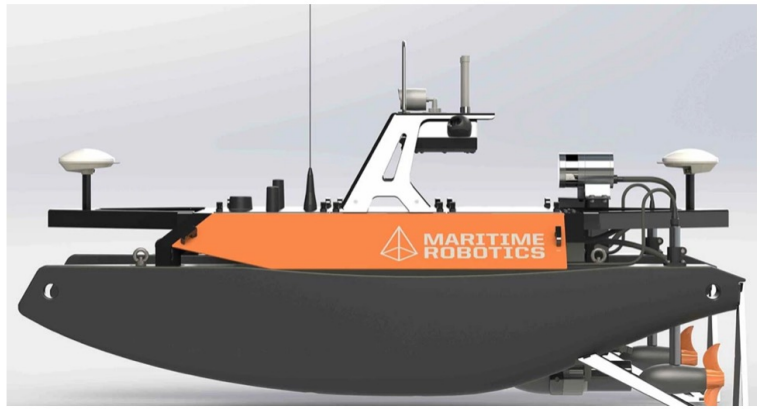
MSS Toolbox: SIMotter.m

```
% SIMotter is compatibel with MATLAB and GNU Octave (www.octave.org).
% This script simulates the Otter Uncrewed Surface Vehicle (USV) under
% various control strategies to handle path following in the presence of
% ocean currents. This script allows the user to select from several control
% methods and simulatesthe USV's performance using a cubic Hermite spline
% or straight-line paths.
%
% The simulation covers:
%   1. PID heading autopilot without path following.
%   2. Adaptive Line-of-Sight (ALOS) control for path following using
%      straight lines and waypoint switching.
%   3. Integral Line-of-Sight (ILOS) control for path following using
%      straight lines and waypoint switching.
%   4. ALOS control for path following using Hermite spline interpolation.
%
% Dependencies:
%   otter.m                 - Dynamics of the Otter USV
%   refModel.m              - Reference model for autopilot systems
%   ALOSpsi.m               - ALOS guidance algorithm for path following
%   ILOSpsi.m               - ILOS guidance algorithm for path following
%   hermiteSpline.m         - Cubic Hermite spline computations
%   crosstrackHermiteLOS.m  - Cross-track error and LOS guidance law for
%                             cubic Hermite splines
%   LOSobserver.m           - Observer for LOS guidance
%   controlMethods.m        - Menu for choosing control law.
%
% Simulink Models:
%   demoOtterUSVPathFollowingHeadingControl.slx
%   demoOtterUSVPathFollowingCourseControl.slx
```

```
function [xdot,U, M, B_prop] = otter(x,n,mp,rp,V_c,beta_c)
% Compatibel with MATLAB and the free software GNU Octave (www.octave.org)
% [xdot,U] = otter(x,n,mp,rp,V_c,beta_c) returns the speed U in m/s
% (optionally), the 6x6 mass matrix M (optionally), and the 2x4 input
% matrix B_prop (optionally) in pitch and yaw and the time derivative of
% the state vector:
%
% x = [ u v w p q r x y z phi theta psi ]'
%
% for the Maritime Robotics Otter USV, see www.maritimerobotics.com.
% The length of the USV is L = 2.0 m, while the state vector is defined as:
%
%  u:     surge velocity          (m/s)
%  v:     sway velocity           (m/s)
%  w:     heave velocity          (m/s)
%  p:     roll velocity           (rad/s)
%  q:     pitch velocity          (rad/s)
%  r:     yaw velocity            (rad/s)
%  x:     position in x direction (m)
%  y:     position in y direction (m)
%  z:     position in z direction (m)
%  phi:   roll angle              (rad)
%  theta: pitch angle             (rad)
%  psi:   yaw angle               (rad)
%
% The other inputs are:
%
%  n = [ n(1) n(2) ]' where
%    n(1): propeller shaft speed, left (rad/s)
%    n(2): propeller shaft speed, right (rad/s)
%
%  mp: payload mass (kg), maximum 45 kg and minimum 0 kg
%  rp = [xp, yp, zp]' (m) is the location of the payload w.r.t. the CO
%  V_c:     ocean current speed (m/s)
%  beta_c:  ocean current direction (rad)
```

Maritime Robotics Otter USV

```
function [xdot,U, M, B_prop] = otter(x,n,mp,rp,V_c,beta_c)
% Compatibel with MATLAB and the free software GNU Octave (www.octave.org)
% [xdot,U] = otter(x,n,mp,rp,V_c,beta_c) returns the speed U in m/s
% (optionally), the 6x6 mass matrix M (optionally), and the 2x4 input
% matrix B_prop (optionally) in pitch and yaw and the time derivative of
% the state vector:
%
% x = [ u v w p q r x y z phi theta psi ]'
%
% for the Maritime Robotics Otter USV, see www.maritimerobotics.com.
% The length of the USV is L = 2.0 m, while the state vector is defined as:
%
%  u:     surge velocity          (m/s)
%  v:     sway velocity           (m/s)
%  w:     heave velocity          (m/s)
%  p:     roll velocity           (rad/s)
%  q:     pitch velocity          (rad/s)
%  r:     yaw velocity            (rad/s)
%  x:     position in x direction (m)
%  y:     position in y direction (m)
%  z:     position in z direction (m)
%  phi:   roll angle              (rad)
%  theta: pitch angle             (rad)
%  psi:   yaw angle               (rad)
```




```
% The other inputs are:
%
%  n = [ n(1) n(2) ]' where
%    n(1): propeller shaft speed, left (rad/s)
%    n(2): propeller shaft speed, right (rad/s)
%
%  mp: payload mass (kg), maximum 45 kg and minimum 0 kg
%  rp = [xp, yp, zp]' (m) is the location of the payload w.r.t. the CO
%  V_c:     ocean current speed (m/s)
%  beta_c:  ocean current direction (rad)
%
% The coordinate origin CO is chosen midtships on the centerline. The
% draft is T = 13.4 cm when the payload mp = 0. This corresponds to z = 0.
% Choosing mp = 25 kg gives T = 19.5 cm and z = 0.04 m in steady state.
% Adding a non-zero payload mp will change the steady-state z value since
% the payload is added as an external force g_0 in the code.
%
% Example usage:
%
%   [~, ~, M, B_prop] = otter()         : Return the 6x6 mass matrix M and
%                                         the 2x2 input matrix B_prop
%   [xdot, U] = otter(x,n,mp,rp)        : Return xdot and U, no ocean currents
%   xdot = otter(x,n,mp,rp)             : Return xdot, no ocean currents
%   xdot = otter(x,n,mp,rp,V_c,beta_c) : Return xdot, 2-D currents
```

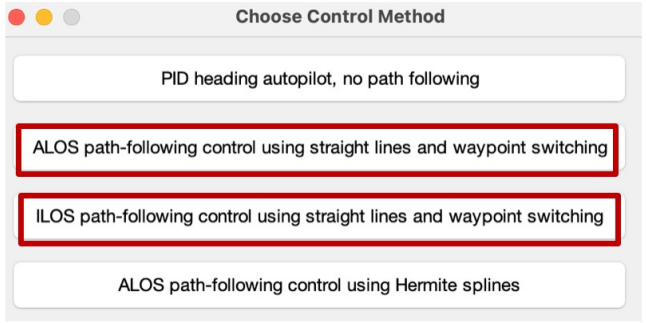
```
% Otter USV input matrix
[~,~,M, B_prop] = otter();
Binv = invQR(B_prop);
```

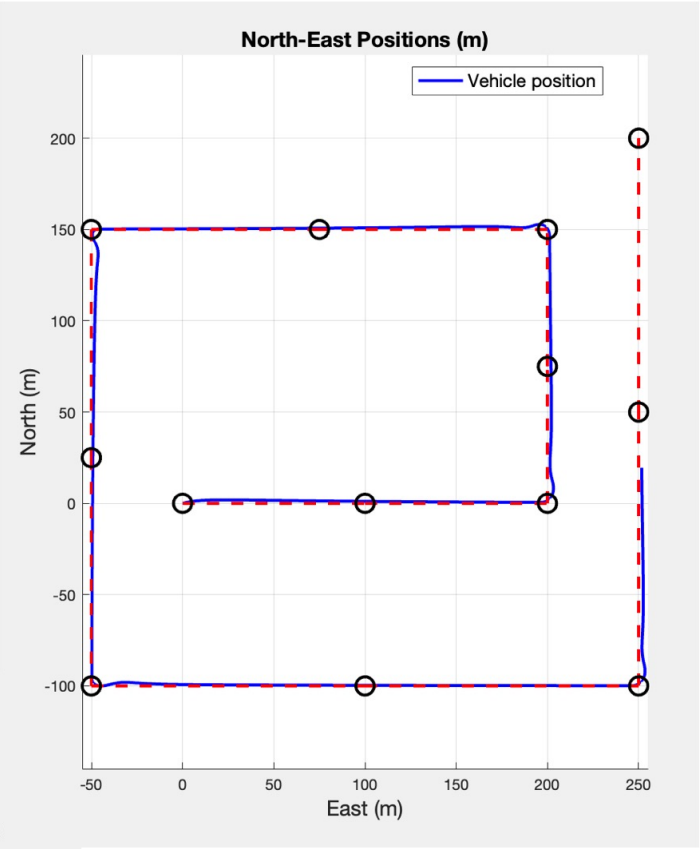
```
% PID heading (yaw moment) autopilot and forward thrust
tau_X = 100;                                % Constant forward thrust
tau_N = (T/K) * a_d + (1/K) * r_d - ...     % Yaw moment control
        Kp * (ssa(psi - psi_d) + ...        % Proportional term
        Td * (r - r_d) + (1/Ti) * z_psi); % Derivative and integral terms

% Control allocation
u = Binv * [tau_X; tau_N];      % Compute control inputs for propellers
n_c = sign(u) .* sqrt(abs(u));  % Convert to required propeller speeds
```

```
% RK4 method x(k+1)
x = rk4(@otter, h, x, n,mp,rp,V_c,beta_c);
```

```
>> SIMotter
------------------------------------------------------------------------
MSS toolbox: Otter USV
ALOS path-following control using straight lines and waypoint switching
Circle of acceptance: R = 5 m
Look-ahead distance: Delta_h = 10 m
------------------------------------------------------------------------
Simulating...
Active waypoints:
  (x1, y1) = (0.00, 0.00)
  (x2, y2) = (0.00, 100.00)
  (x3, y3) = (0.00, 200.00)
  (x4, y4) = (75.00, 200.00)
  (x5, y5) = (150.00, 200.00)
  (x6, y6) = (150.00, 75.00)
  (x7, y7) = (150.00, -50.00)
  (x8, y8) = (25.00, -50.00)
  (x9, y9) = (-100.00, -50.00)
  (x10, y10) = (-100.00, 100.00)
  (x11, y11) = (-100.00, 250.00)
```





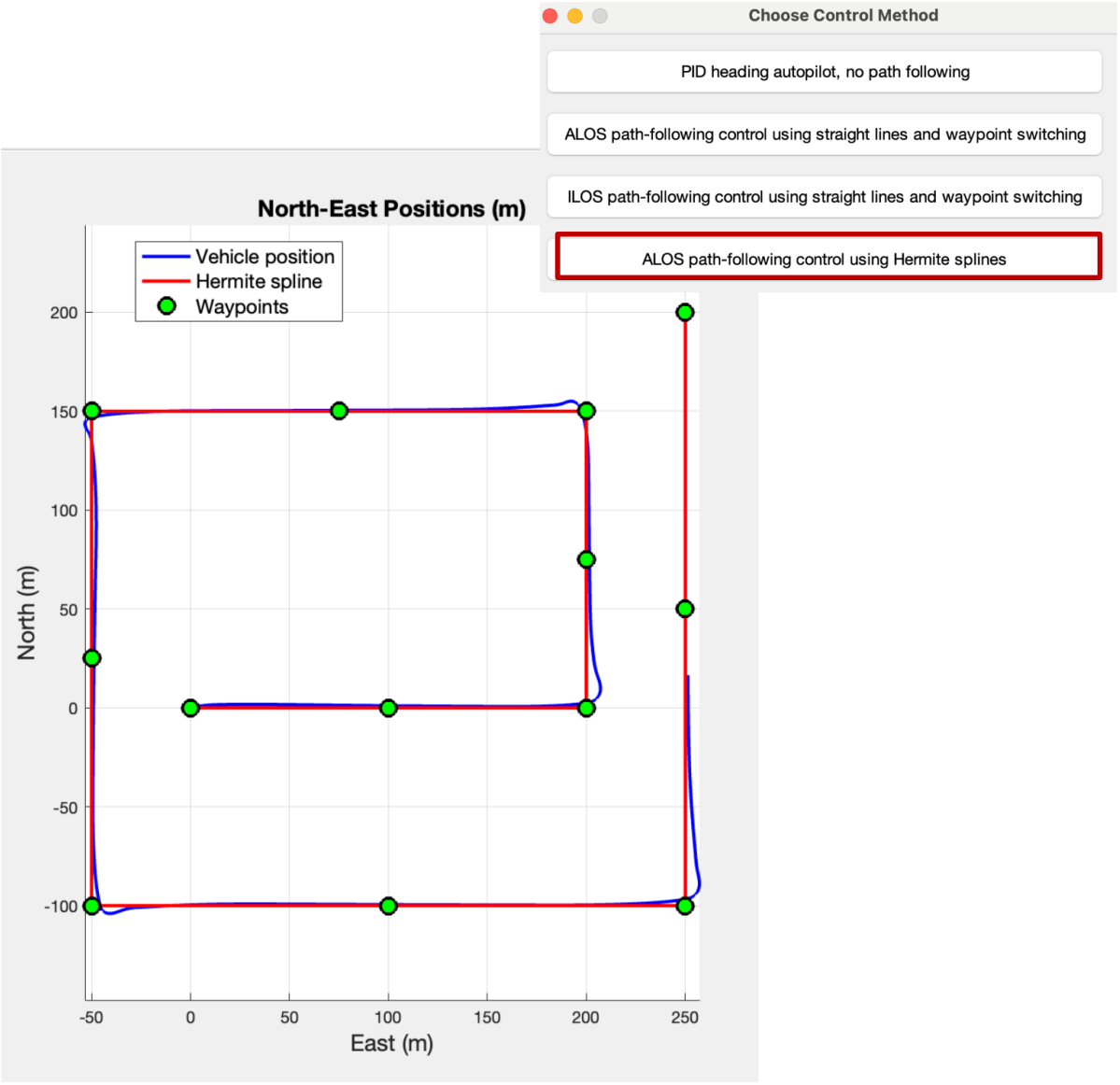

```
>> SIMotter
------------------------------------------------------------------------
MSS toolbox: Otter USV
ILOS path-following control using straight lines and waypoint switching
Circle of acceptance: R =  5 m
Look-ahead distance: Delta_h = 10 m
------------------------------------------------------------------------
Simulating...
Active waypoints:
  (x1, y1) = (0.00, 0.00)
  (x2, y2) = (0.00, 100.00)
  (x3, y3) = (0.00, 200.00)
  (x4, y4) = (75.00, 200.00)
  (x5, y5) = (150.00, 200.00)
  (x6, y6) = (150.00, 75.00)
  (x7, y7) = (150.00, -50.00)
  (x8, y8) = (25.00, -50.00)
  (x9, y9) = (-100.00, -50.00)
  (x10, y10) = (-100.00, 100.00)
  (x11, y11) = (-100.00, 250.00)
```

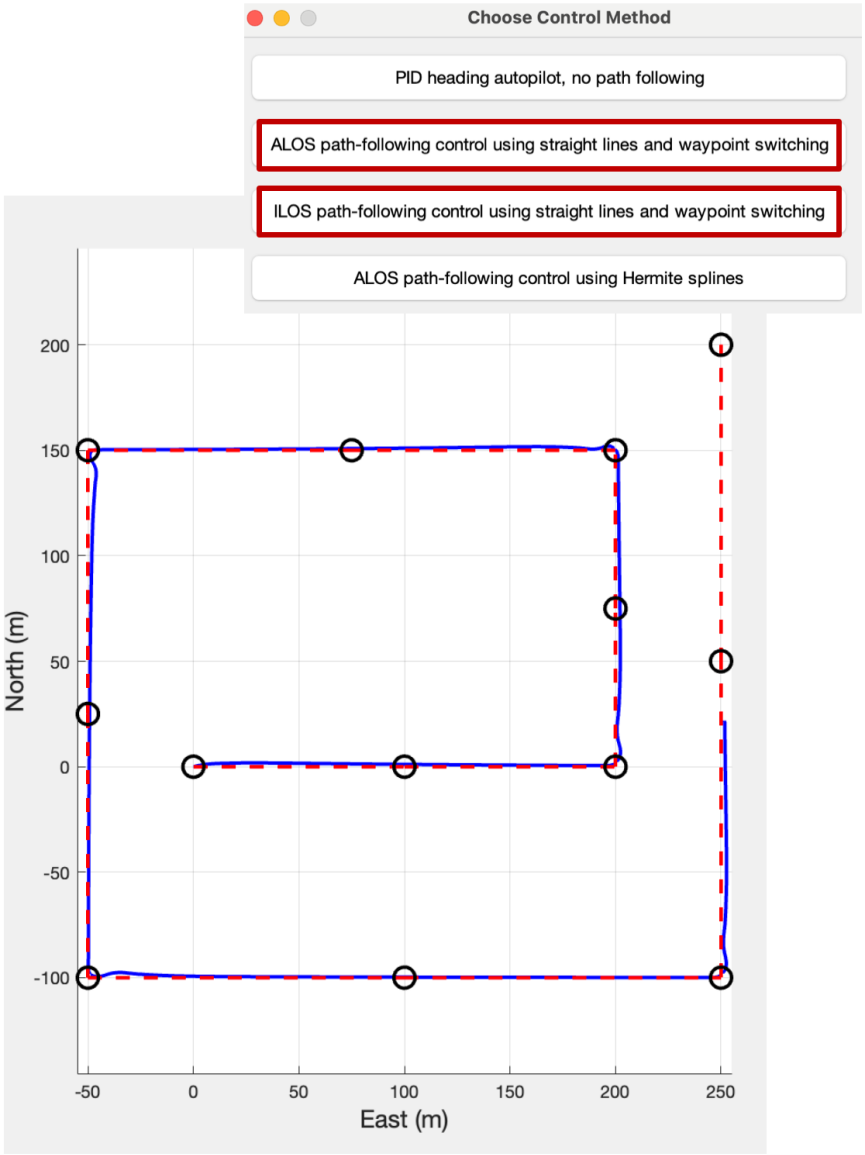
MSS Toolbox
Maritime Robotics Otter USV

Length	: 2.0 m
Beam	: 1.08 m
Draft (no payload)	: 13.4 cm
Draft (25 kg payload)	: 19.5 cm
Mass (no payload)	: 55.0 kg
Max speed	: 3.0 m/s
Max pos. propeller speed	: 993 RPM
Max neg. propeller speed	: -972 RPM



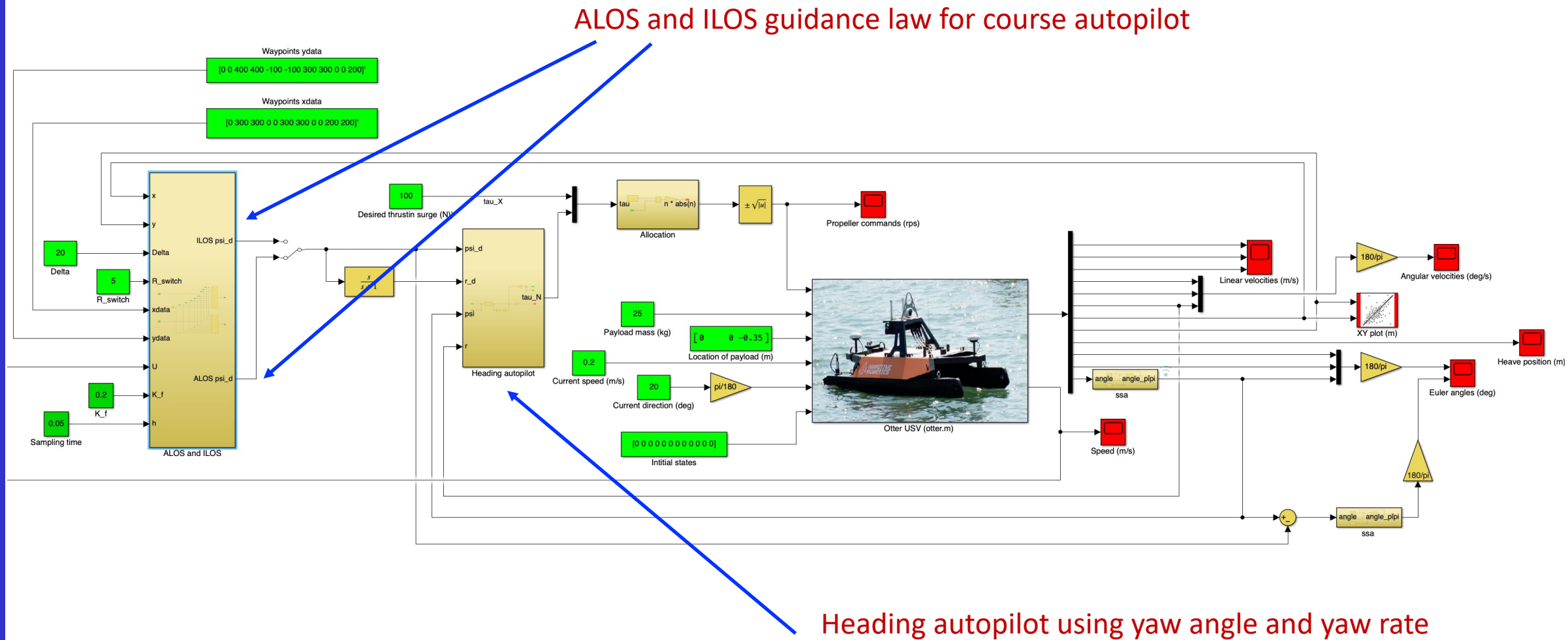
Path Representations





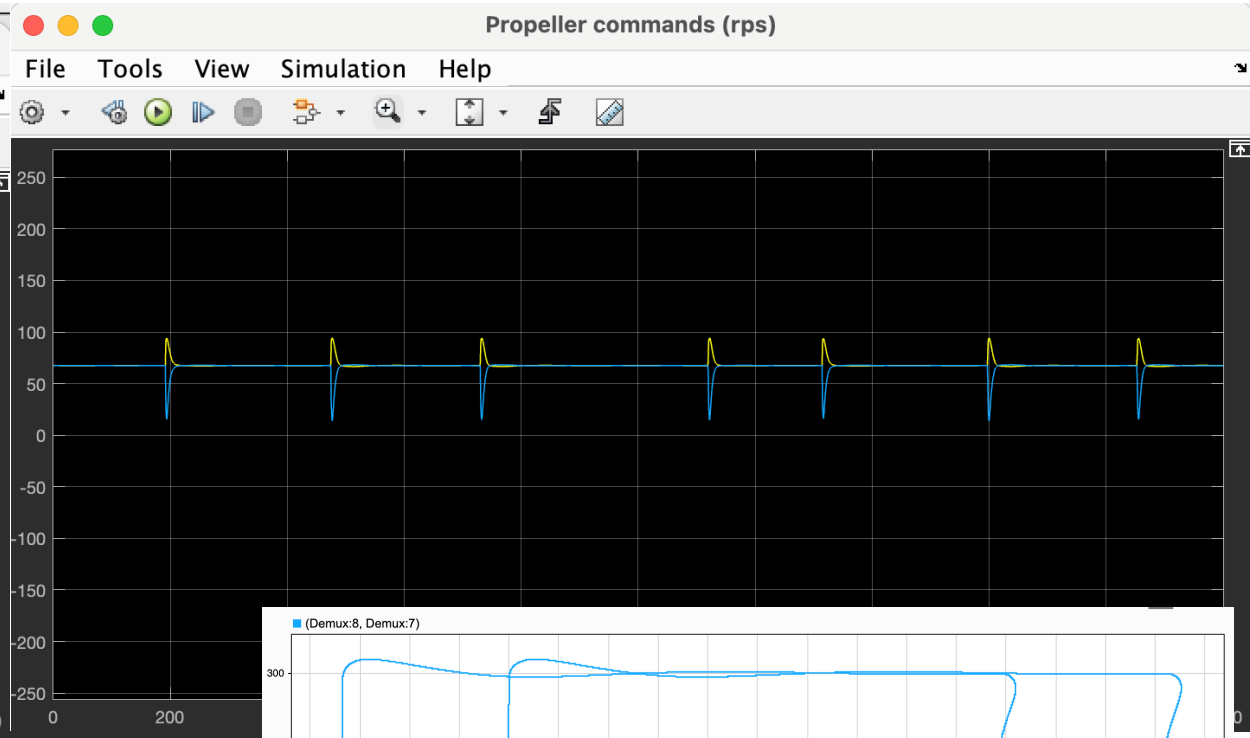
MSS Simulink: demoOtterUSVPathFollowingHeadingControl.slx

Simulink demo calling the MSS Matlab functions ALOSpsi.m and ILOSpsi.m for LOS path-following control using a heading autopilot

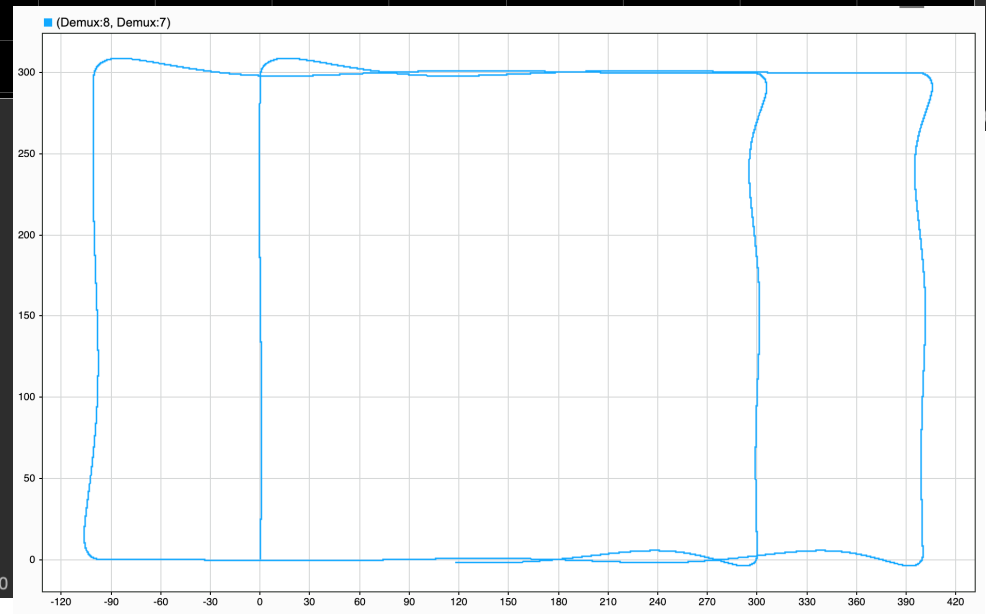


MSS Simulink: demoOtterUSVPathFollowingHeadingControl.slx



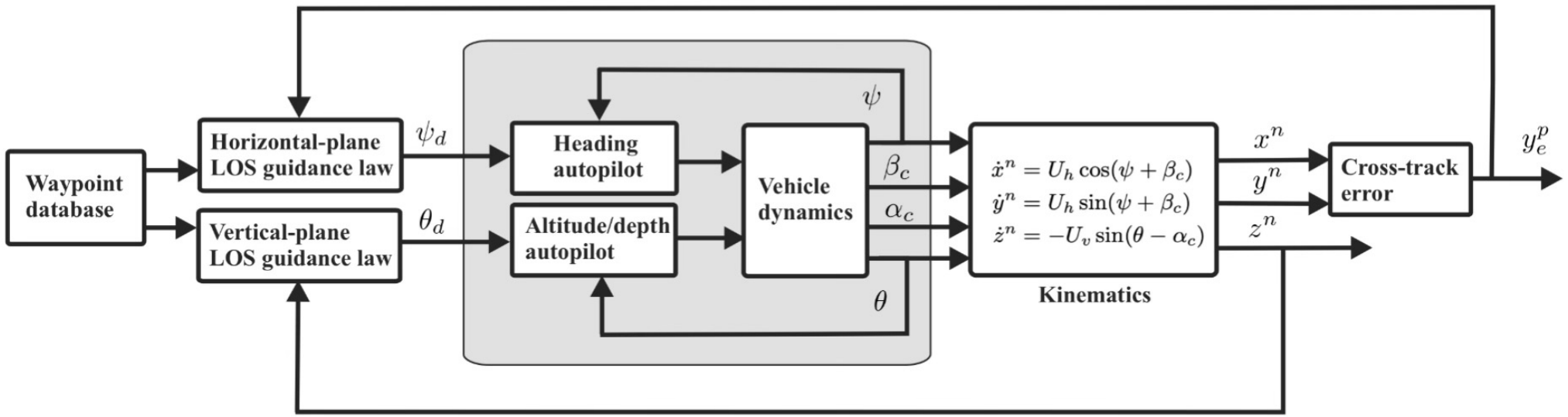






Extensions to 3-D Path-Following Control

Adaptive Line-of-Sight Guidance Laws for 3-D Path Following



Feedback interconnection showing the LOS guidance laws (outer control loops) interacting with two commercial autopilots (inner control loops).

T. I. Fossen and A. P Aguiar (2024). A Uniform Semiglobal Exponential Stable Adaptive Line-of-Sight (ALOS) Guidance Law for 3-D Path Following. *Automatica.*

Compensation of Drift in Heading and Depth Autopilots using the 3-D ALOS Guidance Law for Path Following

Cross-Track Error Expressed in in the Path-Tangential Frame

$$\dot{y}_e^p = U_h \sin(\psi + \beta_c - \pi_h)$$

The crab angle β_c is treated as a disturbance in yaw

The yaw angel ψ is the control input

(We assume that the heading autopilot achieves perfect tracking such that $\psi = \psi_d$)

Vertical-Track Error Expressed in the Path-Tangential frame

$$\dot{z}_e^p = -U \sin(\theta - \alpha_c - \pi_v) + U_h \sin(\pi_v)\left(\cos(\psi + \beta_c - \pi_h) - 1\right)$$

α_c is treated as a disturbance in pitch

The pitch angel θ is the control input

(We assume that the pitch angle autopilot achieves perfect tracking such that $\theta = \theta_d$)

T. I. Fossen and A. P Aguiar (2024). A Uniform Semiglobal Exponential Stable Adaptive Line-of-Sight (ALOS) Guidance Law for 3-D Path Following. *Automatica.*
Coates, E. M. and T. I. Fossen (2025). A Spherical Amplitude-Phase Formulation for 3-D Adaptive Line-of-Sight (ALOS) Guidance with USGES Stability Guarantees. Submitted to Automatica. Open Access arxiv.org/abs/2505.08344

3-D ALOS Guidance Laws ALOS for Compensation of α_c and β_c

$$\psi_d = \pi_h - \hat{\beta}_c - \tan^{-1}\left(\frac{y_e^p}{\Delta_h}\right)$$

$$\dot{\hat{\beta}}_c = \gamma_h \frac{\Delta_h}{\sqrt{\Delta_h^2 + (y_e^p)^2}} y_e^p$$

$$\theta_d = \pi_v + \hat{\alpha}_c + \tan^{-1}\left(\frac{z_e^p}{\Delta_v}\right)$$

$$\dot{\hat{\alpha}}_c = \gamma_v \frac{\Delta_v}{\sqrt{\Delta_v^2 + (z_e^p)^2}} z_e^p$$

- $\Delta_h > 0$ and $\Delta_v > 0$ are the user specified look-ahead distances
- $\gamma_h > 0$ and $\gamma_v > 0$ are the adaptive gains
- $\hat{\beta}_c$ and $\hat{\alpha}_c$ are the parameter estimates of β_c and α_c, respectively

The stability analysis assumes that the heading and altitude/depth autopilots achieve perfect tracking such that $\psi = \psi_d$ and $\theta = \theta_d$.

Robustification of the ALOS Guidance Law by Parameter Projection

$$\dot{\hat{\beta}}_c = \gamma_h \frac{\Delta_h}{\sqrt{\Delta_h^2 + (y_e^p)^2}} \mathrm{Proj}(\hat{\beta}_c, y_e^p)$$

$$\dot{\hat{\alpha}}_c = \gamma_v \frac{\Delta_v}{\sqrt{\Delta_v^2 + (z_e^p)^2}} \mathrm{Proj}(\hat{\alpha}_c, z_e^p)$$

$$\mathrm{Proj}(\hat{\theta}, \tau) = \begin{cases} \left(1 - c(\hat{\theta})\right)\tau & \text{if } |\hat{\theta}| > M_\theta \text{ and } \hat{\theta}^\top \tau > 0 \\ \tau & \text{otherwise} \end{cases}$$

$$c(\hat{\theta}) = \min\{1, (\hat{\theta}^2 - M_\theta^2)/(M_{\hat{\theta}}^2 - M_\theta^2)\}$$

This restricts the parameter estimates to a compact set $|\hat{\beta}_c| \leq M_{\hat{\theta}}$ and $|\hat{\alpha}_c| \leq M_{\hat{\theta}}$ where $M_{\hat{\theta}} = M_\theta + \epsilon$, and $\epsilon > 0$ is a positive constant chosen such that $M_{\hat{\theta}}$ is slighter larger than M_θ where $|\beta_c| \leq M_\theta$ and $|\alpha_c| \leq M_\theta$.

T. I. Fossen and A. P Aguiar (2024). A Uniform Semiglobal Exponential Stable Adaptive Line-of-Sight (ALOS) Guidance Law for 3-D Path Following. *Automatica.*

Remus 100 AUV Case Study



The Remus 100 at the Applied Underwater Robotics Laboratory (AUR-Lab) at NTNU.

The 6-DOF mathematical model of the Remus 100 AUV is available in the MATLAB MSS toolbox (Fossen and Perez, 2004). The script *remus100.m* describes an AUV of length 1.6 m, diameter of 19 cm and mass 31.9 kg.

The vehicle's maximum speed, 2.5 m/s, is obtained by running the propeller at 1525 rpm when there are no ocean currents. Depth is controlled by using the stern plane δ_s, while the tail rudder δ_r controls the yaw angle.

The heading and depth autopilots are implemented as PID controllers

$$\delta_s = -k_{p_\theta}\,\mathrm{ssa}(\tilde{\theta}) - k_{d_\theta}\dot{\theta} - k_{i_\theta}\int_0^t \mathrm{ssa}(\tilde{\theta})\mathrm{d}\tau$$

$$\delta_r = -k_{p_\psi}\,\mathrm{ssa}(\tilde{\psi}) - k_{d_\psi}\dot{\psi} - k_{i_\psi}\int_0^t \mathrm{ssa}(\tilde{\psi})\mathrm{d}\tau$$

Remus 100 AUV Case Study

Control Methods

- ○ PID pole-placement control
- ○ Integral sliding mode control (SMC)
- ◉ ALOS guidance law for 3-D path following

Kinematics Representation

- ○ Euler angles
- ◉ Unit quaternions

OK

```
>> SIMremus100
------------------------------------------------------------------
MSS toolbox: Remus 100 AUV
Unit quaternion representation (13 states)
Heading autopilot: Integral sliding mode control (SMC)
Depth autopilot:   Successive-loop closure
Path-following:    ALOS guidance law for 3-D path following
------------------------------------------------------------------
Simulating...
Active waypoints:
  (x1,y1,z1) = (0.0,0.0,0.0)
  (x2,y2,z2) = (-20.0,200.0,10.0)
  (x3,y3,z3) = (-100.0,600.0,100.0)
  (x4,y4,z4) = (0.0,950.0,100.0)
  (x5,y5,z5) = (200.0,1300.0,50.0)
  (x6,y6,z6) = (200.0,1800.0,50.0)
  (x7,y7,z7) = (400.0,2200.0,50.0)
```

```
% Heading autopilot using the tail rudder (integral SMC)
delta_r = integralSMCheading(psi, r, psi_d, r_d, a_d, ...
    K_d, K_sigma, 1, phi_b, K_yaw, T_yaw, h);

% Depth autopilot using the stern planes (PID)
delta_s = -Kp_theta * ssa( theta - theta_d )...
    - Kd_theta * q - Ki_theta * theta_int - K_w * w;

% ALOS guidance law
[psi_ref, theta_ref, y_e, z_e, alpha_c_hat, beta_c_hat] = ...
    ALOS3D(xn, yn, zn, Delta_h, Delta_v, gamma_h, gamma_v,...
    M_theta, h, R_switch, wpt);

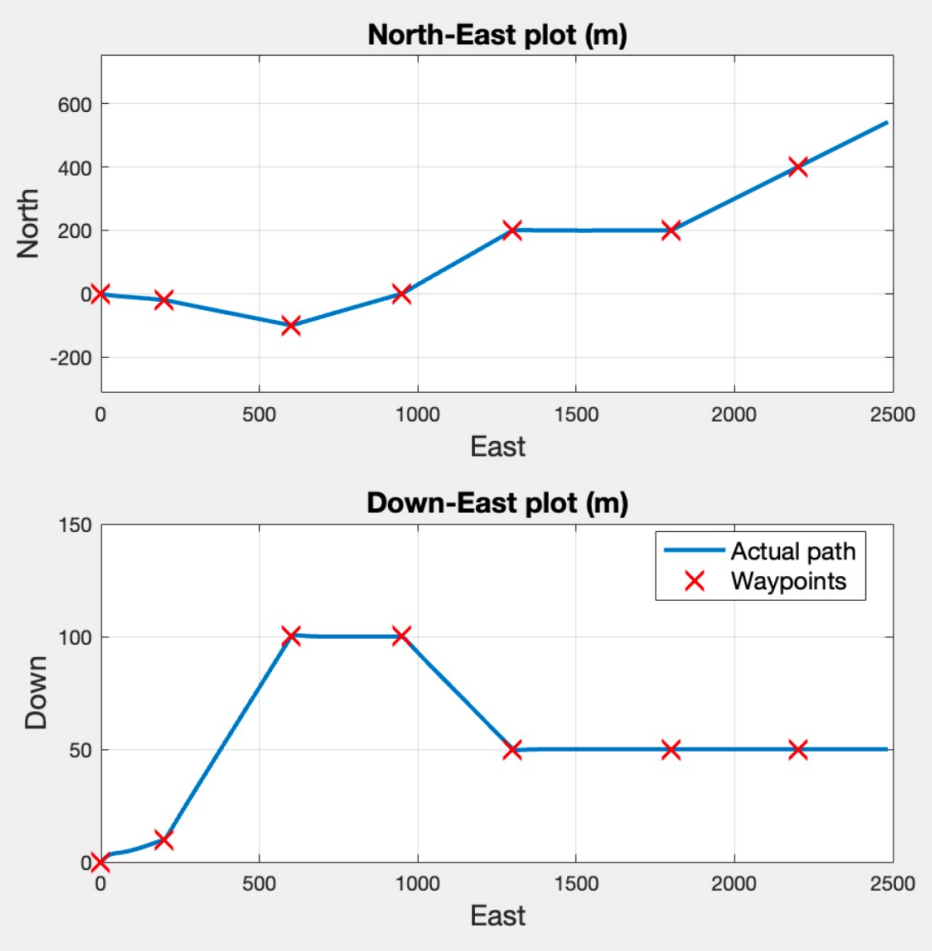
% ALOS observer
[theta_d, q_d] = LOSobserver(theta_d, q_d, theta_ref, h, K_f);
[psi_d, r_d] = LOSobserver(psi_d, r_d, psi_ref, h, K_f);
if abs(r_d) > r_max, r_d = sign(r_d) * r_max; end
```

MSS Toolbox
Remus100 AUV

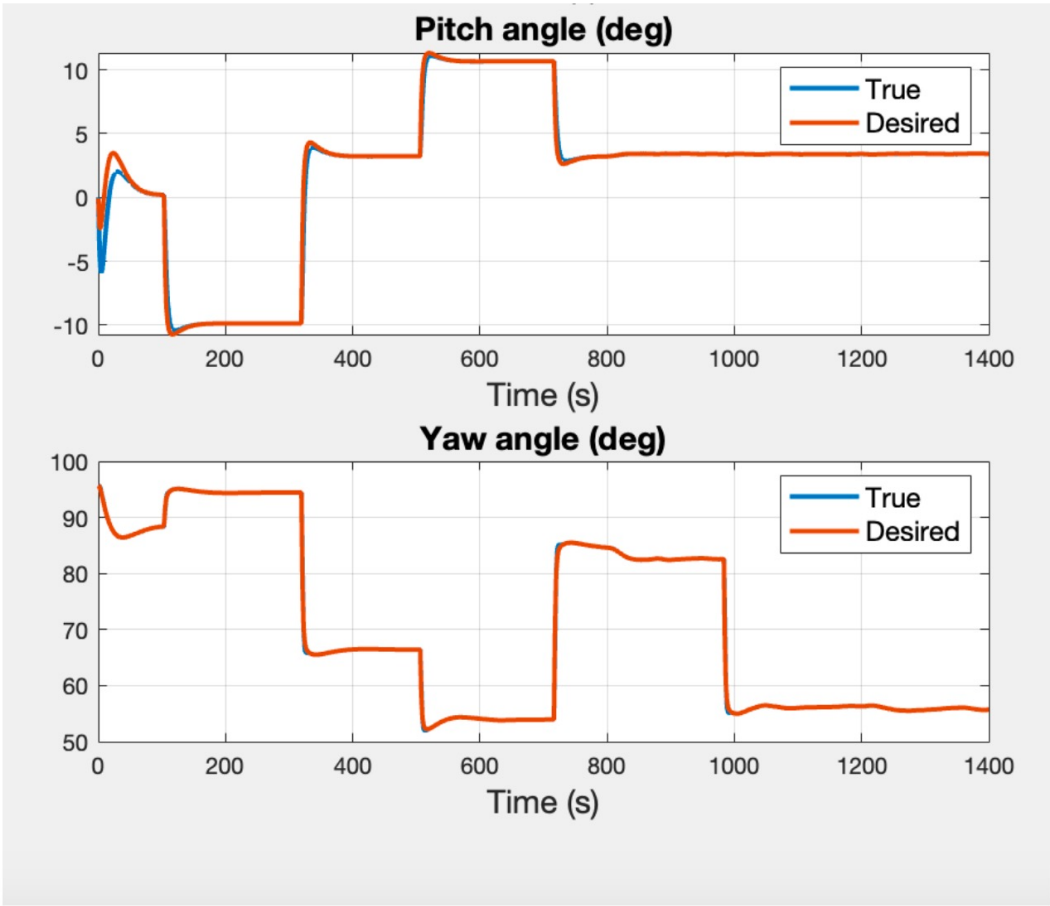
Length	: 1.6 m
Diameter	: 19 cm
Mass	: 31.9 kg
Max speed	: 2.5 m/s
Max propeller speed	: 1525 RPM



AUV Case Study, cont.

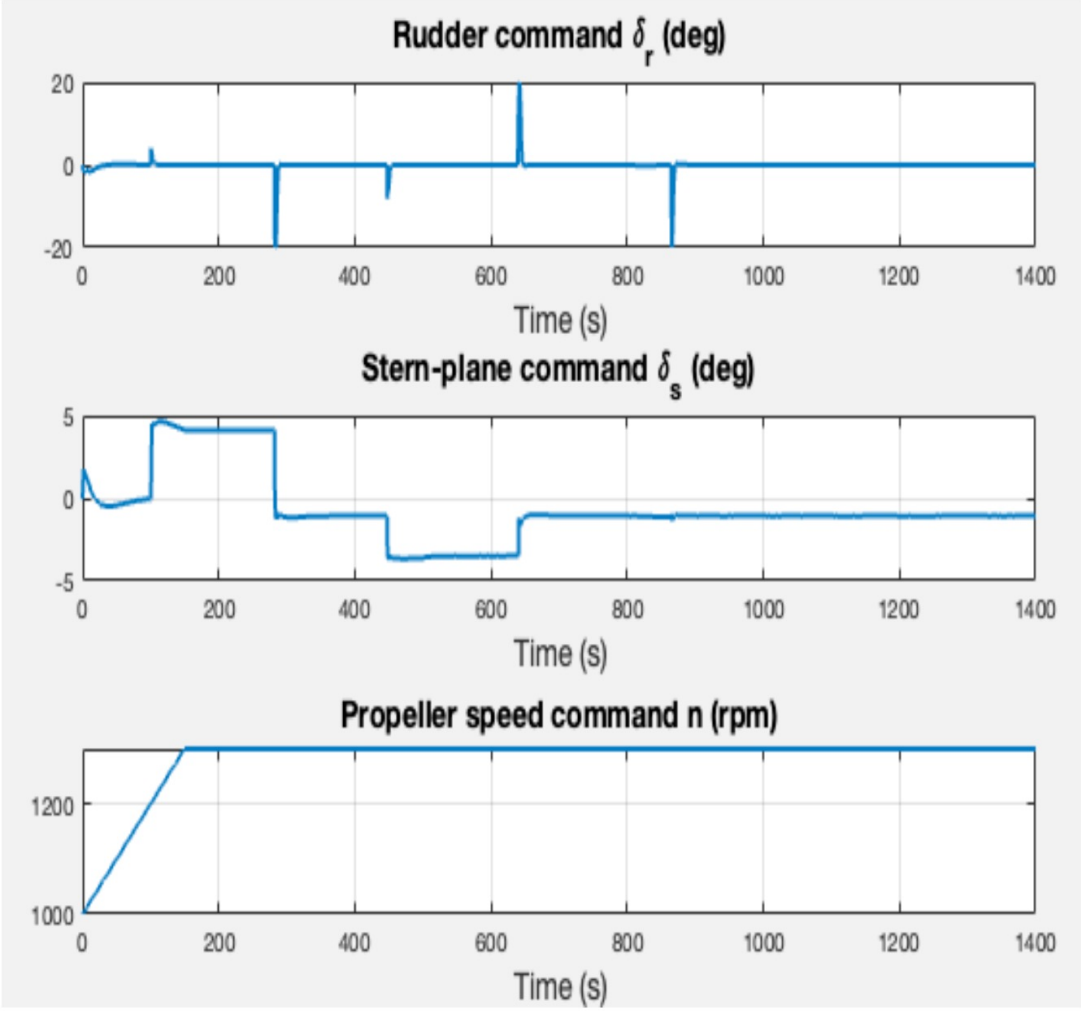


Desired path and waypoints in the horizontal and vertical planes. The maximum depth of the vehicle is 100 m.

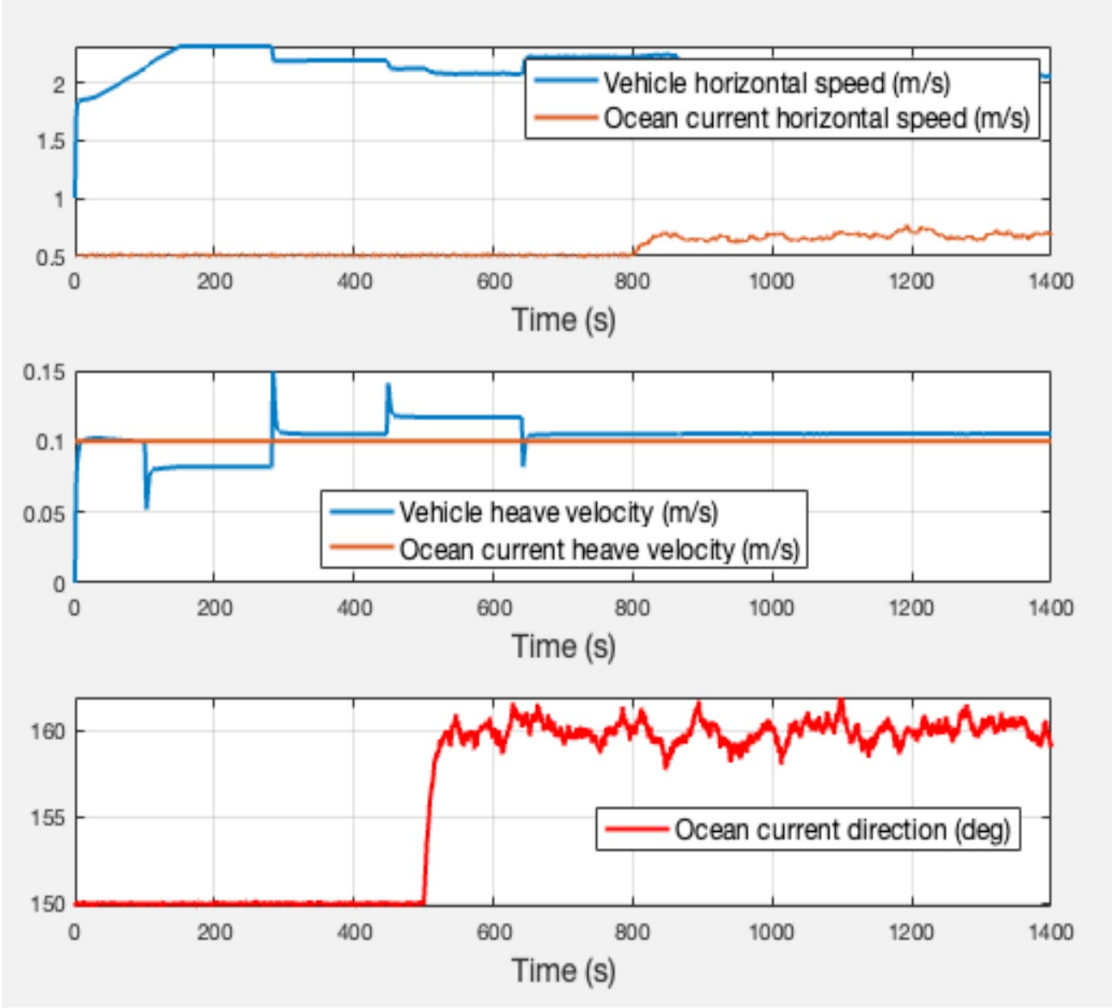


ALOS desired pitch and yaw angles together with their true values obtained by using the depth and heading autopilots.

AUV Case Study, cont.



Rudder angle, stern angle, and propeller revolutions versus time.



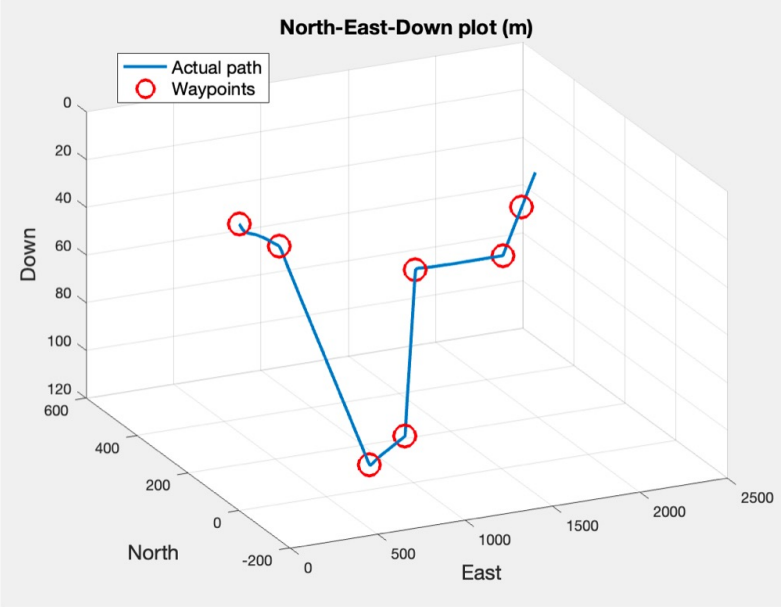
Vehicle surge and sway velocities, speed and ocean current speed/direction in the horizontal plane versus time.

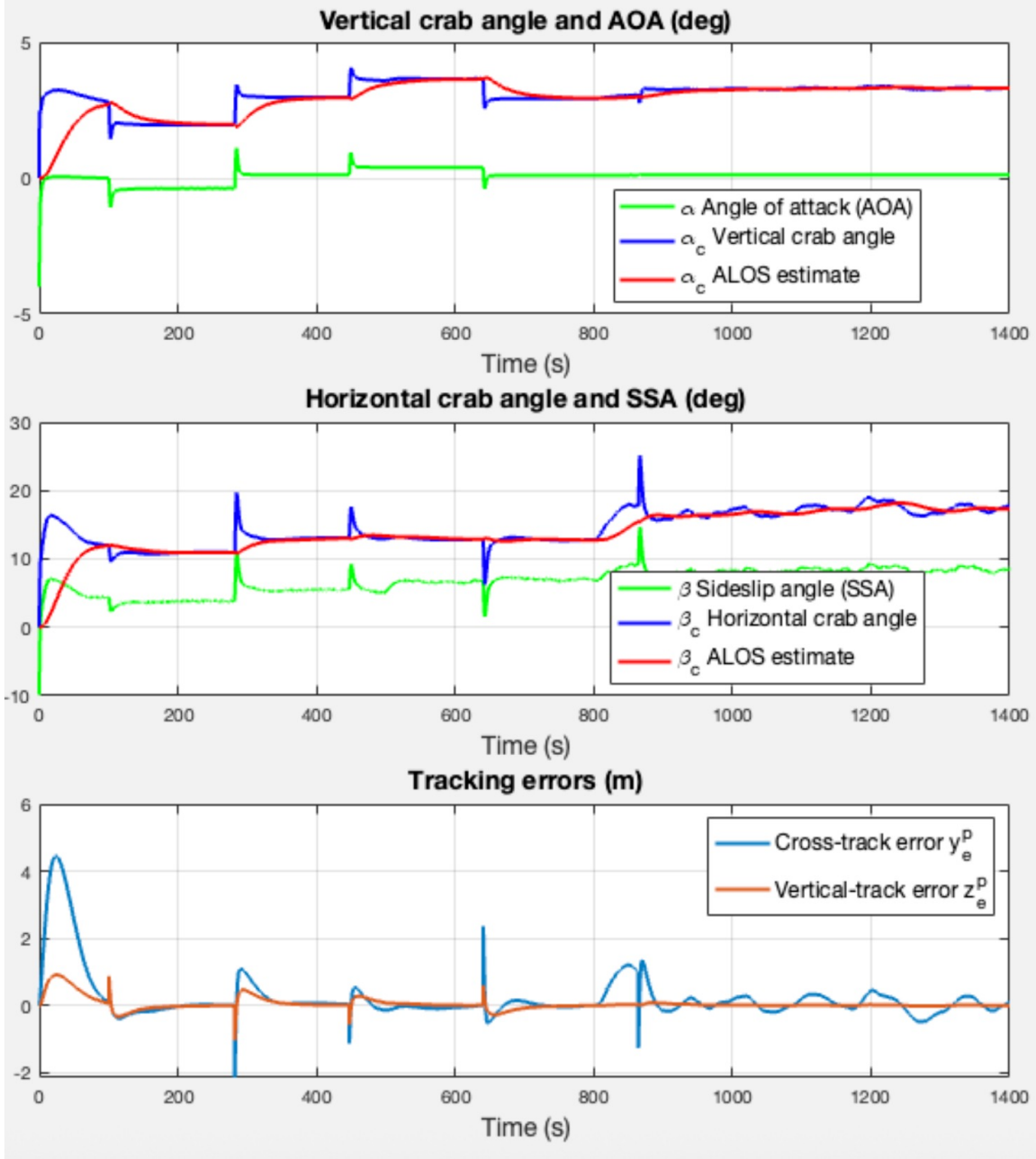
AUV Case Study, cont.

The parameter estimates $\hat{\alpha}_c$ and $\hat{\beta}_c$ track the true parameters α_c and β_c with excellent accuracy, also when the current speed and direction change rapidly.

Note the angle of attack α and β are different from α_c and β_c, and that the vehicle will sideslip with an angle $\beta_c \approx 8.0$ degrees after 1400 s.

The accuracy of the ALOS guidance law is demonstrated by showing the convergence of y_e^p and z_e^p to zero. The initial transient is caused by a 90 degrees initial heading error, while the jumps in the tracking errors are due to waypoint switching.





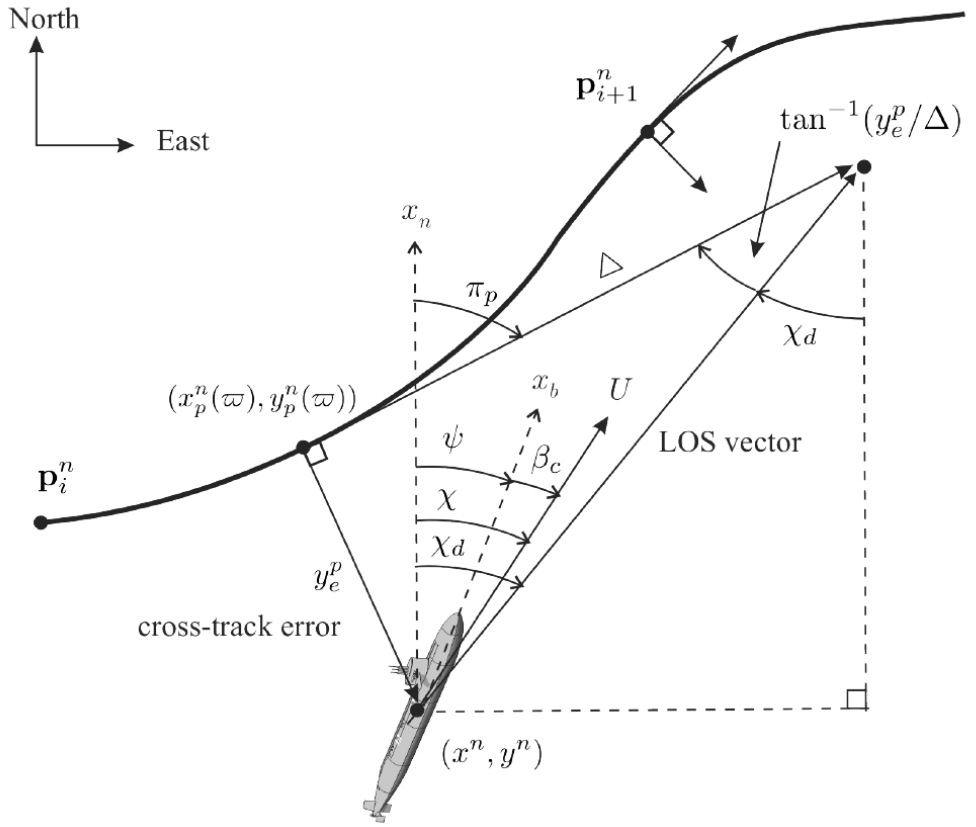
Curved-Path Path Following

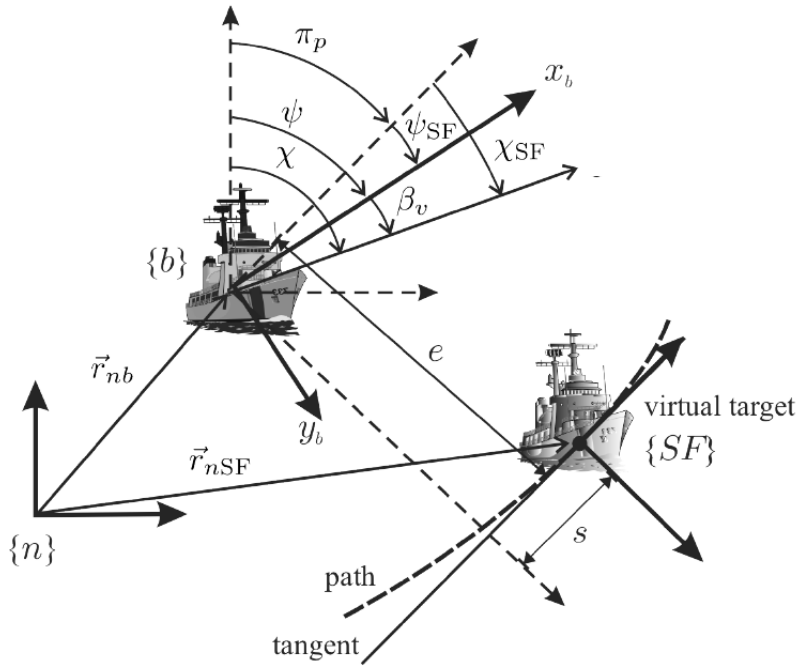
12.6 Curved-Path Path Following

A guidance systems can make use of a predefined parametrized path for which the curvature is known. Methods for this are described in Fossen (2021, Ch. 12.6):

12.6.1 Path generation using interpolation methods
12.6.2 Proportional LOS guidance law for curved paths
12.6.3 Path-following using Serret-Frenet coordinates
12.6.4 Case study: Path-following control using *Serret-Frenet* coordinates

In many cases this is not practical and a simpler path consisting of waypoints and straight lines will be the preferred solution, see Sections 12.3-12.5.





Chapter Goals – Revisited

Guidance systems:

- Be able to design open-loop guidance systems using reference models (low-pass filters) for position and velocity control.
- Understand how to design a closed-loop guidance system using feedback control theory.

Guidance Laws:

- Be able to design guidance laws for target tracking, that is a moving object for which no future motion information, nor the path is known in advance.
- Be able to design a state-of-the-art linear path-following controller. This is the classical design method where the path is generated using waypoints to connect straight lines between the waypoints.
- Be able to design LOS guidance laws for path following using course autopilots.
- Be able to design LOS guidance laws for path following using heading autopilots, with extension to ILOS to compensate for sideslip and unmodelled dynamics.